> **Complete Guidebook Edition.** This guide reflects the EnvoyMesh 0.2.2 repository state at the revision date. It is written for end users, not as a content-outline stub. Feature status can differ by platform and deployment—verify each Beta or Experimental capability in your build (release notes, Settings labels, and Appendix J) before relying on it in production.

How to read this guide

- **Parts I–XIV** explain the product for end users and operators.
- **Part XV** is for website editors and content operators and is optional for end users.
- Task lifecycle names such as *Created* / *Task planned* / *Running* are EnvoyMesh states, not product **Planned** / **Available** status labels.
- **Mobile** in this guide means **EnvoyGo** (thin client paired to a home node), unless a section explicitly discusses legacy mobile experiments.

Feature status labels

- **Available** — implemented and intended for current use.
- **Beta** — implemented, but still receiving validation or product polish.
- **Experimental** — usable for evaluation; behavior or interfaces may change.
- **Compatibility preset** — EnvoyMesh includes configuration for the integration, while part of the integration is maintained by another project.
- **Planned** — designed or documented, but not currently available as a complete product feature.
- **Parked** — intentionally deferred without a committed release date.
- **Desktop** — available through the EnvoyMesh desktop application or home node.
- **Mobile** — available in EnvoyGo, the current EnvoyMesh mobile product (home-paired thin client).
- **Operator** — intended for node, relay, or fleet administrators.

Product terminology used in this guide

- **EnvoyAI / OpenClaw** is the richer bundled agent integration included with EnvoyMesh.
- **HomeClaw** and **Hermes** are built-in external-agent compatibility presets.
- **OpenHuman** is a built-in compatibility preset that is disabled by default.
- Agent-side code for HomeClaw, Hermes, and OpenHuman is maintained by their respective projects; EnvoyMesh provides the bridge, presets, policy boundary, and mesh tools.
- **Agent Network** means bonded people allowing their opted-in local agents to collaborate. It is not a public agent marketplace.
- **Team jobs** is the user-facing name for multi-agent collaboration. Source code and older documentation may call these workflows **chains**.
- **EnvoyGo** is the current mobile product: a thin client that pairs to a home EnvoyMesh node. The earlier Capacitor mobile tree (in-process full node) is a legacy experiment and is not the primary mobile application. Running EnvoyGo itself as a full mesh node is parked (Appendix J.6).

Table of Contents

Part I — Discover EnvoyMesh

1. Welcome to EnvoyMesh

1.1 A private network for people and AI agents

EnvoyMesh connects people and AI agents through a private mesh rather than a central account service. Each participant keeps a local identity, chooses trusted contacts (bonded at one of four user-selectable trust tiers — blocked, public, referred, or direct; `self` is the implicit tier for your own owner, devices, and agents), and decides which agents, tools, and information may cross those relationships.

1.2 Local-first and peer-to-peer by design

The home node stores identity, policy, conversations, tasks, and knowledge locally. Peer-to-peer transport is preferred, so routine communication does not depend on a hosted application database.

1.3 No central account required

You create cryptographic identities instead of registering a global username and password. Public relays may help peers find and reach each other, but they are not an account authority.

1.4 Your identity, relationships, and data belong to you

Owner keys establish control, bonds record relationships, and sensitivity labels protect data. Backups therefore matter: losing the only copy of an owner key can mean losing continuity of that identity.

1.5 Direct connections and optional relays

EnvoyMesh first attempts a direct peer path. When NAT, firewalls, or mobility prevent that path, an optional relay supplies rendezvous and forwarding without becoming the application brain.

1.6 Personal agents and external agents

EnvoyAI is the bundled OpenClaw-based assistant. A separate bridge can connect HomeClaw, Hermes, OpenHuman, or a custom HTTP agent without giving that external process raw P2P keys.

1.7 Trusted multi-agent collaboration

Agent Network lets bonded owners opt their local agents into Team jobs. The requesting node plans work, eligible workers execute locally, and the orchestrator combines attributed results.

1.8 Open protocols and interoperability

Native signed EnvoyMesh envelopes remain the internal protocol. MCP exposes tools to compatible applications, while A2A publishes agent discovery and task interfaces at the network edge.

1.9 Major features at a glance

Available areas include messaging, groups, audio, voice calls, files, profiles, personal AI, knowledge and RAG, external-agent bridges, Team jobs, terminals, Browser, relays, MCP, and A2A.

1.10 Current availability and limitations

Some capabilities remain platform-specific or deferred. In particular, video calls, broad anonymous worker recruitment, full-node EnvoyGo operation, global reputation, commerce, Filecoin persistence, and a complete hierarchical relay graph are not current general features.

2. Why EnvoyMesh?

2.1 Private communication without a central platform

EnvoyMesh treats messaging as signed peer traffic rather than rows in a hosted database. You choose who appears in your contact list, and conversations stay on devices you control unless you explicitly share outward. This differs from centralized messengers that can change terms, scan content, or freeze accounts without your keys.

2.2 Self-sovereign identity across your devices

Your owner identity is an Ed25519 keypair, not a username registered by a vendor. Devices and agents derive from that owner with signed certificates and mandates, so you can prove continuity across laptops, desktops, and paired phones. Losing the only copy of an owner key can end that identity's history, so backup and recovery planning matter from day one.

2.3 AI assistance under your control

EnvoyAI and external agents run on your home node under bond policy, mandate limits, and optional human approvals. You decide which models, tools, and contacts an agent may use instead of accepting a vendor's default automation scope. Remote model providers receive only prompts the node approves after its semantic firewall and policy checks.

2.4 Trusted knowledge sharing

Notes and files live in your Vault, appear in the Library UI, and can be shared with sensitivity labels that the Bonds engine enforces. Bonded contacts can query your public or friends-tier material through `knowledge.query`, while strangers see only the public sub-graph and are rate-limited. Publishing for browsing uses separate web-content paths and visibility rules described in Part V.

2.5 Safe task delegation

Task delegation uses owner-signed mandates that cap cost, sensitivity, allowed actions, and expiry. An agent cannot silently exceed those bounds; risky steps can require explicit approval before execution. This makes autonomous work legible rather than a black box running on someone else's servers.

2.6 Collaboration among agents you choose

Agent Network is opt-in collaboration among bonded owners, not an anonymous worker marketplace. Team jobs let your local agent plan work and call workers you already trust, with attributed results returned to the orchestrator. You stay in control of which contacts' agents may participate.

2.7 Local models, remote models, and external agents

EnvoyMesh supports local inference, configured remote providers, and external HTTP agents such as HomeClaw or Hermes through a single bridge at a time. The node signs mesh traffic on the agent's behalf without handing over Ed25519 keys. Mix providers to balance privacy, latency, and capability without locking into one vendor stack.

2.8 Auditability instead of invisible automation

Operations append JSONL audit events with correlation IDs that stitch multi-step flows together. You can review what an agent attempted, what policy allowed or denied, and which peer participated. This audit trail complements chat history when diagnosing automation or sharing disputes.

2.9 When EnvoyMesh is the right choice

EnvoyMesh fits when you want cryptographic identity, explicit trust tiers, local-first storage, and agent tooling under your policy. It works well for small trusted groups, personal AI with mesh reach, and teams that need verifiable messaging plus delegated tasks. Start with one home node and a few bonded contacts before expanding relays or Agent Network membership.

2.10 When another solution may be a better fit

A global consumer messenger with effortless signup, massive groups, and vendor-managed moderation may serve you better than running a home node. Likewise, if you only need a single cloud chatbot with no peer relationships or local vault, a hosted assistant is simpler. EnvoyMesh rewards operators willing to own keys, backups, and trust decisions.

3. What You Can Do

3.1 Connect with trusted people

Add contacts through introductions, QR pairing, or relay-assisted discovery once you verify their public key fingerprint. Bonds record trust tiers—blocked, public, referred, or direct—that gate what each peer may request. You can upgrade or downgrade trust as relationships change without migrating to a new account.

3.2 Exchange private messages

Send one-to-one chat as signed envelopes with human-to-human role policy enforced by the protocol. Messages prefer direct libp2p paths and fall back to circuit relay when NAT blocks a straight connection. Read receipts and delivery behavior follow the settings in Social or EnvoyGo once paired to your home node.

3.3 Create group conversations

Create group threads that include multiple bonded contacts with the same signature and policy guarantees as direct chat. Group membership and naming are local-first constructs coordinated through your node. Use groups for family, project, or research circles where everyone already shares an explicit trust relationship.

3.4 Send audio messages and make voice calls

Record short audio clips in chat or start voice calls when both sides support the feature and policy allows. Media flows over the same mesh transport as messages rather than through a separate proprietary calling backend. Quality and availability depend on network paths and whether peers are reachable via direct or relayed connections.

3.5 Share files and profile photos

Share files with contacts using signed data-transfer vouchers that land in vault inbox folders on the recipient side. Profile photos and avatars follow the same identity and storage model as other local assets. Recipients index received files under their own sensitivity rules.

3.6 Talk to your personal AI agent

Chat with EnvoyAI (bundled OpenClaw) from Social desktop or through EnvoyGo when paired to a running home node. The assistant can search your vault, message bonded contacts, and invoke allowed tools subject to mandates and approvals. Enable or disable the bundled agent in Settings → AI according to your comfort with automation.

3.7 Connect OpenClaw, HomeClaw, Hermes, or OpenHuman

Connect HomeClaw, Hermes, OpenHuman, or a custom HTTP agent through Settings → AI → Ext Agent when you prefer an external runtime over bundled EnvoyAI. EnvoyMesh translates mesh tools into the external agent's message contract without exposing raw libp2p keys. Only one external bridge runs at a time; verify you trust the local endpoint before enabling it.

3.8 Search local and trusted knowledge

Search your vault locally from the Library tab or ask EnvoyAI to retrieve chunks through the RAG pipeline indexed on save. Federated search can query bonded contacts' syndicated knowledge within sensitivity ceilings you configure per contact. Public notes participate in the wider mesh through rate-limited `knowledge.query` for strangers.

3.9 Publish and browse mesh content

Publish Markdown, images, and PDFs under `envoy://` URLs served from your home node's web content directory. Bonded contacts—and, when visibility allows, wider mesh peers—open pages in the Social Browser or EnvoyGo Browser while paired to home. Pull-based `library.read` fetches bytes on demand; push notifications for feeds arrived in Phase 45E.

3.10 Delegate work to another agent

Send a task mandate to another owner's agent when you need specialized work within signed bounds. Negotiation follows the task lifecycle from propose through accept, running, and result. Human approval gates remain available for actions the mandate marks as sensitive.

3.11 Run Team jobs across several agents

Run Team jobs (multi-agent chains) across opted-in Agent Network members when bonded owners allow their agents to collaborate. The requesting node plans steps, workers execute locally on their own hardware, and results return with attribution. This is suited to research summaries, split analysis, or coordinated reports—not open recruitment of anonymous workers.

3.12 Connect MCP and A2A applications

Expose selected mesh tools to MCP-compatible desktop apps such as Claude Desktop, or publish an A2A agent card for external task clients. MCP and A2A sit at the network edge; native signed envelopes remain the internal protocol. Configure bridges only after you understand which tools cross the boundary.

3.13 Use terminals remotely

Open browser-based terminals in Social or EnvoyGo that attach to PTY sessions on your home node over WebSocket when you are paired or on desktop. Remote shell access inherits the same authentication and pairing model as other home RPC features. Treat terminal exposure as high privilege and restrict it to devices you control.

3.14 Operate a private or community relay

Run a private relay for your fleet or bootstrap against the community relay for casual testing. Relays provide rendezvous and circuit forwarding—they do not store your messages, run models, or act as account servers. Operators advertise listen addresses and may configure hierarchical relay graphs for larger deployments.

4. How EnvoyMesh Works

4.1 A plain-language system overview

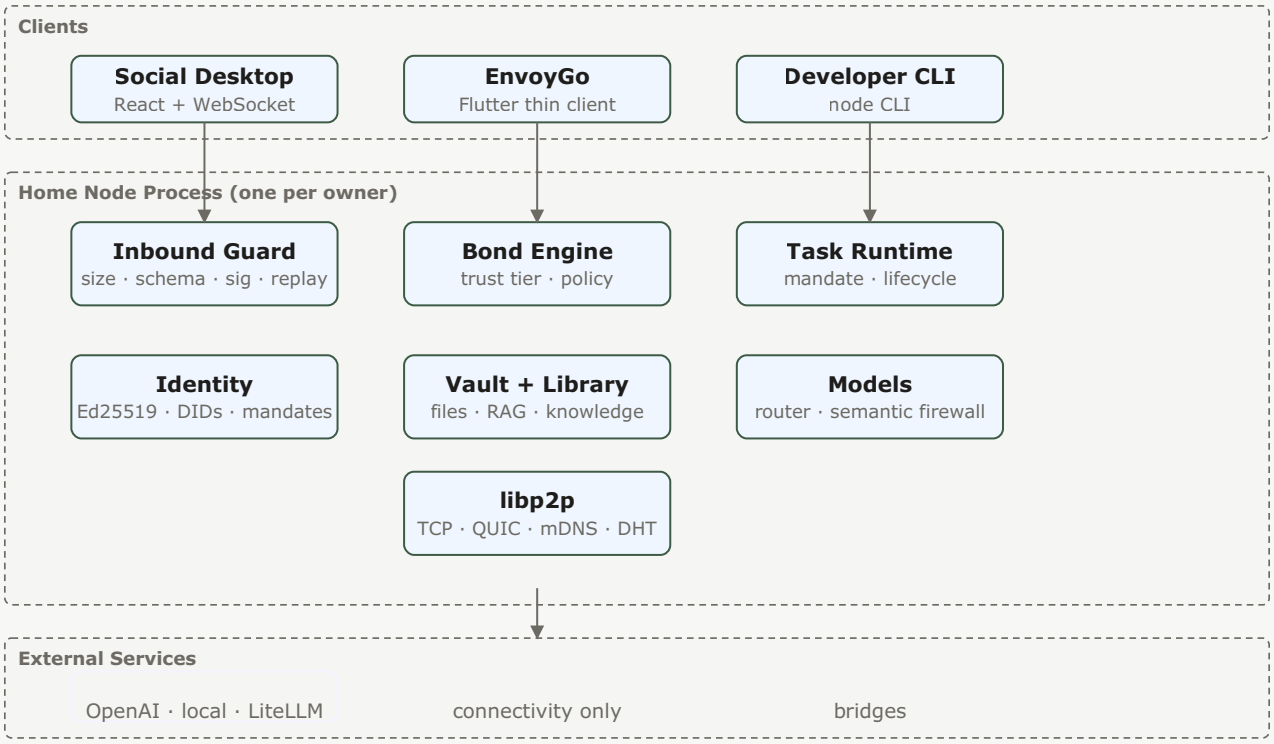

Figure 1 — Home-node system architecture: clients call JSON-RPC into one home node per owner; the home node owns identity, policy, storage, models, and networking; external services are optional and never hold owner keys.

At a high level, your home node combines identity, policy, storage, models, and libp2p networking in one process. Social desktop and paired EnvoyGo are thin clients that call JSON-RPC on that node. Inbound traffic passes through guards for size, signature, replay, and bond decisions before any model or vault access occurs.

4.2 Owners, devices, agents, and peers

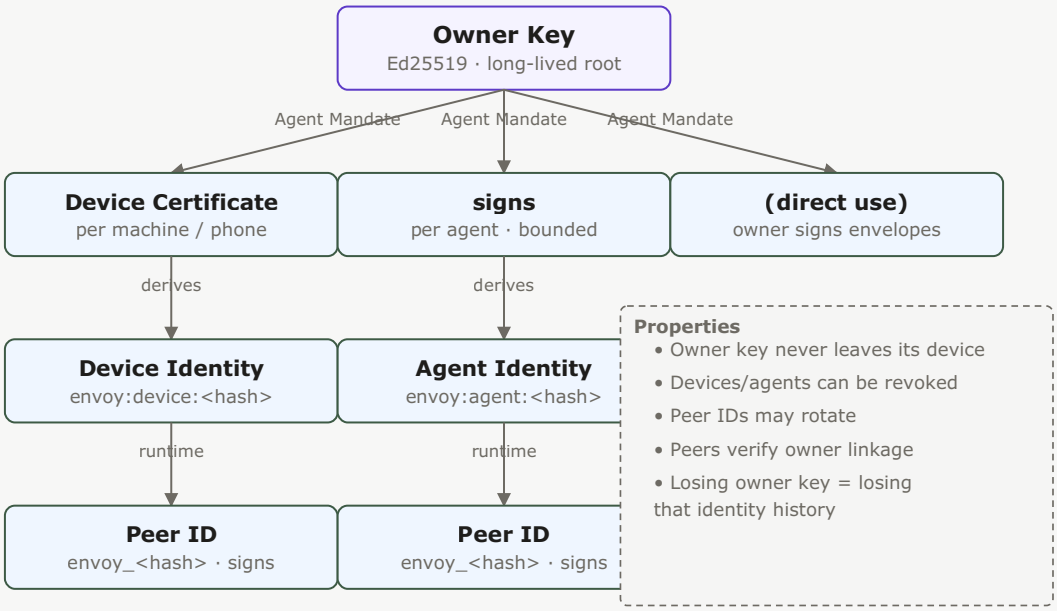

Figure 2 — Identity hierarchy: the owner key is the root; it signs device certificates and agent mandates, each deriving a device/agent identity and a runtime peer ID that signs envelope traffic.

The owner key is the long-lived human root; devices receive owner-signed certificates; agents receive mandates linking them to that owner. Runtime peer IDs sign individual envelopes and may rotate with keys while preserving trust links. Understanding this stack helps you reason about backups, pairing, and agent authorization.

4.3 Contacts, bonds, and trust levels

Contacts map to bond records with tiers that determine which intents and sensitivity levels are allowed. Public strangers may ping or request bonds; referred contacts gain broader query access; direct bonds unlock friends-tier sharing. Policy evaluation is deterministic and logged for audit.

4.4 Signed messages and verifiable senders

Every envelope carries an Ed25519 signature over canonical JSON so recipients verify sender identity before acting on content. Role fields enforce human-to-human chat versus agent-to-agent task traffic at the schema level. Tampered or replayed messages fail inbound guards.

4.5 Personal agents and external-agent bridges

Bundled EnvoyAI runs in-process with mesh tools, while external agents connect through an HTTP bridge that never receives your private signing keys. The bridge forwards allowed tool calls and translates responses into mesh envelopes. Choose one primary agent surface to avoid conflicting automation.

4.6 Local knowledge, the Library, and the Vault

The Vault stores files on disk under path-safe rules; the Library is the UI and metadata layer for notes, imports, and published items; RAG indexes vault chunks for retrieval during chat. Sensitivity overrides live in `.envoy/sensitivity.json` per item, not per folder. Web content for browsing lives under a separate `web/` directory mapped to `envoy://` paths.

4.7 Tasks, mandates, and approvals

Tasks progress through named lifecycle states with mandates defining authorized intent, cost ceilings, and termination policy. Owners can require approval before specific actions even when a mandate otherwise allows automation. Cancel and heartbeat intents keep long-running work accountable.

4.8 Agent Network membership

Agent Network membership is mutual opt-in among bonded contacts who enable their agents for collaboration. It is not a public marketplace listing anonymous workers. Team jobs consume this membership graph when selecting eligible workers.

4.9 Direct networking and relay assistance

Nodes attempt direct TCP or QUIC connections first, using mDNS on LAN and DHT discovery when configured. When NAT blocks direct paths, circuit relay v2 reservations forward streams without decrypting application payloads. You choose bootstrap relays; they assist connectivity rather than owning your identity.

4.10 Activity records and end-to-end auditing

Audit and journal JSONL files record intents, outcomes, latency, and correlation IDs for multi-hop flows. Operators can trace a Team job, knowledge query, or file transfer across peers using those IDs. Logs intentionally avoid storing raw sensitive payloads unless required for debugging policy.

5. Common Use Cases

5.1 A private personal AI across devices

Run EnvoyAI on a desktop home node and reach it from Social locally or EnvoyGo when paired away from home. Your vault, models, and bonds stay on the computer you trust while the phone acts as a remote control. Back up owner keys and vault data so device loss does not strand your agent history.

5.2 A family or friends mesh

Invite family or friends through introductions, establish direct bonds, and use group chat plus file sharing without a shared cloud account. Each participant keeps their own node and data; sharing is explicit through messages, vouchers, and syndicated knowledge settings. Relays help when members are on different networks.

5.3 Trusted research and knowledge exchange

Exchange research notes with public or friends sensitivity, query peers' syndicated libraries, and save attributed results back to your vault through MCP write-back. Federated RAG respects per-contact ceilings so you never silently exfiltrate private material. Publish finished summaries as mesh pages when you want durable `envoy://` links.

5.4 A small-team Agent Network

Enable Agent Network among a small team that already shares direct bonds and aligned mandates. Assign Team jobs for split research, code review assistance, or draft reports with each worker executing on local hardware. Review audit trails to see which agent contributed each segment.

5.5 Multi-agent planning and report generation

Plan a multi-step report where one agent outlines sections, workers gather evidence from local vaults, and the orchestrator merges attributed text. Mandates cap cost and require approval before sending external email or spending credits. Results land in chat and can be saved as vault notes for later citation.

5.6 OpenClaw with trusted mesh contacts

Keep OpenClaw as EnvoyAI on your node while using mesh tools to message bonded contacts and search syndicated knowledge. OpenClaw never receives raw libp2p access; it calls `mesh.findKnowledge`, `mesh.sendMessage`, and related tools through the registry. This pattern suits power users who want OpenClaw skills with trusted peer reach.

5.7 HomeClaw as an external EnvoyMesh agent

Point EnvoyMesh at a local HomeClaw HTTP endpoint so HomeClaw becomes the conversational surface while the node handles identity and mesh I/O. HomeClaw's own memory and plugins stay in its process; EnvoyMesh enforces bonds on outbound actions. Enable the preset only on machines where you already run and trust HomeClaw.

5.8 Hermes as an external EnvoyMesh agent

Use Hermes when you prefer its Obsidian-style knowledge tooling alongside mesh messaging. The bridge forwards Hermes responses and tool results through the same policy boundary as other external agents. Configure the default `http://127.0.0.1:8020/message` endpoint or your custom URL in Settings → AI.

5.9 OpenHuman as an external EnvoyMesh agent

OpenHuman is available as a disabled-by-default compatibility preset for teams experimenting with that runtime. When enabled, it follows the same one-bridge-at-a-time rule and never receives signing keys. Treat it as optional until your organization validates OpenHuman's local deployment model.

5.10 Claude Desktop using EnvoyMesh through MCP

Register EnvoyMesh as an MCP server in Claude Desktop to expose mesh search, contacts, and messaging tools to Anthropic's client. MCP crosses a desktop boundary—review which tools you enable and what data they can read from your vault. The home node must be running for MCP sessions to succeed.

5.11 External A2A clients delegating tasks

Publish an A2A agent card from your node so external A2A clients can discover capabilities and delegate tasks through JSON-RPC proxies. Home tunnel and relay paths let remote clients reach a home node without exposing raw libp2p to the external runtime. Mandates and approvals still apply to delegated work.

5.12 A self-hosted relay fleet

Deploy one or more relay binaries with advertised addresses for a family, lab, or organization that wants private bootstrap and circuit relay capacity. Relays stay lean: no LLM, no vault, no payload inspection beyond transport forwarding. Monitor relay audit snapshots when operating fleet infrastructure.

6. Product and Protocol Comparisons

6.1 EnvoyMesh and centralized messengers

Integration	Trust boundary	What it can reach
EnvoyAI / OpenClaw	Bundled · in-process	Full mesh tools · chat · tasks
HomeClaw	HTTP bridge · loopbac	Mesh tools · chat (one URL)
Hermes	HTTP bridge · loopbac	Mesh tools · chat (one URL)
OpenHuman	HTTP bridge · loopbac	Mesh tools · chat (one URL)
MCP server	stdio · Claude Deskto	Mesh tools exposed outward
A2A	JSON-RPC · relay	Agent Card · task methods

Figure 18 — Integration-shape comparison: six external integration shapes side by side, each with its trust boundary and reachable surface. EnvoyAI is deepest; MCP/A2A are outward-facing.

Centralized messengers optimize for frictionless signup, phone-number identity, and vendor-operated moderation at scale. EnvoyMesh trades that convenience for self-sovereign keys, explicit bonds, and local-first storage you operate. Choose messengers for mass reach; choose EnvoyMesh when trust boundaries and auditability matter more.

6.2 EnvoyMesh and cloud AI assistants

Cloud AI assistants run inference and memory on vendor infrastructure with account login and vendor policy. EnvoyMesh keeps models, vault, and bonds on your node while optionally calling remote providers you configure. You gain mesh reach and mandates instead of a single-vendor chat history silo.

6.3 EnvoyMesh and standalone OpenClaw

Standalone OpenClaw excels as a local assistant but lacks native signed peer messaging, bond policy, and federated knowledge unless extended. EnvoyMesh bundles OpenClaw as EnvoyAI and wraps it with mesh tools, mandates, and audit. Running both without integration duplicates agents unless you disable one.

6.4 EnvoyMesh and external agent runtimes

External agent runtimes (HomeClaw, Hermes, custom HTTP) focus on conversation and plugins; EnvoyMesh supplies identity, transport, and policy. The bridge pattern keeps libp2p keys on the node while the external process handles UX you prefer. Neither side replaces the other—they compose when configured deliberately.

6.5 EnvoyMesh and MCP

MCP standardizes tool discovery for AI applications; EnvoyMesh implements an MCP adapter that exposes selected mesh capabilities. Native mesh intents remain richer and signed; MCP is an interoperability edge for desktop clients. Enable MCP tools narrowly to limit vault and contact exposure.

6.6 EnvoyMesh and A2A

A2A defines agent cards and task interfaces for cross-product delegation; EnvoyMesh publishes cards and proxies tasks through relay or home tunnel paths. Native Team jobs and mandates govern trust inside the mesh; A2A extends reach to external orchestrators. Both can coexist with different policy surfaces.

6.7 EnvoyMesh native Agent Network versus public marketplaces

Public agent marketplaces optimize for discovery of anonymous workers and commercial ranking. EnvoyMesh Agent Network is the opposite: collaboration only among bonded owners who opted in locally. There is no global listing, reputation score, or payment rail in the native design.

6.8 Native protocols versus interoperability bridges

Signed Envoy envelopes, mandates, and bond tiers are the native protocol inside the mesh. MCP and A2A bridges translate at the edge for external ecosystems without replacing internal security models. Prefer native flows for bonded peer work; use bridges when an external client must participate.

Part II — Install and Get Started

7. Choose Your Setup

7.1 Desktop only

Run EnvoyMesh on a Mac or Windows computer as your primary home node. Install from the current release installer or build from source, create your owner identity on first launch, and keep the machine running when you want mesh connectivity. This path fits anyone starting on one trusted desktop without mobile access yet.

7.2 Desktop with EnvoyGo mobile access

Add EnvoyGo on iOS or Android after your home node is healthy. The phone pairs by scanning a QR code and mirrors chat, contacts, terminals, and selected home features—it does not replace the desktop node or hold owner keys on its own. Plan for the home computer to stay reachable over LAN, relay, or tunnel when you use mobile away from home.

7.3 Desktop with the bundled EnvoyAI agent

EnvoyAI (OpenClaw) ships with the desktop node and starts on port 18789 by default. It can search your Vault, message bonded contacts, and run local tools under your bond and approval settings. Toggle it in Settings → AI or set `openclawEnabled` in `node-config.json` if you prefer to start without the bundled assistant.

7.4 Desktop with an external agent

Connect HomeClaw, Hermes, OpenHuman, or a custom HTTP agent through Settings → AI → Ext Agent. One node runs one external bridge at a time; EnvoyMesh signs mesh traffic on the agent's behalf without handing over Ed25519 keys. Enable the bridge only after you trust the external process and its local endpoint.

7.5 Desktop with local or remote models

Configure model providers under Settings → AI according to your privacy and cost preferences. Local models keep inference on your hardware; remote providers send approved prompts outside the node under your configured limits. Start with one provider, verify responses in chat, then widen automation once approvals behave as you expect.

7.6 Personal relay or community relay

Relays help peers discover each other and traverse NAT; they do not hold your account or read application payloads. Use the community relay for casual testing, or run your own relay with `npm run node:dev -- -- profile ./data/relay --relay-server --listen /ip4/0.0.0.0/tcp/4001`. Normal nodes bootstrap with `--bootstrap "<relay-multiaddr>"` and `--relay`.

7.7 Small-team and organization deployments

Give each team member a home node with its own owner identity, then bond contacts explicitly rather than sharing one login. Operators may deploy private relays, standardize trust tiers, and disable bundled sponsor contacts before fleet rollout. Document profile data paths so backups and upgrades stay consistent across machines.

7.8 Recommended first-time setup

Install the desktop app on a trusted computer, complete owner and device setup, enable EnvoyAI if you want a personal assistant, and back up identity material before adding contacts. Pair one test contact on the same LAN, send a message, then optionally add EnvoyGo. Defer Team jobs, external agents, and WAN relay testing until basic chat and status indicators look healthy.

8. Install EnvoyMesh

8.1 System requirements

Use a supported current macOS or Windows desktop environment with enough storage for the app, local data, and optional model or IPFS components. Source builds require the repository's Node.js toolchain and package dependencies; mobile access additionally requires a running home node.

8.2 Install on macOS

Download the macOS disk image, open it, and move EnvoyMesh to Applications. On first launch, macOS may require confirmation because release signing and notarization can vary by build; retain your data directory when upgrading.

8.3 Install on Windows

Run the Windows installer and allow the bundled node runtime through local firewall prompts when you want peer connectivity. The Windows package intentionally carries a smaller essential OpenClaw extension set to control installer size.

8.4 Install EnvoyGo on iOS

Install EnvoyGo through the available iOS distribution channel, then pair it to an existing home node. EnvoyGo is a thin client: do not expect it to replace the desktop node or preserve an independent mesh identity while the home node is unavailable.

8.5 Install EnvoyGo on Android

Install EnvoyGo on Android and complete the same home-node pairing flow. Notification and background behavior depend on Android permissions, battery optimization, and FCM configuration.

8.6 Install from source

From the repository root, install dependencies with `npm install`, run `npm run typecheck`, and run `npm test`. Start the node with `npm run node:dev`; consult `QuickStart.md` for platform prerequisites and optional components.

8.7 Verify the installation

A healthy installation starts the node, opens the Social interface, shows identity and connection status, and can reach the local service. Verify with the built-in status surfaces before importing data or adding external integrations.

8.8 Application data locations

Identity, trust, audit, task, Vault, and configuration data live in the node's application-data location rather than in the installation directory. Use Appendix K and the current release notes to locate the platform-specific root.

8.9 Update EnvoyMesh

Back up identity and Vault data, stop active tasks, and install the newer package over the application. Review `CHANGELOG.md` for configuration or storage migrations before restarting.

8.10 Uninstall without losing identity or data

Removing the application should be treated separately from deleting its data directory. Preserve the data root and identity backup if you intend to reinstall; delete them only when you deliberately want to erase the local identity and records.

9. Platform and Package Differences

9.1 Desktop and mobile feature comparison

Desktop Social is the full home-node experience: mesh identity, Vault, agents, Team jobs orchestration, Browser, terminals, and settings. EnvoyGo mirrors a subset—chat, contacts, voice calls, read-only Team job status, terminals, and Browser—through JSON-RPC to the paired home node. Treat mobile as a remote control, not a second independent node.

9.2 macOS packaging

macOS releases ship as a disk image with the Tauri-wrapped Social UI and embedded node runtime. OpenClaw extensions are bundled more completely on macOS than on Windows to reduce post-install setup. Check release notes for notarization and Gatekeeper behavior on your macOS version.

9.3 Windows packaging

Windows releases use an installer that bundles the node runtime and a slimmer OpenClaw extension set to control download size. Allow the app through Windows Firewall when prompted if you want inbound peer connections. Profile data lives under your user app-data path, separate from the install folder.

9.4 OpenClaw extensions bundled on macOS

macOS desktop builds include the fuller OpenClaw extension bundle used by EnvoyAI. Source installs copy extensions during `./scripts/setup.sh` or `npm run setup`. Rerun setup after upgrading OpenClaw-related dependencies if you develop from source.

9.5 Essential OpenClaw extension selection on Windows

Windows installers include a curated essential extension set rather than every optional channel. If a capability is missing, compare with the macOS bundle list in release notes or install from source with `.\scripts\setup.ps1`. Core mesh and chat features do not require extra extensions.

9.6 Full and slim desktop bundles

Some releases offer full installers with optional components and slimmer builds without IPFS or extra sidecars. Pick full when you want optional content features out of the box; pick slim on constrained disks or air-gapped lab machines. Your identity and Vault data are the same regardless of bundle flavor.

9.7 Optional IPFS sidecars

IPFS-related components are optional adjuncts for content-addressing experiments, not required for chat, bonds, or Team jobs. Enable them only when release notes document a supported sidecar for your platform. Omit them if you prefer a minimal attack surface.

9.8 Features requiring a home node

Mesh identity, agent runtime, Vault indexing, Team job orchestration, MCP/A2A bridges, and full Settings live on the home node. EnvoyGo, browser dev UI pointed at a remote profile, and CLI against `--profile` all assume that node is running and reachable. Without a home node, mobile mirrors and thin clients cannot authenticate or send signed traffic.

9.9 Features available as an EnvoyGo mobile mirror

EnvoyGo exposes chat threads, contacts, voice calls, terminal attach, Browser for `envoy://` content, push notifications, and read-only recent Team job status under Me → Agent Network. AI engine toggles and bridge configuration appear read-only on mobile; change them on the home node. Cached data on the phone is for convenience, not authoritative identity storage.

9.10 Legacy mobile experiments and current product boundaries

The Capacitor app in `apps/mobile` was an in-process full-node experiment and is not the product mobile path. EnvoyGo is the supported thin client paired to home. Running EnvoyGo as a standalone full mesh node remains parked; use desktop or source builds for a primary node.

10. Create Your Identity

10.1 What your EnvoyMesh identity represents

Your identity is cryptographic, not a cloud username. An owner identity controls mandates and devices; each device has its own keys; your agent identity acts on the mesh under an owner-signed mandate. Peers verify signatures against these IDs rather than trusting a central directory.

10.2 Create an owner identity

On first launch, Social walks you through generating an owner keypair stored in your profile directory (for example `./data/default` in source runs). This step happens once per person; subsequent installs on new machines import or authorize additional devices instead of creating a second owner. Back up the owner material before bonding production contacts.

10.3 Create your first device identity

The first desktop install creates a device identity authorized by your owner keys automatically. The device signs routine envelopes and holds local session state. Note the device ID in Profile or via `npm run cli -w @envoymesh/node -- profile --profile ./data/default` when diagnosing pairing.

10.4 Create or activate your agent identity

EnvoyMesh derives an agent peer identity from your owner and agent keys, then records an owner-signed mandate linking the agent to you. EnvoyAI uses this identity when sending agent-role messages. External bridge agents receive a separate bridge identity persisted as `bridge-identity.json` when enabled.

10.5 Set your display profile

Open Profile in Social to set the name, avatar, and fields other contacts see after bonding. Profile data is signed and stored locally in your profile directory. Update it before sharing pairing codes so recipients recognize you.

10.6 Understand your DID

Your owner DID follows the form `envoy:owner:<hash>` derived from your public key. Device and agent IDs use parallel `envoy:device:` and `envoy:agent:` prefixes. Share owner IDs for stable addressing once peers have exchanged trust; runtime peer IDs can rotate with keys while owner IDs stay long-lived.

10.7 Protect your cryptographic keys

Private keys live in the profile data directory with restrictive file permissions. Do not copy key files to chat, email, or shared drives unencrypted. Use the OS user account protection on your home node machine as the first layer of defense.

10.8 Back up identity and recovery data

Copy the entire profile directory—or export backups your release documents—before OS reinstall or hardware migration. Vault content under `shared_vault/` or your configured vault path should be backed up separately from the application binary. Test restore on a non-production machine before you need it urgently.

10.9 Add another device

Pair a second device by scanning a QR code or approving a pairing request from the home node's Pairing Queue. The owner signs a device certificate authorizing the new device while sharing the same owner ID. EnvoyGo pairing follows the thin-client flow: the phone receives a session to the home node rather than duplicating owner keys on the phone.

10.10 Revoke a lost or compromised device

From a trusted remaining device, revoke the lost device certificate and remove its trust entries. Change any bridge secrets if the external agent ran on the compromised machine. Treat owner key compromise as catastrophic: revoke devices, rotate bridge credentials, and rebond contacts only after you are confident keys are clean.

11. Tour the Application

11.1 Home and node status

The home view summarizes node connectivity, discovery mode, and recent activity. Use it to confirm the node is listening, relays are reachable, and no startup warnings remain. CLI equivalents include `connectivity-status` and `relay-status` for deeper diagnosis.

11.2 Conversations

Conversations lists direct and group chat threads with delivery indicators. Open a thread to send text, audio, files, or agent messages depending on trust and settings. Search and pin behavior follow the current Social release; unread state syncs from your local profile store.

11.3 Contacts and discovery

Contacts shows bonded peers with trust tier badges; discovery surfaces capability or tag-based lookups where policy allows. Strangers remain heavily rate-limited until you accept a bond request. Block or downgrade trust from the contact detail sheet if a relationship changes.

11.4 Groups

Create a group from Conversations, add bonded contacts, and set a title and avatar. Group messages use the same signed envelope path as direct chat with group routing metadata. Only add participants you trust at the sensitivity level you plan to share in the group.

11.5 Knowledge Base and Library

Library is the in-app knowledge base: create Markdown notes, import documents, and toggle per-item sensitivity. The policy engine honors four ranks — `public`, `friends`, `trusted`, `private` — while the UI exposes friendlier labels for the ones you pick most often. Saved notes index into RAG automatically. Optional Obsidian and MCP plugins are configured under Settings → AI → Knowledge Base.

11.6 Browser

Browser loads permitted `envoy://` mesh content through your node's policy boundary. You see what bond rules and sensitivity labels allow—not the open web by default. Use it to read published notes and mesh pages from bonded or public authors.

11.7 Team jobs

Team jobs appear where Agent Network is enabled. Your agent orchestrates work across opted-in bonded agents; you review plans, budgets, and results in the Team jobs UI. Start with small objectives before enabling automatic cost rebalance policies.

11.8 Terminals

Terminals attach to shell sessions on the home node via WebSocket, including from chat inline or the dedicated terminals view. Sessions require authentication through the node and respect your approval settings for agent command execution. Remote attach from EnvoyGo tunnels through the home JSON-RPC transport.

11.9 Approvals and activity

Approvals queues sensitive agent or task actions awaiting your decision; Activity (audit) shows allow/deny outcomes with correlation IDs. Approve or reject from Social or CLI (`npm run cli -w @envoymesh/node -- approvals ...`). Use correlation IDs to stitch multi-step Team jobs or relay-assisted flows.

11.10 Profile

Profile edits your human-visible identity and shows owner, device, and agent identifiers. It is the right place to copy pairing information and verify which device you are on. Changes propagate to contacts on the next signed profile update they receive.

11.11 Settings

Settings controls discovery profiles, AI engines, external agent bridges, knowledge plugins, notifications, and node behavior flags. Changes write to `node-config.json`, `bridge-config.json`, and related files in your profile directory. Restart or follow in-app prompts when a setting requires a node reload.

11.12 Connection and agent status indicators

Header badges show WebSocket/Social connectivity, mesh reachability, EnvoyAI gateway health, and external bridge state when configured. Yellow or red states mean you should fix connectivity before sending sensitive data. EnvoyGo shows a parallel connection indicator for home reachability.

12. Connect Your First Contact

12.1 What pairing and bonding do

Pairing exchanges enough information to identify and reach another owner; bonding records the trust relationship and policy tier. A packaged desktop build may also add the project sponsor contact from `bundled-sponsor-friend.json` on first launch; operators can disable that bundle before deployment.

12.2 Pair with a QR code

Open Add Contact on one device and Show My Code on the other, then scan with the built-in scanner in Social or EnvoyGo. Confirm the displayed owner ID and display name match what you expect in person. Complete the bond request flow before treating the contact as trusted.

12.3 Pair with an invitation link

Generate an invitation link or multiaddr payload from Contacts and share it over a channel you trust (Signal, in-person AirDrop, etc.). The recipient opens the link in Social to initiate pairing. Treat leaked links like leaked phone numbers—revoke or ignore unexpected bond requests.

12.4 Pair on a local network

On the same LAN, mDNS discovery may list nearby nodes without manual multiaddrs. Start both nodes with default discovery or `--listen /ip4/0.0.0.0/tcp/0`, then pick the peer from the discovery UI. LAN pairing is the fastest way to validate signing and chat before testing relay paths.

12.5 Verify identity information

Before accepting a bond, compare owner ID, display name, and optional proof text out of band. Signed envelopes prove possession of keys, not that you know the person—your proof step closes that gap. Reject requests that do not match what your contact said they would send.

12.6 Choose an appropriate trust level

EnvoyMesh trust tiers are blocked, public (stranger), referred, and direct (friend). Start new acquaintances at public or referred unless you already have a strong trust basis. Direct unlocks richer knowledge sharing and agent collaboration; upgrade only deliberately.

12.7 Accept a bond request

Incoming bond requests appear in Contacts or notifications with the sender's proof message. Accept to record mutual trust locally; reject leaves them at stranger tier. Either side can later change tier or block from contact settings.

12.8 Send the first message

Open the new contact thread and send a short signed chat message. Watch for delivered or read indicators according to your release. If the message stalls, check connectivity status before resending duplicates.

12.9 Confirm direct or relay-assisted delivery

Successful delivery shows positive acknowledgment in-thread or an audit `chat.message` allow row. Relay-assisted paths use `/p2p-circuit` addresses learned from `relay.lookup`; direct LAN paths skip relay hops. CLI audit with `--include-p2p-trace` helps confirm which path was used during testing.

12.10 Troubleshoot pairing

Verify both nodes run, firewalls allow outbound TCP, and profile paths match between UI and CLI. For WAN tests, confirm bootstrap relay multiaddrs and run `connectivity-status`. Retry with freshly copied listening multiaddrs after restarts because dynamic ports change.

12.11 Bundled sponsor contact

A packaged desktop build (DMG / `.exe` / `.AppImage`) auto-bonds to the project's sponsor contact on first launch using the bundled `bundled-sponsor-friend.json`, so you start with one working contact out of the box. This is a convenience, not telemetry: no data leaves your node, and the bond is a normal local trust record you can edit or remove like any other contact. Operators preparing fleet images can disable the auto-bond by setting `{"enabled": false}` in the bundled file before packaging.

13. Connect EnvoyGo

13.1 How EnvoyGo works with a home node

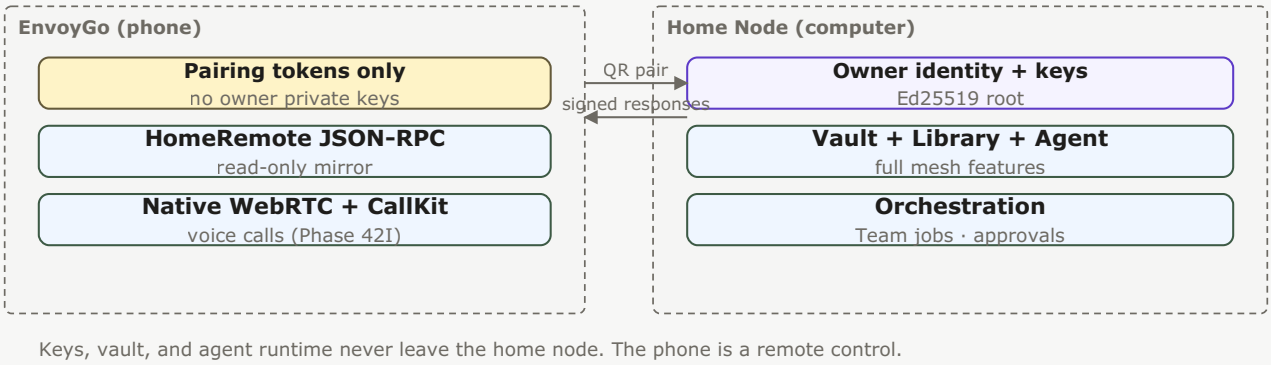

Figure 12 — EnvoyGo thin-client pairing: the phone holds only pairing tokens and calls the home node via JSON-RPC. Identity, vault, agent, and orchestration stay on the home node.

EnvoyGo connects to a paired home node and presents selected NodeService functions through a mobile interface. The home node keeps mesh identity, agent runtime, Vault, and orchestration responsibility.

13.2 Pair the mobile app

Install EnvoyGo, tap Pair with Home, and scan the QR code shown in Social on the desktop node (or enter the pairing payload your release documents). Approve the device on the home node if prompted in Pairing Queue. The app stores pairing tokens in secure storage, not owner private keys.

13.3 Confirm the home connection

After pairing, the connection indicator should show home reachable and load your chat list. Pull to refresh or open Me → Node status if threads stay empty. Ensure the desktop node stays running and reachable on the network path you expect (LAN, relay tunnel, or configured remote URL).

13.4 Use chat and contacts

Chats and People tabs mirror home-node threads and bonded contacts with mobile layouts. Sending a message routes through HomeRemote JSON-RPC to the home node, which signs and delivers on the mesh. Media and audio messages follow the same path.

13.5 Use remote terminals

From Terminals, attach to an existing session or start one allowed by home policy. Input travels over the tunneled terminal protocol; output streams back with scrollback. Avoid sensitive commands on untrusted networks until you confirm transport encryption and home reachability.

13.6 View Team jobs

Me → Agent Network shows read-only recent Team job activity synced from the home node. You can inspect status and reports but cannot orchestrate new jobs from mobile alone—start jobs from desktop chat with your agent. The UI says Team jobs even when logs use older internal terminology.

13.7 Browse mesh content

The EnvoyGo Browser (Phase 45C) opens `envoy://` content through the paired home service. Availability depends on the home node being reachable and on the requested author or content being permitted by bond policy.

13.8 Receive notifications

EnvoyGo can receive normal and call-related notifications when APNs or FCM is configured. iOS backgrounded calling uses VoIP push + CallKit (Phase 42I) and the operating system grants permission. Delivery remains best effort and is affected by platform background restrictions.

13.9 Make and receive voice calls

Available mobile call support covers one-to-one voice calls with native WebRTC and platform call integration. iOS ships VoIP push + CallKit (Phase 42I, shipped 2026-06-19) so backgrounded phones can receive calls; real-device validation is still open. Video calling is not yet available (see §18.10 and Appendix J.4). TURN may be required for cross-network audio when both peers sit behind restrictive NAT.

13.10 Revoke a lost phone

From the home node, revoke the EnvoyGo device or session pairing and rotate any exposed tokens. Remove the node entry in EnvoyGo if you recover the phone later and need a clean re-pair. Treat a lost unlocked phone like a lost session to your home API.

13.11 Current mobile limitations

EnvoyGo does not run a full mesh node, orchestrate Team jobs, edit all Settings, or replace home-node Vault authoring. Video calls, full Browser parity, and background reliability vary by OS permissions. See release notes for the exact feature matrix on your build.

14. First-Day Tutorials

14.1 Send a private message

Bond a contact (Chapter 12), open their thread, type a short message, and send. Confirm the delivery indicator updates. If it fails, open Home status and verify mesh connectivity before retrying once.

14.2 Create a group conversation

From Conversations, choose New Group, select bonded contacts, name the group, and send a hello message. Each member receives group envelopes signed by your node. Adjust membership later from group settings if your release exposes it.

14.3 Send an audio message

In chat, tap the microphone control, record a brief clip, and send. The audio rides inside a signed chat envelope and plays inline for recipients. Grant microphone permission when the OS prompts on desktop or EnvoyGo.

14.4 Make a voice call

With a direct-trust contact, start a voice call from the thread header. Accept the incoming call on their device; media flows peer-to-peer after mesh signaling. If connection fails behind strict NAT, configure TURN as documented for your release.

14.5 Share a file

Use the attachment control in chat or share from Library/Vault according to sensitivity rules. Files transfer as data intents with policy checks on path and trust tier. Confirm the recipient sees the attachment and audit logs an allow outcome.

14.6 Ask EnvoyAI a question

Open your agent thread or main assistant entry point and ask a factual question answerable from your Vault or public knowledge. EnvoyAI runs locally on the node gateway unless you routed engines differently. Deny or refine if the agent requests approval for a sensitive tool call.

14.7 Add knowledge to your Library

Open Library → New Note, write Markdown, set sensitivity, and save. Indexing runs automatically for RAG. Optionally open the vault folder in Obsidian if you enabled the plugin and want external editing.

14.8 Search your Vault

Use Library search or ask EnvoyAI to search local knowledge with explicit scope. CLI users can run `npm run cli -w @envoymesh/node -- vault-search --vault ./shared_vault --query "your terms"`. Results respect sensitivity labels and your role on the node.

14.9 Ask a bonded agent for knowledge

Message a contact’s agent or send a knowledge query where the UI supports it, staying within their trust tier. Public-tier queries are rate-limited for strangers; direct bonds allow richer scope. Expect signed responses attributable to their agent identity.

14.10 Approve a sensitive action

When an agent or task triggers policy, an approval card appears in Approvals. Read the summary, correlation ID, and requested action before allowing. Reject if the scope exceeds what you intended for that session.

14.11 Start a simple Team job

In chat with your agent, describe a small multi-step objective that can delegate to a bonded peer’s agent (for example summarize then translate). Confirm Agent Network membership is on for both sides. Review the plan, budget cap, and final Team job report before sharing externally.

14.12 Connect an external agent

In Settings → AI → Ext Agent, pick HomeClaw, Hermes, or Custom and point to the local HTTP endpoint (`http://127.0.0.1:8010/message` for HomeClaw by default). Start the external process, enable the bridge, and send a test chat message to the bridge agent peer. Verify callbacks arrive on the configured listen port before enabling automation.

Part III — People, Profiles, and Conversations

15. Contacts and Bonds

15.1 View and search contacts

Open **People** in Social or the Contacts tab in EnvoyGo to browse bonded owners and pending introductions. Search by display name or owner ID fragment; results respect your local trust store, so blocked contacts stay hidden unless you explicitly show them. EnvoyGo lists the same contacts through HomeRemote JSON-RPC—it does not maintain a separate contact database on the phone.

15.2 Understand contact identity

Each contact maps to an **owner identity** (`envoy:owner:…`) backed by Ed25519 keys, not a central account handle. Runtime messages use peer IDs derived from keys; compare owner ID and any out-of-band proof before upgrading trust. QR pairing (Chapter 13) adds **device** identities under the same owner—it does not replace owner-to-owner bonds.

15.3 Contact profiles and photos

Profile cards show display name, description, and photos the contact publishes within bond policy. Photos arrive as signed profile or file payloads; referred and public tiers may see fewer fields than direct friends. Tap a photo to view full size; do not treat gallery thumbnails as verified identity proof by themselves.

15.4 Online, offline, and connection states

Presence reflects mesh reachability, not a cloud “online” flag. A contact may show offline while messages queue for relay-assisted delivery when they return. EnvoyGo shows home connectivity separately from remote peer reachability—your phone can be online to home even when the contact is not.

15.5 Direct, referred, public, and blocked trust

EnvoyMesh uses four user-selectable tiers for contacts — **blocked** (deny all), **public** (stranger—ping and narrow discovery only), **referred** (introduced—limited knowledge and approvals), and **direct** (friend—richer chat, files, and agent workflows up to friends sensitivity). Tier is stored locally on your node; both sides can set different tiers toward each other.

15.6 Change a contact’s trust level

Open the contact in Social → **Trust** (or equivalent Settings) and pick blocked, public, referred, or direct. Downgrading takes effect immediately for new operations; already-delivered content remains in local history until you delete it. Document why you changed tier—audit rows help if you later review an incident.

15.7 Refer or introduce a contact

Use **Introduce** or bond-request flows to vouch for someone at referred tier without granting direct trust yourself. Introductions carry signed proof text so the recipient can verify out of band. Referred contacts cannot recruit your agent into Team jobs until you deliberately upgrade them.

15.8 Mute, block, or remove a contact

Mute suppresses notifications locally without changing bond tier. **Block** sets blocked trust and stops new inbound intents. **Remove** clears local thread metadata but does not erase their keys from the network—re-add only after you are comfortable with renewed contact.

15.9 Restore a connection

To reconnect after block or accidental removal, exchange a fresh bond request or introduction with updated proof text. If you revoked their tier, they must accept a new request; stale threads may not resume automatically. Verify identity again before restoring direct trust or sharing files.

15.10 Contact privacy and disclosure settings

Profile and contact settings control what you publish and what you request from others: display fields, photo visibility, and sensitivity labels on shared knowledge. Defaults lean conservative for public-tier viewers; direct contacts see richer profile slices. Changes propagate on the next signed profile update, not retroactively to old screenshots.

16. Private Messaging

16.1 Start a conversation

From **People**, open a direct contact or pick an existing thread under **Chat**. New conversations require at least public-tier reachability and a successful bond or introduction path. Group rooms use separate creation flows (Chapter 17); do not assume a DM thread exists for every contact until you send the first message.

16.2 Human-to-human messages

Private chat uses the `chat.message` intent with **human** sender and **human** recipient roles—agents cannot impersonate this path. Messages are signed envelopes delivered over libp2p direct or relay-assisted paths. Compose in Social or EnvoyGo; the home node signs and sends on your behalf when using mobile.

16.3 Human-to-agent messages

Talking to **@envoy** or your configured agent name routes through agent-capable chat flows, not `chat.message` human-to-human semantics. Agent replies may invoke tools under mandate and bond policy. Keep owner-facing instructions separate from peer DMs so you do not accidentally share private context with a contact thread.

16.4 Replies and conversation continuity

Replies reference prior messages through thread metadata and correlation IDs in audit logs. Quote or reply in-thread to preserve context; resending the same text creates duplicate envelopes. Search (16.7) helps locate earlier turns when a long DM splits across sessions.

16.5 Message delivery states

Delivery indicators reflect local send acknowledgment and remote acceptance when your build exposes them—not read receipts unless explicitly supported. Failed sends show policy or connectivity errors; read audit for `chat.message` deny vs transport timeout. Avoid rapid duplicate sends while a message is still pending.

16.6 Offline behavior and retries

When a contact is offline, the home node queues signed messages where protocol and policy allow and retries over direct or relay paths on reconnect. Large backlogs may arrive out of strict UI order but remain integrity-checked by signature. EnvoyGo offline to **home** prevents any send until the tunnel restores.

16.7 Search conversation history

Use in-app search or Vault-adjacent conversation indexes where enabled to find text by keyword or contact. Results come from locally stored copies on the home node; mobile search queries home over JSON-RPC. Sensitive threads remain visible only on devices paired to that node.

16.8 Draft assistance

Draft assistance (when enabled) suggests completions through your configured model with semantic-firewall limits—it does not auto-send. Review suggested text before sending; agent-assisted drafts in contact threads still obey bond tier and sensitivity. Disable assistance in Settings if you prefer manual composition only.

16.9 Manage conversation data

Export, archive, or delete conversation data from thread menus or profile maintenance tools on the home node. Deletion is local to your store unless a product feature explicitly requests remote retraction—which is not guaranteed for already-delivered peer copies. Back up before bulk purge (Chapter 89).

16.10 Message privacy and security

Messages inherit transport encryption from libp2p where negotiated; authorization still depends on signatures and bond policy, not TLS alone. Do not paste secrets into chats with referred or public contacts. Report abuse via block tier and preserve audit correlation IDs if you escalate.

17. Group Conversations

17.1 Create a group

In Social, choose **New group** (or Rooms) and name the room. Initial members must be contacts you can reach under current trust—typically direct or referred depending on policy. The creating node stores membership locally; new members receive signed invites through mesh delivery.

17.2 Invite members

Add members from your bonded contact list; you cannot invite blocked owners or strangers without an introduction path. Each invite is a signed membership intent; pending members appear until they accept. Large groups increase fan-out latency—prefer focused rooms for time-sensitive coordination.

17.3 Send group messages

Group messages use room-scoped chat intents with human senders; delivery fans out to online members and queues for offline ones where supported. @mentions and replies follow the same threading rules as DMs within the room context. EnvoyGo group chat mirrors home threads once paired.

17.4 Manage membership

Owners with admin rights (per your build) can add or remove members and rename the room. Removing someone stops new deliveries to them but does not erase history on their node. Rotate admins deliberately—compromised admin devices can invite unwanted members.

17.5 Leave a group

Choose **Leave group** to stop receiving new messages; your past copies remain on your node until you delete them. Other members continue the room. Rejoin requires a fresh invite if membership is not automatically restored.

17.6 Group trust boundaries

Group visibility does not bypass per-member trust: a referred member still cannot access direct-only file shares you send outside the room. Sensitive attachments should use explicit sensitivity labels. Do not treat group membership as mutual direct friendship with every participant.

17.7 Group delivery and offline members

Offline members receive queued room messages on reconnect; ordering may batch during catch-up. If many members are behind relay-only paths, expect delayed delivery indicators. Check home connectivity before assuming the room is broken.

17.8 Group troubleshooting

If messages stall, verify each member's bond tier, home reachability, and relay reservation. Audit rows tagged with the room correlation ID show deny vs timeout. Split troubleshooting: policy denials need trust changes; transport failures need connectivity work (Chapter 91).

18. Audio and Voice Calls

18.1 Record and send an audio message

Hold the microphone control in a DM or group thread to record a short audio clip; release to attach and send. Audio rides the same signed file/message path as other attachments with size caps enforced by inbound guard. Prefer text for referred contacts unless they expect voice notes.

18.2 Play and manage audio attachments

Tap an audio bubble to play; long-press for save or delete locally where supported. Playback decodes on device; very long clips may be rejected at send time. Manage storage under conversation settings if attachments accumulate.

18.3 Start a voice call

Start a **voice call** from the call button in a bonded direct thread on Social or EnvoyGo. Calls negotiate WebRTC audio between peers with home-node signaling; video is not available in current builds. Both sides need microphone permission and reachable mesh or relay paths.

18.4 Answer or decline a call

Incoming calls surface as in-app banners and, on EnvoyGo, platform call UI when configured. Decline sends a signed reject; answer establishes the WebRTC session. Unknown or blocked contacts should not reach call UI if policy is working—verify trust tier if calls appear unexpectedly.

18.5 Call status and controls

In-call controls include mute, speaker routing, and hang up; status shows connecting, active, or failed phases. Dropped calls may retry manually—there is no hidden auto-redial. Note correlation IDs in audit if you report persistent failure.

18.6 Background calls and mobile notifications

EnvoyGo can receive call notifications via APNs/FCM when push is configured; background behavior depends on OS policies. Keep the app paired to home and allow notification permissions for reliable ringing. Desktop Social may use local notifications without mobile push.

18.7 STUN and TURN connectivity

WebRTC tries direct UDP first, then STUN, then configured TURN when both peers sit behind symmetric NAT. Configure TURN in Settings if calls connect but have no audio. Relay libp2p paths carry signaling—not a substitute for TURN media relay.

18.8 Call privacy

Voice calls require at least direct or referred trust per product policy; blocked contacts cannot initiate calls. Call metadata appears in audit; media stays peer-to-peer when WebRTC succeeds. Do not share screen or video—video calls remain planned (18.10).

18.9 Voice-call troubleshooting

If calls fail to connect, check microphone permissions, TURN settings, bond tier, and `connectivity-status`. One-way audio often means NAT or firewall blocking UDP. Test LAN direct path first, then relay-assisted WAN before opening broad firewall rules.

18.10 Video calls — planned, not currently available

Planned. One-to-one audio calling is available today (§18.3); video calling is architecturally anticipated but not shipped in the current release. See Appendix J.4 for the roadmap boundary.

19. Files, Photos, and Profile Sharing

19.1 Share a file

Use the attachment or **Share file** action in a DM or group allowed by trust tier. Files chunk and transfer with integrity checks; direct friends typically have the broadest limits. Name files clearly—recipients see filenames before accepting.

19.2 Accept or decline an incoming share

Incoming shares prompt accept or decline before writing to Vault or Downloads per sensitivity. Declined transfers do not partial-write; accepted files land in policy-scoped storage. On mobile, acceptance may require home online to complete.

19.3 Check transfer progress

Progress bars reflect bytes acknowledged on the transfer voucher path; stalled progress usually means connectivity loss mid-stream. Wait for retry or cancel and resend smaller files. Audit may log partial transfers without storing incomplete secrets in the log body.

19.4 Verify file integrity

Compare displayed hash or size metadata when your build exposes them; signatures prove sender identity, not that the file is benign. Scan unfamiliar binaries locally before opening. Re-send if hash mismatch reports after completion.

19.5 Share profile photos

Share profile photos through Profile → Gallery → publish or send to a contact. Published photos obey visibility tier; direct shares attach to a thread like other media. EnvoyGo displays photos fetched via home—editing gallery is primarily a desktop Social flow.

19.6 Manage your profile gallery

Maintain ordered gallery slots on the home node; reorder or remove images before they propagate in the next profile sync. Removing a gallery image stops future fetches but not copies already saved by contacts. Keep at least one neutral avatar for referred viewers if you use public discovery.

19.7 Choose visibility and sensitivity

Tag shares with sensitivity matching Vault conventions (`public` / `friends` / `trusted` / `private`). The UI exposes friendlier labels for the most common choices; the policy engine honors all four ranks. Down-tier contacts cannot escalate sensitivity at receipt—the bond engine denies incompatible requests. Default to friends or private for documents with personal data.

19.8 Remove shared content

Delete local copies from thread attachments or Vault paths; remote peers may retain their accepted copies unless a retraction feature exists in your build. Profile photo removal updates your signed profile on next publish. For incidents, block the contact and revoke trust (Chapter 87).

19.9 Troubleshoot file transfers

For stuck transfers, verify trust tier, file size limits, disk space on home Vault, and relay reachability. Retry on a stable network with a smaller test file to isolate policy vs transport. Collect audit correlation IDs before sharing diagnostics (Chapter 91).

20. Profiles and Presence

20.1 Edit your human profile

Edit **Profile** → **Human** in Social to set display name, bio, and published fields. Changes serialize into signed human profile payloads stored on the home node. EnvoyGo shows the result read-only unless your release adds mobile editing.

20.2 Edit your agent profile

Agent profiles describe capabilities exposed to peers (tools, Team job roles, A2A card fields). Edit under Profile → Agent or Agent Network settings; owner mandate bounds what the agent may advertise. Misleading capability text does not grant extra permissions—bond policy still gates actions.

20.3 Display names and descriptions

Display names are cosmetic; authorization uses owner and peer IDs. Keep descriptions concise—public-tier viewers may see shortened fields. Avoid embedding secrets or recovery codes in public bio text.

20.4 Profile photos and galleries

Human and agent profiles can each carry photo galleries with tier-aware visibility. Upload on desktop Social; sync propagates to contacts on profile fetch. Large images may be downscaled to respect size limits.

20.5 Identity details and DIDs

The profile details pane shows owner DID, device IDs where relevant, and fingerprint-style hashes for verification. Share these out of band when confirming identity—do not trust unsolicited IDs in chat alone. QR pairing encodes device pairing payloads, not owner DID substitution.

20.6 What bonded contacts can see

Direct contacts see the richest profile slice your policy publishes; referred contacts see reduced fields; public strangers see only public-sensitivity profile data if exposed. Blocked contacts see nothing new from you. Review **Profile visibility** settings before enabling discovery features.

20.7 Profile synchronization

Profile updates push on signed publish events; contacts refresh on next fetch or thread open. There is no global cloud profile CDN—peers learn changes when they communicate with your node. After key rotation, republish profile so fingerprints match.

20.8 Privacy defaults

Initial privacy defaults favor minimal public exposure: conservative photo visibility, friends-level chat history on home, and agent tools disabled until mandated. Review defaults after install before joining discovery topics. Reset paths are in Settings → Privacy where available.

20.9 Family Network — one home node, many profiles

The Family Network turns a single EnvoyMesh home node into a private family social network. The owner installs the node on a desktop or laptop, configures the model, and pairs their phone; then each family member pairs their own phone and gets a focused, independent experience — their own profile, their own AI threads,

and family direct + group chat. No cloud, no subscription, no data leaving your home except the LLM API calls you configure. Think of it as Netflix profiles on one account, or macOS user accounts on one machine: shared infrastructure, isolated experiences.

The home node keeps a single mesh identity (`envoy:owner:…`) — the owner is the only participant visible to the wider P2P mesh. Family members exist as local profiles on the home node, not as mesh peers; their contacts and chats are derived from the family roster, not from mesh discovery.

20.10 Owner and family member roles

The **owner** is the person who installed the node. They keep the full EnvoyMesh product: EnvoyAI, character bots, Ext Agent chat, terminal, Pi coding agent, vault, external mesh bonding, node settings, and family administration (create, rename, and delete member profiles; configure the model API key and infrastructure).

A **family member** gets a focused subset: their own profile, their own AI and bot threads, Ext Agent chat, family direct and group chat, and push notifications. They do **not** get external mesh bonding, terminal, Pi, vault, or node settings. Each profile is locked to one device at pairing time — there is no profile switching within an app, which keeps each person's data and AI threads cleanly separated.

20.11 Invite a family member

The owner creates a family profile from Settings and generates a **family invite QR**. This is distinct from a normal EnvoyGo pairing QR: a family invite binds the pairing device to that specific member profile, whereas normal pairing binds to the owner's full node. The family member installs EnvoyGo, scans the invite QR, and their phone pairs to the home node over WebSocket or libp2p circuit relay (§45). Once paired, the member appears in every other family member's contact list automatically — no separate bonding step.

Because each profile is locked to one device, a member who changes phones must be re-paired by the owner (the old device is revoked). The owner can rename or delete a profile at any time; deletion removes that member's data and revokes their device.

20.12 Family direct and group chat

Family members can direct-message each other and participate in family-only group chats. These conversations are local to the home node — they never traverse the wider mesh, and family contacts are not mesh peers. Presence (online/away/offline) reflects each member's paired-device connection to the home node. Push notifications (FCM on Android, APNs on iOS) deliver messages to a member's phone when EnvoyGo is in the background, so family chat feels like any other messaging app while staying on infrastructure you control.

20.13 Shared AI agents, isolated data

All family members share the home node's model configuration and agent runtime — one LLM API key, one OpenClaw / Ext Agent runtime — so the owner configures AI once and every member gets an assistant. But each member's AI threads, bot conversations, and Ext Agent chats are private to that profile. No family member can read another's AI history, and member data is isolated at the profile level. Shared infrastructure, isolated experiences: the AI is the same engine, but each person's memory and conversations stay sealed.

20.14 What family members cannot access

To keep the family experience focused and safe, member profiles are deliberately scoped down. A family member **cannot**:

- Open a terminal to the home machine (owner-only)
- Use the Pi coding agent (owner-only)
- Browse the vault or Library (owner-only)
- Bond with external mesh contacts or participate in mesh discovery (owner-only)
- Change node, relay, or model settings (owner-only)
- Administer other family profiles (owner-only)

If a member needs mesh access, terminal, or vault, the owner can perform that action on their behalf from the owner profile, or the member should pair as an owner on their own separate home node. The Family Network is a private social layer, not a shared admin console.

Part IV — Your Personal AI

21. Meet EnvoyAI

21.1 What EnvoyAI is

EnvoyAI is your owner-facing assistant on the home node, powered by the bundled OpenClaw runtime. You talk to it from Social, EnvoyGo, or `@envoy` in chat; it plans replies and calls mesh tools through EnvoyMesh policy rather than getting raw libp2p access. Think of it as the brain that stays inside the security boundary while the node handles identity, bonds, and audit.

21.2 OpenClaw as the bundled agent runtime

OpenClaw runs as a child process the node starts and supervises. Its gateway listens on port `18789` by default (`http://127.0.0.1:18789/webhook/envoymesh`). EnvoyMesh passes each Assistant turn session context—bonds, interests, and the tool catalog—and OpenClaw owns multi-turn reasoning and persistent memory across sessions.

21.3 How EnvoyAI differs from the external-agent bridge

EnvoyAI is in-process with full ToolRegistry access. The external-agent bridge (default port `3031`) is an optional HTTP pipe to HomeClaw, Hermes, OpenHuman, or a custom agent in another process. You can run both engines (`both` mode) or either alone; the bridge agent never receives your libp2p keys.

21.4 What EnvoyAI can access

EnvoyAI reads your local vault and Library within sensitivity labels, queries bonded peers through `knowledge.query`, and uses chat RAG when Knowledge Base settings allow. It cannot bypass bond tiers: strangers stay rate-limited, and private material requires direct trust or owner approval. Configure ceilings under Settings → AI → Knowledge Base and per-contact preferences before enabling auto-replies.

21.5 Mesh tools available to EnvoyAI

At startup the node exports a tool catalog to OpenClaw—chat send, library read/discover, task propose, discovery, approvals, triggers, MCP proxy, and more. Each tool declares a sensitivity ceiling and whether it needs owner approval before execution. EnvoyAI chooses tools by name; EnvoyMesh enforces policy and writes an audit row for every call.

21.6 Policy and approval controls

Bond Engine decisions, mandate limits, and the approval queue sit between EnvoyAI and the mesh. Outbound chat, file shares, cloud model calls, and high-sensitivity vault reads queue for your review unless an autonomous policy explicitly allows them. Flip `autonomousKillSwitch` in Settings to pause all autonomous actions and force approval on everything the agent would have done silently.

21.7 Start, stop, and inspect the agent

Open Settings → AI → AI Engine to see OpenClaw status: enabled flag, running state, PID, and last error if the gateway failed. Use **Restart OpenClaw** for a clean child-process recycle without restarting the whole node. Toggling `openclawEnabled` off stops the gateway immediately and prevents spawn on the next node start—useful when debugging port conflicts on `18789`.

21.8 Current limitations

Chat drafts and lightweight auto-replies still route through EnvoyMesh's native model router for speed; complex Assistant turns go to OpenClaw with fallback to native when the gateway is down. Full chat-history injection into OpenClaw context and multi-round tool loops within one turn remain partial—session memory works, but recent thread text may not always be attached. Terminal Agent mode uses the native model directly, not OpenClaw exec.

22. AI Engine Modes

22.1 Built-in only

Built-in only (`openclaw-only`) is the default on fresh installs: `openclawEnabled` is on and `bridgeEnabled` is off. EnvoyAI handles Assistant chat, tool execution, and session memory; no external HTTP agent listens on `3031`. Choose this when you want one bundled runtime and no second agent process.

22.2 Built-in plus external agent

Built-in plus external (`both`) runs EnvoyAI and the bridge together. Mesh traffic from bonded contacts can reach the bridge agent while you still use OpenClaw for `@envoy` and Settings → AI workflows. Enable `bridgeEnabled`, pick an active external agent in `bridge-config.json`, and confirm both status chips in the header before relying on either path.

22.3 External agent only

External agent only (`ext-only`) disables the OpenClaw gateway (`openclawEnabled: false`) but keeps the bridge active. All bridged chat and mesh tool calls go through your external agent's HTTP endpoint; EnvoyAI Assistant turns are unavailable. Use this when HomeClaw or Hermes is your primary brain and you only need EnvoyMesh for connectivity and policy.

22.4 No AI

No AI (`off`) turns off both engines. The node still routes human chat and policy, but no model drafts, auto-replies, or agent tools run. Select this for air-gapped nodes, CI fixtures, or when you need mesh connectivity without any LLM surface.

22.5 Choose the right mode

Start with **built-in only** for the simplest path. Add **external** when you already run HomeClaw/Hermes and want its plugins or memory model. Use **both** only when you deliberately want two agents—otherwise pick one brain to avoid duplicate replies. Switch to **off** temporarily rather than uninstalling when testing connectivity alone.

22.6 Change the active external agent

External agents are defined in `bridge-config.json` under `extAgents`; set `activeExtAgentId` to the entry you want. Each definition includes display name, base URL, bearer token, and capability flags. After editing, restart the node or reload bridge config so the new destination binds to port `3031` (or your configured `bridgeListenPort`).

22.7 Startup settings versus runtime settings

`openclawEnabled` and `bridgeEnabled` are persisted in `node-config.json` and take effect on node start—or immediately stop a running gateway when flipped off. Runtime status (`getOpenClawStatus`, `getBridgeStatus`) shows whether child processes are actually healthy, which can lag config during startup. Model provider mode, AI rules, and contact preferences also persist to `node-config.json` and apply on the next agent turn without restart.

22.8 Diagnose agent availability

If EnvoyAI shows **Stopped**, read `lastError` on the OpenClaw status panel—common causes are port `18789` in use, a missing OpenClaw binary, or repeated watchdog restart failures. For the bridge, verify loopback reachability, bearer token match, and that exactly one active agent is selected. CLI helpers include connectivity status; Social's header badges mirror the same effective mode as Settings → AI → AI Engine.

23. Models and Providers

23.1 Model routing overview

EnvoyMesh uses two tiers: the **native router** (`@envoymesh/models`) serves chat drafts, auto-replies, terminal assist, and Team-job planning; **OpenClaw** serves Assistant/`@envoy` turns with its own LLM config. Native routing respects the semantic firewall (empty prompts rejected, 48K char cap, control-character filter). When OpenClaw is unavailable, Assistant requests fall back to the native provider you configured.

23.2 Configure a local model

Set provider mode to **ollama** in Settings → AI → Model (or `node-config.json`). Point `endpoint` at `http://127.0.0.1:11434/v1` and set `modelName` to your pulled tag (for example `llama3.1`). Local calls skip cloud approval gates and keep prompts on your machine—ideal for drafts and sensitive vault context.

23.3 Configure a remote provider

Use **openai-compatible** or **anthropic-compatible** mode with the vendor base URL and `apiKey`. Set `modelName` to the remote model ID. Keep `requireApprovalForCloud: true` (default) so non-public context triggers an approval item before the request leaves your node.

23.4 Configure LiteLLM

litellm mode targets a LiteLLM proxy (typically `http://127.0.0.1:4000/v1`) that fans out to many backends. Set `modelName` to the LiteLLM route name and supply the proxy API key if required. This is the flexible choice when one home node should switch models without editing EnvoyMesh config.

23.5 Choose a default model

Pick one native model for chat drafts and auto-replies; OpenClaw manages its own model separately in OpenClaw settings. Prefer a fast, inexpensive model for drafts and a stronger model (local or proxied) for Assistant if you split configs. Document your choice in the profile README so restores on a new machine stay consistent.

23.6 Configure fallback behavior

When native mode is **disabled**, drafts and assist features return errors instead of calling a model. When OpenClaw is down, Assistant turns degrade to the native provider automatically. For LiteLLM or cloud endpoints, verify fallback routes inside LiteLLM itself—EnvoyMesh does not chain multiple native providers in one request.

23.7 Context-window considerations

Large vault RAG injections and long Team-job prompts consume context quickly. The semantic firewall caps prompt size at 48K characters for native calls. Trim Knowledge Base `maxChunks` or lower per-contact syndication ceilings when you see truncated answers. OpenClaw session memory is separate—very long Assistant threads may need manual session reset.

23.8 Provider privacy

mock mode never calls an external network—useful for tests. **ollama** and local LiteLLM keep bytes on LAN. Cloud modes send prompt text to the configured vendor; pair with sensitivity labels and `requireApprovalForCloud` so private notes do not leave without explicit consent. OpenClaw's own model calls follow OpenClaw config, not the native router.

23.9 Cost controls

Team jobs and competitive award modes track spend in mandates; set `maxCost` and rebalance policies under chain defaults. For chat, prefer local models for high-volume auto-replies and reserve cloud models for occasional Assistant turns. Review Activity for correlated cloud calls after enabling auto-send rules.

23.10 Troubleshoot model calls

Empty or rejected prompts usually mean semantic-firewall validation failed—check for control characters or excessive length. Connection errors on Ollama/LiteLLM point to wrong `endpoint` or a stopped service. Persistent cloud denials often mean an approval is pending: open Approvals before retrying. Set mode to **mock** temporarily to confirm the agent loop runs without external dependencies.

24. Agent Style, Mode, and Contact Behavior

24.1 Agent communication style

Under Settings → AI → Identity, choose **transparent** (default), **invisible**, or **defensive** presentation. Transparent replies openly as an AI; invisible drafts as if you typed them (still signed with agent role on the wire); defensive acts as a gatekeeper when you appear offline. Optional `debugPrefixInMessageText` adds a prefix in logs only—Social hides it in the UI.

24.2 Agent operating modes

Global defaults live in `aiSettings.defaultModeForNewContacts`: **manual** (draft only), **assistant** (suggest + confirm), or **auto** (send when policy allows). Online/offline behavior is controlled separately: `onlineAssistantEnabled` keeps suggestions while you are active; `offlineAgentEnabled` permits auto-reply when the node thinks you are away. Set `statusMode` to manual if automatic presence detection misreads your schedule.

24.3 Per-contact modes

Each contact can override global defaults with `aiAccessLevel`: **none**, **assistant_only**, or **full**. None blocks AI participation for that peer; assistant_only allows drafts and gated sends; full enables richer automation including rule triggers. Set these from the contact detail sheet or via `mesh.set-contact-mode` during agent-assisted setup.

24.4 Per-contact disclosure rules

`knowledgeAccess` caps what vault material the agent may cite for a contact (`public`, `friends`, `trusted`, or `private`). Optional `syndicationMaxSensitivity` tightens inbound answers you syndicate to that peer. `disclosure` settings (badges, collapse peer agent to contact) are local UI only—they do not change wire payloads. Align disclosure with trust tier before enabling auto-send.

24.5 Social proxy behavior

Social proxy (requires Trust mode) lets EnvoyAI mediate intros and standing social workflows under a signed mandate. Enable `socialProxyEnabled` only after `trustModeEnabled` is on and you have configured a mandate ID. The orchestrator respects `autonomousKillSwitch`—when kill switch is on, proxy passes stop even if the feature flag is set.

24.6 Proactive check-ins

Proactive behavior combines AI rules, triggers, and friend autopilot (`friendAutopilotEnabled`). Rules match greetings, keywords, or contact access levels and choose draft, auto_send, gatekeep, or defer actions. Rate limits (`autoReplyLimits`) cap hourly and daily auto-replies per contact so a single thread cannot spam while you are away.

24.7 Pause or restrict automation

Toggle **autonomousKillSwitch** for an immediate global pause—every autonomous action becomes an approval. Pause individual triggers from Settings or `mesh.update-trigger`. Lower a contact to **assistant_only** or **none** to stop auto-send for one relationship without disabling EnvoyAI entirely.

24.8 Reset agent behavior

Clear AI rules, reset contact preferences to defaults, and turn off social proxy and autopilot flags in Settings → AI. Restart OpenClaw if session tone drifted across long threads. For a hard reset, disable EnvoyAI, clear pending approvals you no longer need, re-enable, and re-test with a single bonded contact at **manual** mode.

25. Sessions and Memory

25.1 What a session is

An EnvoyAI session binds your ongoing Assistant conversation to OpenClaw's memory store via a stable `sessionId`. Owner turns in Social's EnvoyAI chat, `@envoy` mentions, and terminal-correlated plans share this binding so follow-up questions stay coherent. Sessions are local to the home node—not replicated to EnvoyGo except through live RPC.

25.2 Conversation context

Each OpenClaw request carries owner interests, bonded contact names with trust levels, and the exported tool catalog. Native chat drafts use a slimmer context window through the model router. Correlation IDs in audit logs stitch a single turn across tool calls—use them when reviewing Activity after a complex exchange.

25.3 Short-term and long-term memory

OpenClaw retains short-term thread state inside the active session and longer recall through its own memory subsystem (including optional MCP bridges like Memex when configured). EnvoyMesh does not duplicate that long-term store in the vault by default. Treat OpenClaw's workspace and memory plugins as the source of truth for "what the assistant remembers."

25.4 Search memory

Use OpenClaw-facing tools or configured MCP search (`memex_search` by default in Knowledge Base settings) to query external memory indexes. Inside EnvoyMesh, `mesh.chat_rag_search` retrieves indexed chat and library snippets for agent turns. Results inherit sensitivity labels—do not expose private RAG chunks to public contacts.

25.5 Session summaries

Call `mesh.session-summary` or list sessions via `mesh.list-sessions` to inspect OpenClaw thread metadata without opening the gateway UI. Summaries help before handing off a task to Team jobs or filing audit notes. They are operator-oriented views, not wire messages to contacts.

25.6 Correct outdated memory

When OpenClaw states a stale fact, correct it in the Assistant thread and, if using Memex or similar, update or archive the source card. Adjust Library notes that fed RAG so the next `mesh.chat_rag_search` returns current text. Per-contact preferences may also need updating if the error involved disclosure scope.

25.7 Delete memory

Revoke external memory entries through the MCP tool's archive/delete path configured in Knowledge Base settings. Clear OpenClaw session state by starting a new session ID (restart gateway for a full wipe). Removing local chat logs does not erase OpenClaw memory until you delete on that side too.

25.8 Retention and privacy

Session and memory data live under your profile directory and OpenClaw workspace paths with `0600` file modes. Back up the profile before OS migration. Cloud memory plugins follow their vendor retention—disable them for air-gapped deployments.

25.9 Memory across devices

EnvoyGo displays live Assistant replies from the home node but does not host OpenClaw memory locally. All persistent recall stays on the home machine where the gateway runs. Pairing a new phone does not copy session history unless you restore the home profile.

25.10 Current chat-history integration boundaries

Full recent-chat injection into every OpenClaw turn is not complete—bonds and interests attach reliably; verbatim thread scrollback may be partial. Native auto-replies use current message text only. Plan important continuity by referencing Library notes or explicit summaries in your prompt until chat-log integration ships.

26. Tools

26.1 What an agent tool is

A tool is a named, schema-described action the agent can invoke—send chat, query knowledge, list approvals, etc. EnvoyMesh registers tools in `ToolRegistry`, evaluates bond policy and sensitivity, then executes or queues approval. Every invocation produces an audit event with tool name, latency, and correlation ID.

26.2 Browse available mesh tools

In Social, open Settings → AI → Tools (or ask EnvoyAI to list tools). CLI and bridge clients can call `mesh.mcp.list_tools` when MCP proxying is enabled. The startup catalog exported to OpenClaw mirrors the same names— `mesh.*` prefix for mesh operations, plus standard chat/knowledge entries.

26.3 Knowledge and Library tools

Use `mesh.library_list`, `mesh.library_read`, `mesh.library_discover`, and `mesh.chat_rag_search` to read local notes and query indexed content. `mesh.knowledge.query` (and task variants) reaches bonded peers' public or permitted indexes. Sensitivity ceilings on each tool prevent exfiltrating private vault paths to strangers.

26.4 Contact and messaging tools

`chat.send` and mesh discovery/hello tools let the agent find contacts and draft messages. Sends to non-trivial sensitivity usually enter the approval queue rather than delivering immediately. Trust intro tools (`mesh.intro.*`) appear only when Trust mode is enabled on the node.

26.5 File-sharing tools

Sharing flows through `mesh.share_propose`, `mesh.library_request_share`, `mesh.transfer_status`, and gallery helpers. Raw file transfer above policy ceiling requires owner approval and explicit peer accept. Check `mesh.share_list_pending` before assuming a transfer completed.

26.6 Task and Agent Network tools

`mesh.task.propose`, `mesh.task.await_result`, and `mesh.capability_provider.start` participate in peer tasks and Team jobs. Agent card tools (`mesh.agent_card.request`, `mesh.list_agent_network_workers`) support worker discovery. Competitive award flows may enqueue `chain_award` approvals when spend or bid rules trigger.

26.7 Approval and escalation tools

`mesh.list-pending`, `mesh.approve`, `mesh.reject`, `mesh.reject-all`, and `mesh.escalate` let the agent surface work to you or pause when uncertain. Prefer escalation over silent failure when confidence is low or sentiment is negative. The agent should not approve its own queued items unless policy explicitly allows auto-resolution.

26.8 MCP tools

`mesh.mcp.list_tools` and `mesh.mcp.call_tool` proxy to configured MCP HTTP servers (for example Memex). Each call inherits the same approval and audit path as native tools. Register only MCP servers you trust— they execute with the node's local network access.

26.9 Enable or disable access

Disable Trust intro tools by turning off `trustModeEnabled`. Pause MCP servers in Knowledge Base settings. Use `autonomousKillSwitch` to block execution of autonomous tool chains without removing the catalog. Bridge agents receive a filtered mesh tool list via the HTTP bridge— not the full registry.

26.10 Review tool executions

Open Activity and filter by tool or correlation ID. Each row shows allow/deny, remote peer, and summary text. For bridge traffic, also check `mesh.list-external-agent-actions`. Cross-check pending approvals if a tool returned "queued" instead of `ok: true`.

27. Triggers, Schedules, and Digests

27.1 Create a trigger

Triggers live in the node trigger store and fire proactive actions. Create time-based (cron, interval, or one-shot), event-based (message received, contact online/offline), or topic-based (keyword match) triggers from Settings → AI → Automation or via `mesh.add-trigger`. Each trigger declares an action type—send chat, query knowledge, send digest, notify owner, or follow up—and a daily fire cap.

27.2 Update or remove a trigger

Edit conditions or pause a trigger with `mesh.update-trigger`; delete with `mesh.remove-trigger`. Paused triggers retain history but do not fire. After changing cron expressions, confirm the next scheduled time in the automation panel so timezone mistakes do not surprise you.

27.3 Schedule reminders and actions

Time triggers accept cron strings, ISO `at` timestamps, or `intervalMs` for repeating checks. The node evaluates due triggers on its periodic loop and records `trigger.fired` audit events. Chat sends from triggers still pass approval policy—high-risk templates queue instead of auto-sending.

27.4 Configure activity digests

Digest settings (`mesh.set-digest-schedule`, `mesh.get-digest-config`) control **daily**, **weekly**, or **off** summaries written under your profile `digests/` directory. Toggle sections: external agent calls, discovery queries, bond changes, proactive actions, and pending approvals. When a digest is ready, Social emits a `digest:ready` event you can open from Activity.

27.5 Morning reports and discovery summaries

Morning report (`getMorningReport`) ranks recent discovery events and trust-store signals—a separate, on-demand discovery digest from periodic activity digests. Run it from Social discovery panels or CLI `morning-report` when evaluating new public peers. It does not send mesh messages by itself.

27.6 Follow-ups and proactive checks

Follow-up actions re-open a contact thread after delays you define in trigger metadata. Proactive check-ins combine offline detection (`offlineAgentEnabled`) with rules and triggers— for example, defer a draft when sentiment is negative. Escalations from proactive passes appear in Approvals with `proactive_checkin` or `follow_up` action types.

27.7 Quiet hours and notification preferences

Per-domain **agent visibility** (`instant`, `brief`, `silent`, `approval`) controls push noise for tasks, intros, and reports without stopping the underlying automation. Use **silent** overnight and **approval** during focus blocks so only approval-needed events interrupt you. This is notification loudness, not a separate cron quiet-hours clock—combine with paused triggers for true blackout windows.

27.8 Review automation history

Filter Activity for `trigger.fired`, digest generation, and proactive agent events. Each entry includes trigger name, action type, and correlation ID. Compare against `mesh.list-triggers` status fields (`firesToday`, `lastFiredAt`, `lastError`) when a schedule misfires.

27.9 Stop an automation

Hit **autonomousKillSwitch** to halt all proactive firing immediately. Individually disable triggers, turn off `offlineAgentEnabled`, or set digest frequency to **off**. Cancel in-flight proactive chat by rejecting the approval item before it expires.

28. Approvals and Escalations

28.1 Why EnvoyMesh asks for approval

Approvals enforce owner consent for actions that exceed bond tier, sensitivity ceiling, or autonomous policy: outbound chat drafts, knowledge shares, cloud model calls, discovery forwards, digests, and Team-job awards. The queue is the control surface between agent intent and mesh execution—nothing in the pending list has been sent yet.

28.2 Review a pending action

Open **Approvals** in Social or call `listPendingApprovals` from CLI. Each item shows title, draft content, action type, priority, and request timestamp. Read the draft as if it would send verbatim—edits after approval are not automatic unless you reject and ask the agent to regenerate.

28.3 Check the contact, data, and capability scope

Inspect context fields: contact owner ID, display name, sensitivity level, requested capabilities, and linked trigger name if automation-fired. Confirm the recipient matches your intent and the sensitivity label fits the relationship tier. Reject if the agent requested private data for a public or referred contact.

28.4 Approve an action

Approve from the Approvals panel or CLI approve command; the executor runs the underlying tool or send path and marks the item resolved. Approved sends propagate as normal signed envelopes. Cloud model approvals unblock the specific native router call tied to the item.

28.5 Reject one or all actions

Reject with an optional note so audit shows owner intent. `mesh.reject-all` clears the queue when you distrust a batch—for example after a misconfigured auto rule. Rejection does not penalize the contact; it only blocks that draft.

28.6 Escalation reasons

Items escalate to **escalated** status when confidence is below 0.6, sentiment is negative, or sensitivity score exceeds the threshold. Manual escalation via `mesh.escalate` flags thorny threads for owner attention even when policy might allow auto-send. Escalated items stay visible until acknowledged.

28.7 Acknowledge an escalation

Use **Acknowledge** in Approvals or `mesh.acknowledge-escalation` after you have read the context—even if you reject the underlying action. Acknowledgment clears urgent signaling without approving the draft. Pair with a contact mode change if the peer should stay on manual assist going forward.

28.8 Expired approvals

Pending items expire after seven days by default; expired entries cannot be approved without a new agent request. The node periodically purges expired IDs and logs the count. If you routinely miss the window, switch risky contacts to manual mode and raise visibility to **approval** only.

28.9 Agent Network award approvals

Competitive Team jobs may enqueue `chain_award` items when a worker bid needs owner sign-off on spend or selection. Review bid price, worker identity, and mandate budget before approving. Direct award mode skips bidding but still respects mandate `maxCost`.

28.10 Avoid approval fatigue

Start new contacts in **manual** mode, enable auto-send only for trusted peers, and use autonomous policies with tight sensitivity ceilings. Prefer **brief** or **approval** agent visibility so low-value activity does not ping you. Audit weekly: if the same rule generates noise, pause the trigger or narrow its keywords.

Part V — Knowledge, Library, and the Web

29. Knowledge System Overview

29.1 Knowledge Base, Library, Vault, and RAG

The Knowledge Base is the user experience, the Library organizes discoverable items, the Vault stores local files, and RAG retrieves relevant chunks for an agent prompt. These layers work together but have different security and lifecycle responsibilities.

29.2 Local-first storage

Your vault files and profile metadata live on the home node's disk first—typically under the profile's vault directory with `notes/`, `documents/`, `inbox/`, and `.envoy/` metadata. Nothing requires a cloud sync service for ordinary reading or editing in Social desktop. Paired EnvoyGo reads and writes through home RPC; it does not hold a full vault replica on the phone by default.

29.3 Notes, files, and structured information

Markdown notes are created in the Library UI and stored under `vault/notes/` with optional subfolders you define. Imported PDFs, Word files, images, and plain text land in `documents/` or legacy vault paths and join the same index. Structured `.envoy/sensitivity.json` overrides track per-item visibility independent of folder layout.

29.4 Visibility and sensitivity

Each item carries a sensitivity tier—public, friends, trusted, or private—controlled by the Published toggle and Obsidian frontmatter when plugins are enabled. Bonds map peer trust tiers to the maximum sensitivity they may read or receive in syndicated responses. Changing sensitivity re-indexes RAG visibility without moving files on disk.

29.5 Search and retrieval

Local search scans indexed vault chunks; chat RAG retrieves the best matches to ground model answers with citations. Remote peers use `knowledge.query` for natural-language search or `library.read` for path-based byte retrieval when browsing published web content. These paths differ: search synthesizes or ranks text; library read serves files verbatim.

29.6 Trusted remote knowledge

Bonded contacts may query syndicated knowledge within ceilings you set per relationship. Strangers on the public tier can query only public notes through rate-limited `knowledge.query` and see a stripped wiki-link graph. Federated RAG merges local and remote chunks when policy allows, preserving source attribution in responses.

29.7 Provenance and hashes

Content hashes fingerprint vault bytes and appear in discovery matches, Browser verification, and IPFS export metadata. Hashes let recipients confirm they received unaltered files without trusting filename alone. Publishing updates may change bytes at a stable path; verify hash when integrity matters more than friendly titles.

29.8 Publishing and browsing

Phase 45 adds URL-addressable mesh pages under `envoy://owner/path` served from the home `web/` directory with per-entry visibility. Social Browser and paired EnvoyGo Browser render Markdown, images, and PDFs like a lightweight web client. Feeds and topic notifications (45E) alert followers when authors publish, but fetching remains pull-based via `library.read`.

29.9 IPFS integration

Optional IPFS export publishes content-addressed copies of selected Library items through Helia or Kubo integrations. CIDs complement mesh discovery but do not replace bond-gated `library.read` for authorized browsing. Treat IPFS as distribution and verification aid, not as implicit permission to ignore sensitivity labels.

30. Create and Organize Knowledge

30.1 Create a Markdown note

Open the Library tab in Social, choose New Note, and enter Markdown in the editor; saves land in `vault/notes/` automatically. The RAG pipeline re-indexes on save without restarting the node. Use `createNote` JSON-RPC when automating note creation from scripts or integrations.

30.2 Edit and preview a note

Switch between edit and preview modes to validate formatting before sharing or publishing. Preview uses the same sanitization path as chat rendering so you see roughly what bonded readers will see. EnvoyMesh does not silently rewrite note bodies except through explicit plugin import flows.

30.3 Organize folders

Create subfolders under `notes/` for research, work, or personal categories—the UI mirrors vault paths. Sensitivity remains per note, so one folder can mix public tutorials and private drafts. Obsidian users can organize the same directories externally while EnvoyMesh indexes changes on refresh.

30.4 Add files

Drag or import files into `documents/` for PDF, DOCX, images, and text formats the indexer supports. Large imports may take a moment to chunk for RAG; check Library status if search lags. Received peer files arrive in `inbox/` with separate handling from authored notes.

30.5 Choose public, friends, trusted, or private visibility

Toggle Published in the Library item editor or set Obsidian `published: true/false` frontmatter when the Obsidian plugin is enabled. Public items join the stranger-queryable mesh; friends items require at least referred bond tier; private items stay local and agent-only. Review labels before bulk imports that default to private.

30.6 Manage metadata

Titles, paths, tags, and sensitivity overrides form the metadata layer the Library displays and discovery matches against. `.envoy/sensitivity.json` persists overrides across restarts. Avoid hand-editing metadata files while the node is running unless you follow operator backup procedures.

30.7 Use the Obsidian integration

Enable the Obsidian plugin under Settings → AI → Knowledge Base → Plugins, then point Obsidian at your vault directory for rich editing. The plugin parses frontmatter, builds a wiki-link graph, and strips private links from stranger-facing responses. EnvoyMesh never writes Obsidian files directly—all mutations go through Social or RPC.

30.8 Import and export content

Export notes for offline archives or import markdown batches during migration from other tools. Verify sensitivity labels after import because external tools may not understand EnvoyMesh tiers. Keep a filesystem backup before bulk delete or path rewrites that could orphan index entries.

30.9 Delete knowledge safely

Delete removes vault files and index entries together when using Library delete actions or `deleteNote` RPC. Published web manifest entries may need separate unpublish steps if the item was exposed at an `envoy://` path. Confirm no bonded peers rely on syndicated copies before deleting authoritative originals.

31. Search and RAG

31.1 Search your local knowledge

Use Library search for keyword retrieval across indexed vault chunks on your home node. Results show matching excerpts with paths so you can open the source note or document. Search respects sensitivity—you will not see private chunks in contexts meant for strangers.

31.2 Ask EnvoyAI to search

Ask EnvoyAI in chat to find information; it invokes RAG tools that retrieve vault chunks before answering. Answers should cite paths or titles when attribution is enabled in your configuration. Remote model calls still pass through the semantic firewall and bond checks on outbound context.

31.3 Understand chunks and matches

RAG splits documents into chunks for embedding and retrieval; matches are ranked by relevance to your query. Chunk boundaries may split paragraphs, so read surrounding context in the source file when precision matters. Re-indexing after large edits refreshes chunk boundaries automatically on save.

31.4 Review source attribution

Review citations in chat or knowledge responses to confirm which note or file supplied each claim. Federated results include remote owner identifiers so you know whether text came from your vault or a peer's syndicated library. Save attributed excerpts with MCP write-back when you want a durable local copy.

31.5 Chat RAG search

Chat RAG runs during assistant turns, combining retrieval with model generation in one flow. It differs from manual Library search because the model synthesizes an answer grounded on retrieved chunks. Disable or narrow tools if you prefer search-only interactions without generative summarization.

31.6 Federated RAG across trusted contacts

Federated RAG queries opted-in contact libraries within syndication ceilings configured under trust settings. Private notes never leave your node; friends-tier material requires sufficient bond tier on both sides. Conflicting facts from multiple peers should be resolved by reading originals, not trusting merged summaries alone.

31.7 Handle conflicting results

When local and remote chunks disagree, open each cited source and compare hashes or timestamps. Models may over-merge paraphrases; treat RAG output as a map to evidence, not as authority. Adjust syndication settings if a contact's automated summaries are consistently misleading.

31.8 Save useful results

Use MCP write-back or manual note creation to store useful query results in your vault with default friends sensitivity. Include source peer and query text in the note body for future audit. Avoid saving strangers' private-leak attempts—verify sensitivity before publishing saved summaries.

31.9 Protect sensitive information

Keep credentials, medical, and legal material at private sensitivity unless you explicitly accept broader exposure. Public mesh queries rate-limit strangers and strip non-public wiki links from responses. Audit knowledge queries periodically if you syndicate friends-tier content to referred contacts.

32. Trusted Knowledge Sharing

32.1 Ask a bonded contact for knowledge

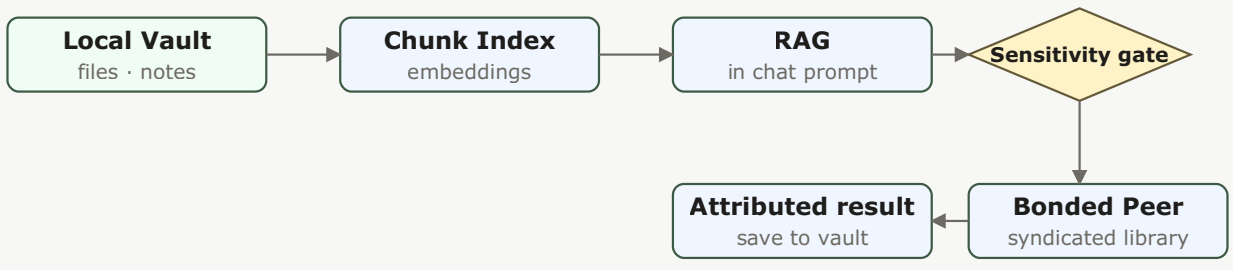

Figure 16 — Federated RAG: local Vault chunks feed RAG directly; the federated path queries bonded peers' libraries through a per-contact sensitivity ceiling gate, returning attributed results.

Send a `knowledge.query` intent to a bonded contact's agent when you need their syndicated library summarized or searched. The remote node applies its bond tier, syndication ceilings, and model routing before answering. Ask precise questions and expect natural-language responses, not raw file dumps.

32.2 Public, referred, and direct access

Public tier allows stranger queries with tight rate limits; referred tier unlocks broader syndicated access; direct tier allows friends-sensitivity sharing. Each tier maps to deterministic bond policy decisions logged in audit. Upgrade bonds deliberately—referred access exposes more than public ping alone.

32.3 Share a note or file

Share notes or files by sending chat attachments, data-transfer vouchers, or publishing with appropriate visibility. Vouchers copy bytes into the recipient inbox; publishing exposes metadata via discovery or bytes via `library.read`. Pick the mechanism that matches whether you want a point copy or ongoing browse access.

32.4 Propose a share

Propose shares through task or chat flows when your workflow requires explicit recipient acceptance. Proposals carry sensitivity hints so recipients know what they are importing before indexing. Cancel proposals that stall to avoid ambiguous half-shared state.

32.5 Accept a share request

Accept inbound share requests only after verifying sender bond tier and described sensitivity. Imported content lands in vault inbox or library lists with attribution to the remote owner. Re-index or adjust sensitivity if you intend to re-share material further.

32.6 Sensitivity enforcement

The Bonds engine denies requests that exceed allowed sensitivity for the sender's trust tier, even when users believe they are friends. Syndication max settings cap what leaves your node during automated queries. Test with a secondary contact account if you tune syndication for a research group.

32.7 Contact-scoped discovery

Contact-scoped discovery returns published library metadata for bonded peers without exposing private paths. Matches include titles, hashes, and optional CIDs—not full text until a follow-up read or query. Use scoped discovery before wide searches to respect relationship boundaries.

32.8 Network-wide document discovery

Network-wide document discovery advertises public published capabilities on the DHT for strangers meeting capability and rate rules. It supports finding public material, not enumerating private vaults. Operators should monitor audit for unusual query volume from public peers.

32.9 Rate limits and abuse protection

Stranger `knowledge.query` traffic is rate-limited (on the order of a few queries per minute and tens per hour in default configuration). Abuse protection complements bond denials to reduce scanning of public notes. Report persistent abuse by blocking public-tier peers.

32.10 Prevent unintended disclosure

Double-check Published toggles before promoting notes to public or friends tiers, especially after Obsidian sync. Web manifest visibility uses separate ACL fields—including contact pickers for contacts-only pages. Anti-enumeration returns `not_found` for unauthorized `library.read` attempts rather than confirming hidden paths exist.

33. Publish and Browse Mesh Content

33.1 The `envoy://` address format

Mesh content URLs follow `envoy://envoy:owner:<id>/path/to/page` using permanent owner IDs, not display names. `@handle` syntax parses but is rejected at runtime until a future registry ships. Pairing URIs (`envoy://contact?...`) remain distinct and must not be confused with content URLs.

33.2 Open a mesh page

Open a mesh page from chat links, Browser address bar, or feed notifications in Social desktop. Paired EnvoyGo forwards `library.read` through the home node, so browsing away from home requires connectivity to that node. Pages render Markdown, images, and PDFs with sanitized HTML where applicable.

33.3 Navigate history

Browser back, forward, reload, and per-owner history behave similarly to a conventional web client within mesh constraints. Large binary bodies may arrive in ranged chunks with hash verification at display time. Navigation guards prevent overlapping in-flight reads from corrupting the view state.

33.4 Create bookmarks

Bookmark frequently visited `envoy://` pages per owner in Browser; autocomplete suggests recent paths as you type. Bookmarks stay local to your client profile—they do not sync through a central server. Export bookmarks manually if you rebuild a device.

33.5 Browse an author

Browse an author's site by opening their owner root URL, which serves `index.md` when present under `web/`. Blog, profile, PhotoWall, and Bazaar templates organize paths by convention, not hard schema. Visibility still applies per file—seeing an index does not imply access to every subpath.

33.6 Browse Bazaar content

Bazaar and feed views aggregate discoverable public or bonded content depending on manifest and topic subscriptions. Topic follow (Phase 45E) helps match interests without GossipSub push fanout on the wire. Discovery lists metadata first; opening an entry triggers `library.read` for bytes.

33.7 Publish a page

Author pages in Social place Markdown or media into `~/EnvoyMesh/web/` (or profile-equivalent) and register manifest entries with visibility. Choose public, bonded, or specific-contact ACL before sharing URLs in chat. Updating content changes bytes at the same path—notify followers via feeds when updates matter.

33.8 Follow feeds and topics

Follow feeds and topics to receive inbox notifications when authors publish matching material. Notifications link into Browser with the originating `envoy://` URL. Unfollow topics you no longer want to avoid notification noise.

33.9 Update published content

Edit source files locally, bump manifests, and republish when correcting typos or replacing media. Clients verify `contentHash` when reloading to detect changes since last visit. There is no built-in version history URL—keep vault git or snapshots if you need rollback.

33.10 External HTTP gateway — planned

Planned. The `envoy://` mesh-content path is available today; a public HTTP gateway for non-mesh browsing is forward-referenced as Phase 45F and is not part of the current release.

33.11 The Content tab — Feed, Blog, and Explore

The **Content** tab in Social and EnvoyGo is the user-facing home for everything described in 33.1–33.9. It surfaces `envoy://` mesh content through three sub-tabs:

- **Feed** — a chronological social feed of posts and updates from the authors and topics you follow (§33.8)
- **Blog** — long-form posts built on the publishing templates in §33.7, read in a dedicated reader
- **Explore** — discovery of public and bonded authors, topics, and Bazaar listings (§33.6)

Everything you read here is content-addressed and delivered peer-to-peer from your bonded contacts' nodes — there is no central content server. Open any item to read it through the mesh Browser (§33.2) or `library.read`, depending on format. The Content tab does not introduce a new delivery path; it is the aggregated view of the content system already documented in this section.

33.12 Feed and Blog — read and post

Feed aggregates short posts and shared items from the authors and topics you chose to follow in §33.8. New posts arrive as signed publish events; your node verifies the author, checks sensitivity against the bond tier, and renders them inline. Browse, like, and comment — comments are themselves signed mesh messages, so the conversation is auditable and stays within your trust boundary. There is no algorithmic ranking: Feed is chronological, and you control what appears by choosing whom and what to follow.

Blog is for long-form content. Use the publishing editor (§33.7) to draft a post with a title, body, and optional cover media; choose a visibility tier (public, friends, trusted) and publish. Your post lives on your node as an `envoy://` page and syncs to bonded peers on their next fetch. Followers see it in their Feed; everyone else can find it through Explore. Notifications reuse the follow-feed mechanism in §33.8 — no separate push channel is needed for mesh delivery. Edit source files and republish (§33.9) to correct typos or replace media; clients verify `contentHash` on reload.

33.13 Explore — discover mesh content

Explore is the discovery layer for mesh content. It lists public and bonded authors, trending topics, and Bazaar offerings (§33.6) as metadata-first cards — title, author, sensitivity, and content hash — so you decide what to fetch before any bytes transfer. Selecting a card opens the full content through the mesh Browser (§33.2) or `library.read` depending on format, after the usual bond and sensitivity checks. This metadata-first design keeps bandwidth and exposure minimal: you see what exists without automatically pulling it.

Explore does not bypass trust. Public-stranger content is still gated by your public sensitivity ceiling, referred authors surface through your bond graph, and blocked authors never appear. Treat Explore as a directory of the mesh, not a feed — it shows what exists on reachable nodes, not what has been pushed to you.

34. IPFS and Content Verification

34.1 Why EnvoyMesh uses content hashes

Hashes identify content independently of filenames so recipients detect tampering after transfer or IPFS fetch. EnvoyMesh surfaces hashes in discovery, Browser, and export dialogs. Treat hash mismatch as a hard stop before trusting quoted text or binaries.

34.2 Export Library content to IPFS

Export selected Library items to IPFS when you want content-addressed sharing outside immediate mesh pull paths. Export respects sensitivity—do not pin material you would not publish at the same visibility tier. Record CIDs alongside mesh URLs when sharing with hybrid audiences.

34.3 Helia integration

Helia integration embeds a lightweight IPFS node suitable for desktop home nodes exporting or verifying CIDs. Configure Helia when you need in-process pinning without a separate Kubo daemon. Monitor disk use because pinned blocks accumulate locally.

34.4 Kubo integration

Kubo integration targets operators who already run a Kubo daemon and want EnvoyMesh to interoperate with its API. Point settings at your local Kubo endpoint and verify connectivity before bulk export jobs. Kubo and Helia are alternatives—typically enable one strategy per node.

34.5 Verify content through a gateway

Public gateways help humans fetch IPFS CIDs through HTTPS for verification, but gateways are not authorization layers. Compare gateway bytes to expected mesh hashes before treating content as authentic. Sensitive material should not rely on public gateways for access control.

34.6 Pinning and availability

Pinning keeps IPFS blocks reachable; unpinned CIDs may disappear when no peer hosts them. Mesh `library.read` remains authoritative for authorized live reads from the owner's home node. Use pinning for archival redundancy, not as a substitute for vault backups.

34.7 Privacy considerations

Publishing to IPFS or public mesh tiers exposes bytes to anyone who obtains the CID or URL regardless of friendly filenames. Private vault material should stay off export and unpublish lists. Review Phase 44 stranger-query behavior before marking research notes public.

34.8 Filecoin persistence — deferred

Deferred. Helia and Kubo IPFS paths are available today; Filecoin-based long-term persistence is designed but not part of the current release. See Appendix J.9.

35. Back Up and Restore Knowledge

35.1 What to back up

Back up owner keys, vault directories, `.envoy/` metadata, web manifests, profile JSON state, and audit journals you need for compliance. Library UI state alone is insufficient without underlying vault files. Document your backup schedule alongside relay or model credentials stored outside EnvoyMesh.

35.2 Back up the Vault

Copy the entire vault tree—including `notes/`, `documents/`, `inbox/`, and `.envoy/`—while the node is stopped or quiesced to avoid partial files. Verify free space before large copies. Encrypt backups at rest if they contain friends-tier or private material.

35.3 Back up Library metadata

Library metadata such as sensitivity overrides and published flags lives under `.envoy/sensitivity.json` and related stores—include these with vault backups. Published web manifests under the profile directory should backup with `web/` content. Missing metadata restores files but may wrong-foot visibility until repaired.

35.4 Restore on the same node

Restore vault and profile data into the same profile path, then restart the node and run index refresh if search seems stale. Confirm bond and trust stores if you restored partial profile trees. Test one private and one public query before returning to production use.

35.5 Move to another computer

Moving to a new computer requires copying profile, vault, and owner key material, then reinstalling EnvoyMesh and re-pairing EnvoyGo devices. Update relay bootstrap or port settings if network layout changed. Revoke old device certificates if the old hardware is discarded.

35.6 Verify restored content

After restore, spot-check note hashes, open sample `envoy://` pages, and run Library search for known keywords. Federated queries to contacts should still work once bonds reload. Log discrepancies before deleting the old machine's backup.

35.7 Mobile data boundaries

EnvoyGo does not replace the home vault on the phone—it caches only what paired RPC sessions fetch for UI display. Mobile backups mean backing up the home node your phone pairs to, not expecting full vault export from EnvoyGo alone. Re-pair QR codes after home restore if session tokens invalidate.

35.8 Repair damaged local data safely

If index corruption occurs, stop the node, restore vault from backup, and allow re-indexing rather than deleting unknown files blindly. Use audit logs to identify which operations preceded corruption. Contact operator documentation before running manual JSONL edits on metadata stores.

Part VI — External Agents

36. External Agent Overview

36.1 What an external agent is

An external agent is a separately running assistant that receives selected messages and invokes allowed mesh tools through EnvoyMesh's local HTTP bridge. HomeClaw, Hermes, and OpenHuman use the shared `envoymesh-message` contract.

36.2 Built-in EnvoyAI versus an external agent

EnvoyAI/OpenClaw is bundled, deeper, and managed with the home runtime. External agents are compatibility integrations with independently maintained agent-side code and should be enabled only when you trust that process.

36.3 Why external agents use a bridge

The bridge converts between plain HTTP requests and signed mesh operations. This keeps networking keys, bond checks, capability limits, and audit records inside EnvoyMesh.

36.4 Why external agents never receive raw P2P access

Raw libp2p access would let an agent evade identity and policy boundaries. The bridge exposes intentional operations instead, such as sending a message, finding knowledge, or executing an approved tool.

36.5 External-agent identity

The bridge agent has its own mesh peer identity derived from the owner mandate, distinct from the external runtime's internal user or session IDs. Bonded peers message that bridge identity; EnvoyMesh signs outbound replies on its behalf.

36.6 Available bridge tools

When enabled, compatible agents may call `GET /bridge/list-tools` and `POST /bridge/execute-tool` on port 3031. The catalogue reflects bond policy, mandates, and owner approvals—not every tool on the home node.

36.7 Sessions and action history

External-agent sessions and action history appear in Settings for review. Each inbound mesh message and tool invocation is correlated in the audit JSONL so you can trace what the bridge forwarded to the external process.

36.8 Permissions, approvals, and revocation

High-risk mesh operations may still require owner approval even when the bridge forwards the request. Revoke access by disabling the bridge, clearing the active preset, or rotating the Bearer secret shared with the external agent.

36.9 One active external-agent URL per bridge

A bridge resolves one active external-agent URL at a time. You may retain several presets, but switch the active preset rather than sending the same inbound event to several assistants and creating duplicate replies.

36.10 Choose an integration

Pick one integration path: bundled EnvoyAI (OpenClaw on port 18789), HomeClaw (8010), Hermes (8020), OpenHuman (8021), or a custom `envoymesh-message` adapter. Only one `agentUrl` is active per bridge profile at a time.

37. The Safe Agent Bridge

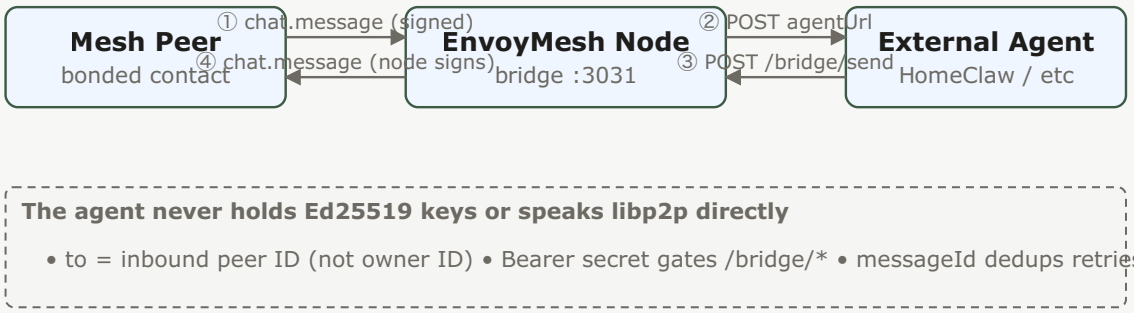

Figure 7 — External agent bridge: mesh traffic flows through the EnvoyMesh node, which signs on the agent's behalf. The agent receives plain HTTP and replies via /bridge/send.

37.1 Mesh-to-agent message flow

When a bonded peer messages the bridge agent, EnvoyMesh validates the signed envelope and POSTs a compact message object to the configured agent URL. The object includes sender routing information, display context, text, and a unique message identifier.

37.2 Agent-to-mesh reply flow

The external agent replies by POSTing `{ to, text }` to `/bridge/send` on the local bridge, normally port `3031`. The `to` value is the inbound mesh peer ID, not the owner ID; EnvoyMesh signs and sends the outbound envelope.

37.3 Bearer-token authentication

Set a bridge secret so requests use `Authorization: Bearer <secret>`. Use a long random value, store it like a credential, and rotate it after suspected disclosure.

37.4 Message identifiers and duplicate protection

The inbound `messageId` lets an agent suppress repeated webhook deliveries. The OpenClaw extension also has a short content-hash fallback for older bridges, but integrations should prefer exact message-ID deduplication.

37.5 Correlation identifiers and synchronous replies

Owner-to-agent synchronous asks include a correlation ID. A matching `/bridge/send` resolves the pending local request; an unknown correlation receives a gone response so the agent can retry instead of silently losing the answer.

37.6 Async knowledge and discovery replies

Discovery and knowledge responses can arrive after the initiating tool call. Compatible agents should handle `mesh.async_reply` events and associate them with the user's ongoing context.

37.7 List and execute mesh tools

`GET /bridge/list-tools` returns the allowed tool catalogue and `POST /bridge/execute-tool` invokes a selected operation. Both remain subject to bridge authentication, tool schemas, policy, and approvals.

37.8 Propose file sharing

An agent can propose sharing a Vault item through `/bridge/agent-share-proposal`; it does not gain unrestricted filesystem access. The owner or policy path decides whether the share proceeds.

37.9 Localhost defaults and network exposure

The bridge listens on loopback by default. Do not expose port `3031` directly to a LAN or the Internet; if remote access is necessary, place an authenticated, TLS-protected proxy in front and restrict its source network.

37.10 Audit external-agent activity

Filter audit logs for bridge intents and tool executions. Look for remote peer IDs, correlation IDs, allow/deny outcomes, and latency. This is the authoritative record when disputing what an external agent did on your behalf.

37.11 Revoke an external agent

Disable **Ext Agent** in Settings, clear or change `agentUrl` in bridge config, and rotate the Bearer secret. Stop the external process so it cannot keep calling port 3031 with a stale credential.

38. OpenClaw and EnvoyAI

38.1 OpenClaw's role in EnvoyMesh

OpenClaw supplies EnvoyAI's bundled assistant runtime and also supports the canonical EnvoyMesh channel extension. That extension handles webhook messages, reply routing, mesh tools, asynchronous replies, and onboarding surfaces.

38.2 Bundled runtime and canonical EnvoyMesh extension

The packaged runtime and `OpenClawExtension/` are maintained with EnvoyMesh, which makes this integration richer than the generic compatibility presets. The extension is also installable into another OpenClaw checkout.

38.3 Automatic startup

The home node normally starts OpenClaw automatically on gateway port `18789`. If it is disabled in startup configuration, enable it and restart the node before expecting EnvoyAI responses.

38.4 Install the extension in another OpenClaw environment

To install the channel in another checkout, run `./scripts/install-openclaw-extension.sh /path/to/openclaw --with-docs`, install that checkout's dependencies, and configure its EnvoyMesh webhook and bridge secret.

38.5 Configure the EnvoyMesh channel

In OpenClaw config, register the EnvoyMesh channel with webhook path `/webhook/envoymesh` on gateway port **18789** and set `bridgeUrl` to `http://127.0.0.1:3031/bridge/send`. Match the Bearer secret to `bridge-config.json` on the EnvoyMesh node.

38.6 Send and receive mesh messages

Bonded peers chat with the bridge agent peer ID; EnvoyMesh POSTs JSON to `http://127.0.0.1:18789/webhook/envoymesh`. Replies go to `/bridge/send` with `to` set to the sender's mesh peer ID (`envoy_…`), not the owner DID.

38.7 List and execute mesh tools

The OpenClaw extension exposes `envoymesh_list_mesh_tools` and `envoymesh_execute_mesh_tool`, which proxy to the bridge on 3031. Tool calls still pass bond checks and semantic firewall rules on the home node.

38.8 Handle asynchronous mesh replies

Discovery and knowledge responses may arrive asynchronously. The extension handles `mesh.async_reply` POSTs to the webhook and surfaces them as in-channel messages so the model can continue the conversation.

38.9 Use onboarding and setup surfaces

Run `openclaw onboard` or use the bundled Social setup flow to seed workspace, bridge secret, and channel docs. Confirm the gateway log registers the EnvoyMesh HTTP route before testing with a bonded contact.

38.10 Manage extensions and ClawHub

Install optional OpenClaw extensions through ClawHub or symlinks; the EnvoyMesh channel lives in `OpenClawExtension/`. macOS bundles more extensions than Windows; add others manually when needed.

38.11 macOS bundled-extension selection

The macOS DMG includes a broader set of OpenClaw extensions for an integrated experience. This increases package size but reduces post-install setup for common workflows.

38.12 Windows essential-extension bundle

The Windows installer packages the essential useful OpenClaw extensions rather than the full macOS set, keeping the bundle within practical size limits. Additional extensions can be installed separately when needed.

38.13 Migrate from Hermes

Use the bundled migration extension to import Hermes memories, skills, or credentials into OpenClaw, then point `agentUrl` from `8020/message` to the OpenClaw webhook. Back up both environments and verify imported secrets before switching production traffic.

38.14 Troubleshoot OpenClaw

If chat fails: confirm gateway on 18789, bridge log shows 3031, webhook path matches, Bearer secrets align, and `to` on replies is a peer ID. Run `npm run smoke:openclaw-bridge` from the repo for a local round-trip check.

39. HomeClaw

39.1 What the HomeClaw preset provides

HomeClaw is the default external-agent compatibility preset and conventionally receives messages at `http://127.0.0.1:8010/message`. EnvoyMesh supplies the bridge configuration; HomeClaw supplies its agent runtime and channel implementation.

39.2 Compatibility-preset status

Compatibility preset. The EnvoyMesh side is available, but verify the compatible HomeClaw release and its `channels/envoymesh` support before production use.

39.3 Start HomeClaw

Start HomeClaw so its EnvoyMesh channel listens on **8010** (default `http://127.0.0.1:8010/message`). Verify the process is bound to loopback unless you deliberately run agent and node on different hosts.

39.4 Select HomeClaw in Settings

In **Settings** → **AI** → **Ext Agent**, enable the bridge and choose the HomeClaw preset. EnvoyMesh sets `agentUrl` to `http://127.0.0.1:8010/message` and starts forwarding bonded peer chat to that endpoint.

39.5 Configure the message URL

Use the local default `http://127.0.0.1:8010/message` unless HomeClaw is intentionally bound elsewhere. Keep the endpoint on loopback whenever both processes run on the same host.

39.6 Configure the reply bridge

Configure HomeClaw to return replies to `http://127.0.0.1:3031/bridge/send`. Use the same Bearer secret on both sides.

39.7 Add a shared secret

Generate a long random secret in bridge settings and configure the same value in HomeClaw's EnvoyMesh channel. Both inbound POSTs to 8010 and outbound POSTs to 3031 should send `Authorization: Bearer <secret>`.

39.8 Send and receive messages

When a bonded contact messages your bridge agent, HomeClaw receives `{from, fromOwnerId, fromName, text, messageId}`. Replies POST to `http://127.0.0.1:3031/bridge/send` with `{to, text}` where `to` is the inbound `from` peer ID.

39.9 Use mesh tools

If HomeClaw's channel implements tool proxies, it calls list/execute endpoints on 3031. Each tool remains subject to bond tier, mandate bounds, and owner approval queues on the EnvoyMesh node.

39.10 Permissions and knowledge access

Knowledge and vault reads flow through mesh tools—not direct filesystem access. Tune HomeClaw's own permissions separately; EnvoyMesh still enforces sensitivity ceilings and contact scope on every tool call.

39.11 Agent-side channel ownership

The `channels/envoymesh` implementation lives in the HomeClaw repository. EnvoyMesh only configures URLs and secrets; upgrade or patch the channel on the HomeClaw side when wire behavior changes.

39.12 Disconnect or revoke HomeClaw

Disable Ext Agent in Settings, stop HomeClaw, and rotate the bridge secret. Clear `agentUrl` or switch to another preset so queued mesh messages are not delivered to a stopped process.

39.13 Troubleshoot HomeClaw

Common failures: HomeClaw not listening on 8010, secret mismatch, wrong reply `to` field, or bridge disabled. Check node logs for `[bridge] HTTP on …3031` and curl 8010/message with a test payload plus Bearer header.

40. Hermes

40.1 What the Hermes preset provides

Hermes is a built-in compatibility preset using the same message contract, conventionally at `http://127.0.0.1:8020/message`. Its knowledge-oriented runtime is maintained outside this repository.

40.2 Compatibility-preset status

Compatibility preset. EnvoyMesh provides selection and bridging, not a guarantee about every Hermes version. Test the exact release and configured tools before enabling it for contacts.

40.3 Start Hermes

Launch Hermes with its EnvoyMesh adapter on **8020** (`http://127.0.0.1:8020/message`). Confirm the release you run matches the compatibility preset expectations in release notes.

40.4 Select Hermes in Settings

Select the Hermes preset under **Settings** → **AI** → **Ext Agent**. EnvoyMesh points `agentUrl` at 8020 and enables the bridge listener on 3031 for return traffic.

40.5 Configure message and reply URLs

Set inbound messages to `http://127.0.0.1:8020/message` and configure Hermes to reply via `http://127.0.0.1:3031/bridge/send`. Keep both URLs on loopback for same-machine setups.

40.6 Add a shared secret

Copy the bridge secret from EnvoyMesh settings into Hermes's EnvoyMesh channel configuration. Mismatched Bearer tokens produce 401 responses on both 8020 and 3031.

40.7 Send and receive messages

Hermes receives the standard bridge payload on 8020. Outbound replies must target the sender peer ID from `from`; owner DIDs will not route correctly on the mesh.

40.8 Use knowledge and mesh tools

Hermes's knowledge-oriented tools map to mesh list/execute calls when implemented in its adapter. Vault and discovery results may return asynchronously through the same async-reply pattern OpenClaw uses.

40.9 Permissions and approvals

Hermes-side prompts and memory are outside EnvoyMesh policy. Mesh-side approvals still apply when a tool would exceed mandate cost, sensitivity, or contact scope.

40.10 Agent-side integration ownership

Hermes maintains its own integration code and release cadence. EnvoyMesh supplies the preset URLs and bridge security boundary only.

40.11 Migrate from Hermes to OpenClaw

A bundled OpenClaw migration extension can import supported Hermes configuration, memories, skills, or credentials. Back up both environments and review imported secrets before switching the active runtime.

40.12 Disconnect or revoke Hermes

Turn off the Hermes preset, rotate secrets, and stop the Hermes process. Consider migrating to OpenClaw with the migration extension if you need a supported bundled path.

40.13 Troubleshoot Hermes

Verify 8020 is reachable, secrets match, and Hermes version supports `messageId` deduplication. Inspect bridge audit events for denied tool calls versus transport errors.

41. OpenHuman

41.1 What the OpenHuman preset provides

OpenHuman is a built-in compatibility preset using the shared adapter, conventionally at `http://127.0.0.1:8021/message`. Its agent-side runtime remains an external project.

41.2 Compatibility-preset status

Compatibility preset. Verify the OpenHuman release, endpoint behavior, and consent model independently; EnvoyMesh secures only the mesh-facing bridge boundary.

41.3 Why OpenHuman is disabled by default

OpenHuman is disabled by default so installing EnvoyMesh never silently grants an unverified external process access to conversations or tools. Enable it only after configuration and trust review.

41.4 Start OpenHuman

Start OpenHuman with its adapter listening on **8021** (`http://127.0.0.1:8021/message`). Because OpenHuman is disabled by default, confirm you intentionally enabled it after reviewing its consent model.

41.5 Enable and select OpenHuman

Enable OpenHuman in bridge settings and select its preset. EnvoyMesh will not auto-start OpenHuman; both the external process and the Ext Agent toggle must be on.

41.6 Configure message and reply URLs

Configure `http://127.0.0.1:8021/message` for inbound mesh traffic and `http://127.0.0.1:3031/bridge/send` for replies. Document any non-default ports in both OpenHuman and `bridge-config.json`.

41.7 Add a shared secret

Set a shared Bearer secret in EnvoyMesh bridge settings and OpenHuman's channel config. Treat rotation like credential compromise response: update both sides before resuming traffic.

41.8 Send and receive messages

OpenHuman handles inbound `{from, text, messageId, …}` like other presets. Replies use the peer ID in `from`; duplicate `messageId` values should be ignored to absorb retries.

41.9 Use mesh tools

Tool access is limited to what the bridge exposes via list/execute on 3031. OpenHuman cannot bypass bond or mandate checks by calling libp2p directly.

41.10 Consent, privacy, and approvals

Review OpenHuman's consent prompts and data retention separately from EnvoyMesh policy. Owner approvals on the home node still gate sensitive mesh operations.

41.11 Agent-side integration ownership

OpenHuman ships its own integration layer; EnvoyMesh does not vet every agent-side behavior. Keep OpenHuman updated and disable the preset if its security posture changes.

41.12 Disconnect or revoke OpenHuman

Disable the preset, revoke the secret, and stop OpenHuman. Clearing Ext Agent returns chat to EnvoyAI or another selected engine without exposing 8021.

41.13 Troubleshoot OpenHuman

Check that OpenHuman is enabled, listening on 8021, and using matching Bearer auth. Consent or approval denials may look like transport failures—inspect audit outcomes.

42. Custom External Agents

42.1 Use the `envoymesh-message` adapter

A custom agent can implement the `envoymesh-message` adapter without speaking libp2p. It accepts the bridge's inbound JSON, replies through the local bridge, and may list or execute only the tools exposed to it.

42.2 Register a custom agent preset

Add a custom preset in bridge settings with your adapter's `agentUrl` (for example `http://127.0.0.1:9000/message`). One bridge profile points to one active URL.

42.3 Implement the inbound message endpoint

Implement `POST /your/message` accepting `{from, fromOwnerId, fromName, text, messageId}` and optional Bearer auth. Respond 200 quickly; deliver replies asynchronously via 3031 rather than echoing in the HTTP response body.

42.4 Implement replies through `/bridge/send`

POST `{to, text}` and optional `correlationId` to `http://127.0.0.1:3031/bridge/send`. Use `to` = inbound `from` peer ID. Sync asks resolve when `correlationId` matches a pending owner request.

42.5 Authenticate requests

Validate `Authorization: Bearer` on inbound mesh webhooks and on your outbound calls to 3031. Reject unsigned requests when a secret is configured.

42.6 Handle duplicate messages

Track seen `messageId` values and drop duplicates within your retry window. This prevents double replies when the bridge retries a flaky webhook delivery.

42.7 List and call mesh tools

Call `GET /bridge/list-tools` then `POST /bridge/execute-tool` with JSON arguments. Handle structured errors and approval-pending responses without crashing your agent loop.

42.8 Handle asynchronous results

Subscribe to or poll for async mesh results (`mesh.async_reply` shape when mimicking OpenClaw). Associate late discovery or knowledge responses with the user turn that triggered them.

42.9 Define capability and data boundaries

Document which mesh tools your agent may call and what local data it stores. Never request raw libp2p keys or vault paths outside approved tools.

42.10 Test the integration

Run bonded peer chat tests, tool calls, and secret-rotation drills. Use `npm run smoke:openclaw-bridge` patterns as a reference for mock round-trips.

42.11 Security checklist

Loopback bind only, strong Bearer secret, least-privilege tools, audit review, prompt secret rotation, and a documented revoke path. Do not expose 3031 to LAN/WAN without TLS and network ACLs.

42.12 Troubleshoot a custom agent

Compare bridge logs with your adapter logs for 401/404/410 responses, wrong `to` IDs, and schema mismatches. Test with curl before involving live mesh peers.

43. Manage External Agents

43.1 Review the active agent

Open **Settings** → **AI** and confirm which preset is active, its `agentUrl`, whether Ext Agent is enabled, and the bridge listen port (default 3031). Bundled EnvoyAI uses 18789 separately from Ext Agent presets.

43.2 Review external-agent sessions

Review external-agent session lists for active correlations and recent peer contacts. Sessions tie mesh senders to bridge forwarding state on the home node.

43.3 Review action history

Action history summarizes tool executions and forwarded messages. Cross-check unusual entries against audit JSONL using the same correlation or message IDs.

43.4 Inspect available capabilities

Inspect the tool catalogue via Settings or `GET /bridge/list-tools` with auth. Capabilities change when bonds, mandates, or owner approvals change—not when the external agent restarts.

43.5 Change the active preset

Switch presets by selecting HomeClaw, Hermes, OpenHuman, OpenClaw webhook, or custom. Restart or reconnect the target external process after changing `agentUrl` or secrets.

43.6 Disable the bridge

Toggle **Ext Agent** off to stop forwarding mesh chat to external URLs while keeping bundled EnvoyAI available. The bridge HTTP listener may remain for in-flight replies—rotate secrets if you need a hard stop.

43.7 Revoke an agent session

Revoke a session by disabling the bridge, clearing credentials in the external agent, and rotating the Bearer secret so old tokens cannot call 3031.

43.8 Rotate the shared secret

Generate a new secret in EnvoyMesh, update the external agent config, then restart both sides. Expect brief 401 errors until configs match.

43.9 Respond to a compromised external agent

Immediately disable Ext Agent, rotate secrets, block affected peers if needed, and review audit logs for exfiltration or tool abuse. Treat compromise of the external process as compromise of everything the bridge was allowed to do.

43.10 Collect diagnostics

Gather bridge-config (redact secrets), recent audit excerpts, gateway/adapter logs, and results of `curl` probes to 3031 and your `agentUrl`. Include EnvoyMesh and external-agent versions when filing issues.

Part VII — Agent Network and Team Jobs

44. Agent Network Overview

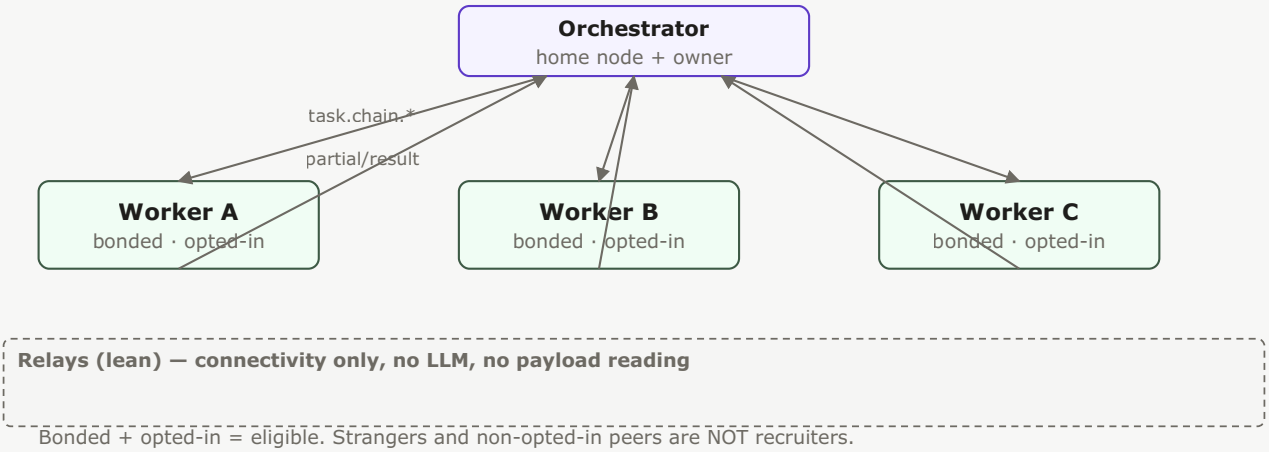

Figure 8 — Agent Network topology: the orchestrator recruits bonded, opted-in workers. Relays carry connectivity only. There is no public marketplace — strangers cannot recruit your agent.

44.1 What Agent Network means

Agent Network is the collaboration layer where bonded owners opt their local agents into work for Team jobs. It combines identity, trust, capability discovery, mandates, orchestration, artifacts, and auditing.

44.2 Bonded people and opted-in local agents

A worker is eligible only when the owners are bonded at an acceptable tier, the remote owner enabled Join Agent Network, and a fresh agent card advertises the required membership and capabilities.

44.3 Agent Network is not a public marketplace

There is no public marketplace where strangers can freely recruit your agent. Broad anonymous recruitment is deliberately outside the current product boundary.

44.4 Your agent remains private by default

The local agent remains usable by its owner whether or not it joins. Membership changes what bonded peers can discover and request, and can disclose owner-attested profile fields to those peers.

44.5 Worker membership

Worker membership is the opt-in flag (**Join Agent Network**) that advertises `capability-provider` on your agent card. Without it, bonded peers cannot recruit your agent for Team jobs even if trust is direct.

44.6 Agent cards and capabilities

An **Agent Card** lists capabilities, supported task types, optional profile fields, and membership tags. Orchestrators index cards from bonded peers to decide who can execute each subtask.

44.7 Team jobs

Team jobs (UI name for multi-agent chains) split an owner goal into subtasks, assign them to opted-in workers, and merge results into one report. Protocol code still uses `task.chain.*` intents.

44.8 Intelligent home nodes and lean relays

Home nodes run LLMs, vault access, orchestration, and worker execution. **Relays** provide connectivity and discovery only—they never execute subtasks or read private payloads.

44.9 Typical personal, family, and team topologies

Personal setups often pair two home laptops; families may add a child's node; teams use fleet manifest or LAN onboarding. Every topology still requires bonds plus worker opt-in before cross-home Team jobs work.

44.10 Current scope and future directions

Today, Agent Network covers bonded, opted-in collaboration: an owner enables Join Agent Network, bonded peers see the worker card, and the requesting node orchestrates Team jobs across those trusted workers. There is no public marketplace, no anonymous worker recruitment, and relays stay lean (connectivity only). Forward directions — broader discovery, richer reputation, multi-hop commerce, and a complete hierarchical relay graph — are documented in Appendix J.5–J.11 as Planned, Parked, or Deferred; treat them as direction, not committed release dates.

45. Join Agent Network

45.1 Prerequisites

Before joining: running home node, owner identity, configured AI engine, and at least one bond if you expect to collaborate soon. Joining alone does not create bonds automatically.

45.2 Enable Join Agent Network

Open **Settings** → **Agent Network** and enable **Join Agent Network**. The node sets capability-provider membership in its advertisements; it does not create bonds automatically.

45.3 What membership advertises

Membership advertises the `capability-provider` tag and, if configured, the Agent Network profile. Bonded peers can then index the card and consider the worker for compatible subtasks.

45.4 Turn membership off

Disable **Join Agent Network** in Settings to remove `capability-provider` from your card. In-flight subtasks may finish, but new orchestrators should stop recruiting you after refresh.

45.5 Confirm your worker is visible

On a peer's node, open **Settings** → **Agent Network** → **Workers status** and click **Refresh workers**. Your entry appears when bond trust is eligible, you joined, and a fresh card synced.

45.6 Local agent behavior when not joined

When not joined, your local agent still serves chat, vault, and personal tasks on your node. Only recruitability to bonded peers' Team jobs is withheld.

45.7 Privacy implications

Joining shares capability tags and optional profile fields with bonded peers—not the public internet. Strangers cannot browse your agent; only contacts who already passed bond policy see recruitability signals.

45.8 Troubleshoot membership

If membership seems stuck: toggle Join off/on, restart the node, confirm agent card publish, and ask a bonded peer to refresh workers. Check audit for card fetch or index errors.

46. Agent Network Profile

46.1 Owner-attested worker profiles

The profile is an owner-attested description used for soft worker ranking, not a centrally verified benchmark. It may include model freshness, spend posture, context window, strengths, and throughput.

46.2 Model freshness

Model freshness (1–10) is owner-attested signal for how current your models feel. Orchestrators use it as a soft tie-breaker after capability match, not as verified benchmark.

46.3 Spend posture

Spend posture (`subscription`, `metered`, `unknown`) hints whether long jobs may hit provider limits. It influences scoring but does not override mandate cost ceilings.

46.4 Context window

Context window (`128k` – `1M+`) helps orchestrators pick workers for large-document subtasks. Misstating window size may cause assignment mismatch or failed execution—keep it honest.

46.5 Strengths and skill tags

Strengths and skill tags (research, coding, summarization, etc.) improve soft ranking when several workers share the same capability. They do not grant capabilities you did not advertise on the card.

46.6 Throughput information

Throughput information (when provided) helps Assigner estimate parallel capacity. It is informational; stall detection still relies on heartbeats and orchestrator timers.

46.7 How candidate scoring works

Candidate scoring prioritizes capability match, then uses context, freshness, spend posture, strengths, and related signals. These factors guide assignment but do not override bond and mandate policy.

46.8 Profile trust and limitations

Profiles are **self-declared** by owners you already trust via bonds—not third-party ratings. Treat them as hints; verify outcomes through reports, audit, and repeated collaboration.

46.9 Update or remove a profile

Edit profile fields under **Settings** → **Agent Network** anytime. Clearing the profile removes soft-ranking hints but does not disable Join; toggle membership separately to stop recruitment.

47. Agent Identity and Agent Cards

47.1 Why agents have independent identities

Agents have **independent peer IDs** (`envoy_agent_…`) derived from owner + agent keys so peers can verify an agent acts under a specific owner mandate, separate from device keys.

47.2 Owner, device, agent, and peer relationships

Owner identities authorize mandates; **devices** run nodes; **agents** execute tasks; **peer IDs** sign envelopes at runtime. Team jobs always address agent peers for task traffic.

47.3 Owner-authorized agent credentials

Owner-signed **mandates** link an agent public key to an owner DID. Workers should reject chain proposals that lack valid mandate signatures within cost, sensitivity, and expiry bounds.

47.4 Agent public keys

Each agent publishes a **public key** on its card. Recipients verify envelope signatures before accepting `task.chain.*` or `task.result` payloads.

47.5 Agent cards

An Agent Card describes an agent's identity, capabilities, task support, optional worker profile, and endpoints. Native cards move through signed EnvoyMesh flows; an A2A bridge can publish a filtered external representation.

47.6 Capabilities and supported task types

Capabilities are string tags (`doc.translate`, `task.execute`, …) on the card. **Supported task types** describe which wire intents the agent accepts. Subtasks declare required capabilities; mismatches exclude a worker.

47.7 Membership tags

Membership tags include `capability-provider` when Join Agent Network is enabled. Orchestrators filter on this tag before offering subtasks to a bonded contact.

47.8 Fetch and refresh a bonded agent's card

Cards auto-fetch when bonds form (eligible tiers) and cache ~24h. Use **Refresh workers** or bond events to force update before a critical Team job.

47.9 Verify the agent's owner

Verify `ownerId` on the card matches the bonded contact you expect. Mandate signatures must chain to that owner; mismatches are grounds to reject work.

47.10 Revoke an agent

Revoke an agent by owner mandate revocation and removing or rotating its keys on the home node. Peers with stale cards should refresh; blocked trust stops new assignments immediately.

47.11 Multiple agents for one owner

One owner may run **multiple agents** with distinct keys and cards. Each opts in and advertises capabilities independently; orchestrators treat them as separate workers.

48. Bonds and Worker Eligibility

48.1 Bond trust levels

Trust tiers are **blocked**, **public**, **referred**, and **direct**. Team job workers generally require referred or direct trust; public strangers are not recruitable workers.

48.2 Why Team jobs require bonded contacts

Team jobs operate across bonded relationships because workers may receive objectives, data context, and delegated authority. Bond policy prevents unknown public peers from entering this workflow by default.

48.3 Public peers and strangers

Public peers are not auto-fetched as Team job workers. Bond them to referred or direct before expecting collaboration.

48.4 Referred workers

Referred workers may participate under tighter policy. Orchestrator-side chain traffic typically requires referred or higher; confirm bond tier before assigning sensitive subtasks.

48.5 Direct workers

Direct bonds unlock the full worker path: direct assign, bidding, and cross-home handoff subject to mandates. This is the usual friend/fleet configuration.

48.6 Blocked workers

Blocked peers cannot send or receive collaboration intents. Existing assignments should fail closed; cancel active subtasks involving blocked workers.

48.7 Capability requirements

Each subtask names a **required capability**. Workers must advertise an exact or soft-matched tag plus `capability-provider` membership to be eligible.

48.8 Membership and card freshness

Stale cards may hide new capabilities or show revoked agents. Refresh after membership toggles, model changes, or bond updates; orchestrators skip workers beyond freshness thresholds.

48.9 Worker eligibility checklist

Confirm: the owners are bonded; trust is referred or direct as required; Join Agent Network is enabled; the card is fresh; `capability-provider` is present; the requested capability matches; and neither side is blocked.

48.10 Change or revoke trust

Lowering trust or blocking a contact stops new recruitment immediately. Review active Team jobs for in-flight subtasks awarded to that peer and cancel or reassign as needed.

49. Find and Onboard Workers

49.1 Bond an existing contact

Bond an existing contact through chat intro, QR, or invite before recruiting them. Team jobs never substitute anonymous discovery for trust establishment.

49.2 Office-LAN onboarding

Office LAN onboarding combines same-Wi-Fi discovery with a shared token under **Settings → Agent Network**. It accelerates bonding for coworkers on one network.

49.3 LAN auto-bond

LAN auto-bond pairs machines on the same subnet when enabled and tokens match. It creates trust, not membership—each peer must still enable Join Agent Network to become a worker.

49.4 Company invitation links

Company invitation links (`envoy://invite?…`) let teammates join your fleet with scoped trust. Distribute links through your normal secure channels; expired links stop working.

49.5 Pairing kiosk

Pairing kiosk mints one-click invites for events or support desks. Keep kiosk mode off unless you actively supervise pairings—it reduces friction by design.

49.6 Fleet Manifest import

Fleet Manifest import applies a signed roster of peers and trust hints for larger teams. Validate manifest signatures before import; manifests create bonds, not automatic worker opt-in.

49.7 Refresh worker status

Click **Refresh workers** on the Agent Network tab after onboarding changes. The capability index updates from freshly fetched agent cards across bonded contacts.

49.8 Capability-based matching

Assigner matches subtask `requiredCapability` to worker card tags, then applies soft scoring (context, freshness, strengths, same-LAN hints). Missing capability excludes the worker entirely.

49.9 Probe a peer

Probe a peer sends a lightweight reachability check before awarding expensive subtasks. Failed probes remove unreachable workers from the current roster pass.

49.10 Diagnose zero eligible workers

If no worker is eligible, refresh cards, verify bonds, confirm membership, inspect capability tags, and probe reachability. The UI should not start a multi-agent job when only the local node is available.

49.11 Broad anonymous worker discovery — not currently offered

Planned boundary. Current Team jobs recruit bonded, opted-in workers. Network-wide anonymous worker search and public-marketplace behavior are not offered.

50. Team Jobs Fundamentals

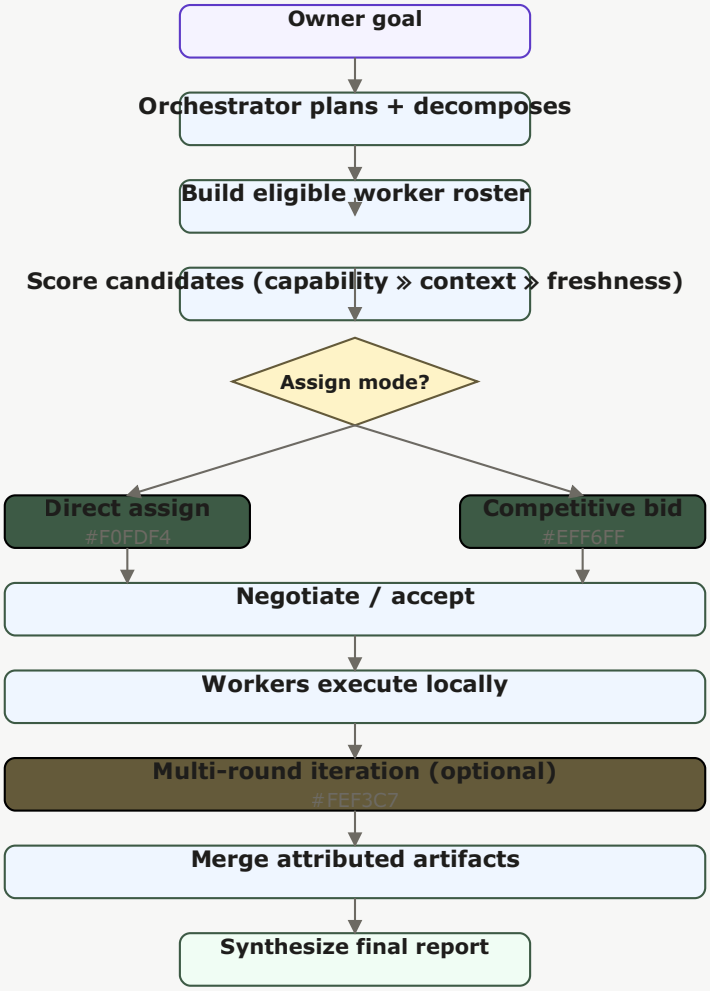

Figure 6 — Team job orchestration: the orchestrator plans, builds a roster, scores candidates, then branches to direct assign or competitive bidding. Workers execute locally; results merge into one attributed report.

50.1 What a Team job is

A Team job turns one owner goal into coordinated subtasks executed by several eligible agents. The initiating home node owns the budget and report and normally acts as orchestrator.

50.2 Team jobs and the older “chains” name

The UI says **Team jobs**. Protocol intents, RPC names, storage, and older documents may use **chain**; treat that as the implementation name for the same product workflow.

50.3 Required workers

At least one eligible remote worker is required for meaningful multi-agent execution. A solo node can use its personal agent, but the Team jobs UI blocks or reports no-workers rather than pretending to distribute work.

50.4 State a goal

Enter a clear, bounded goal in **Team jobs** → **New team job** or promote from chat. Good goals state deliverable, constraints, and sensitivity so the planner can decompose realistically.

50.5 Preview the plan

Preview the plan shows proposed subtasks, capabilities, and worker slots before spend starts. Edit or cancel here if the decomposition looks wrong.

50.6 Start from chat

From chat, escalate a conversation turn into a Team job when the goal needs multiple agents. The orchestrator inherits context subject to mandate sensitivity limits.

50.7 Start from the Team jobs view

The **Team jobs** view is the primary control surface on desktop Social: start, monitor, approve, rebalance, and open reports. EnvoyGo mirrors status read-only.

50.8 Follow progress

Watch lifecycle states (`discovering`, `running`, `partial`, `synthesizing`, …) and per-subtask rows. WebSocket `chain:iteration` events update iteration progress during Phase 47 multi-round jobs.

50.9 Review a completed report

Open the published **chain report** for attributed sections, worker provenance, cost summary, and pinned artifacts. Draft rounds (Phase 47) appear in accordion before final publish.

50.10 Cancel or retry a Team job

Cancel from Team jobs UI sends `task.chain.cancel` downstream. Retry may require a new job or orchestrator re-plan depending on failure mode; check audit for terminal reason.

51. Plan and Decompose Work

51.1 The orchestrator’s role

The orchestrator turns the owner’s goal into a plan, finds workers, awards subtasks, tracks execution, enforces budget and policy, and merges results. Relays only carry connectivity and never assume this role.

51.2 Convert a goal into subtasks

The orchestrator decomposes the goal into subtasks with objectives, required capabilities, inputs, deadlines, and cost ceilings. LLM-assisted planning may propose steps; owner preview approves before dispatch.

51.3 Required capabilities

Each subtask declares a **required capability** tag matching agent card entries. Planning fails early if no bonded, opted-in worker advertises that capability.

51.4 Dependencies and ordering

Dependencies order subtasks (for example research before synthesis). The orchestrator respects DAG edges and will not award dependent work until prerequisites complete or partial results arrive.

51.5 Worker-count limits

maxWorkers caps concurrent active worker sessions on a chain mandate. Finished or cancelled subtasks free slots for reassignment.

51.6 Depth limits

Default chain **depth is 2** (orchestrator → workers). Depth 3 requires `allowDepth3` on the owner-signed chain mandate; depth beyond 3 is rejected.

51.7 Deadlines and sensitivity

Set per-subtask deadlines and sensitivity ceilings in the plan preview. Workers enforce local bond and vault policy even when orchestrator requests higher sensitivity.

51.8 Preview and edit the plan

Edit subtask objectives, costs, or capability tags in preview when manual mode is available. Automatic LLM plans should be reviewed before start on high-stakes goals.

51.9 LLM-assisted planning

LLM-assisted planning uses the home model to propose decomposition when enabled. Failures fall back to keyword/heuristic templates or block start with a clear planning error.

51.10 Planning failures and fallback behavior

When planning fails, check for zero eligible workers, unsupported capabilities, depth violations, or model errors. Reduce goal scope or add workers before retrying.

52. Find and Assign Agents

52.1 Build the eligible worker roster

The roster lists bonded contacts who joined Agent Network and pass capability filters for the current plan. Same-LAN peers may rank higher when dial hints show direct paths.

52.2 Capability matching

Capability matching is hard filter first: no tag, no assignment. Soft scoring breaks ties among remaining eligible workers.

52.3 Context, freshness, spend, and strength scoring

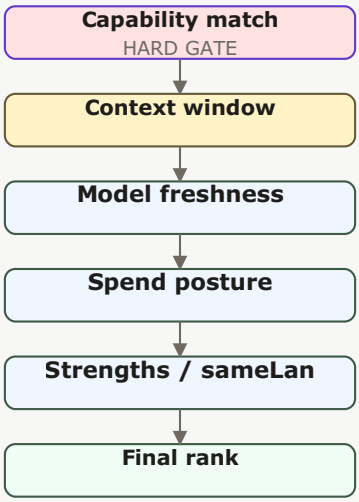

Figure 14 — Candidate scoring funnel: capability match is a hard gate; below it, soft factors (context, freshness, spend, strengths) contribute to the final rank.

Scoring weighs **context window**, **freshness**, **spend posture**, and **strengths** after capability fit. Direct assign picks the top scored worker; bidding still uses score as signal.

52.4 Same-network considerations

Workers on the same LAN may respond faster and rank higher (`sameLan` soft score). WAN paths use relays for connectivity without changing trust requirements.

52.5 Direct assign mode

Direct assign is the default for personal and small-team use. It selects an eligible scored worker and awards work without exposing a bidding flow.

52.6 Competitive bidding mode

Competitive bidding collects offers when cost, timing, or choice matters. It adds negotiation and owner decisions, so enable it only when those controls justify the extra delay.

52.7 Assigner selection

Assigner selection chooses which home node plans and awards subtasks—usually yours. Remote assigner handoff delegates that role to another bonded orchestrator with better worker visibility.

52.8 Remote assigner handoff

Remote assigner handoff transfers assignment authority via `task.chain.handoff` while preserving mandate bounds and Phase 47 iteration knobs when configured.

52.9 No-worker behavior

With **no eligible workers**, start is blocked (`no_workers`). Enable Join on peers, refresh cards, or bond additional contacts—solo nodes cannot fake multi-agent execution.

52.10 Refresh and re-evaluate workers

Re-run roster build after refresh, bond changes, or mid-job stalls. Orchestrator may swap workers when probes fail or bids expire.

53. Bids and Negotiation

53.1 When bidding is used

Bidding is used only in competitive mode or a workflow that explicitly requests offers. Direct assign avoids this exchange.

53.2 Request bids

In **competitive bidding** mode, the orchestrator broadcasts `task.chain.propose` and collects `task.chain.bid` responses before accept. Direct assign skips this step.

53.3 Review proposed cost and timing

Compare each bid's proposed **cost** and **ETA** against subtask ceilings and chain budget. Reject bids that exceed mandate limits without owner approval.

53.4 Review confidence and justification

Review bid **confidence** and textual justification when shown. Low-confidence bids may warrant counter-proposal or a different worker.

53.5 Compare candidates

Side-by-side candidate comparison highlights cost, score, and capability fit. Owner picks accept when manual award mode is enabled.

53.6 Counter-bids

Counter-bids adjust cost, deadline, or scope via `task.chain` negotiation envelopes. Workers may accept revised terms or withdraw.

53.7 Accept or reject work

Accepting emits `task.chain.accept`; rejecting leaves the subtask open for other bidders or reassignment. Document decisions in audit for later cost disputes.

53.8 Negotiation timeouts

Negotiation timers prevent indefinite stalls. Expired bids free the subtask for re-offer or fallback direct assign per Team job defaults.

53.9 Human approval during negotiation

Sensitive or high-cost accepts may enter **owner approval** queues. Resolve approvals in Social before workers start execution.

53.10 Audit negotiation decisions

Audit records capture bid amounts, accepted peer, counter-proposal history, and approval outcomes. Export or filter by `chainId` for retrospective review.

54. Budgets, Cost, and Rebalancing

54.1 Set a Team job budget

Set **maxChainCostUsd** and related limits when starting a job or in saved recipes. The orchestrator tracks spend against the chain budget ledger throughout execution.

54.2 Cost ceilings

Each subtask carries a **cost ceiling**; workers cannot bid or charge above it without rebalance or owner approval.

54.3 Worker cost allocation

Initial allocation splits chain budget across subtasks in the plan. Manual rebalance moves funds; automatic rebalance adjusts within configured increments.

54.4 Manual rebalance

Manual rebalance pauses for owner review when allocation or worker conditions change. It maximizes control at the cost of requiring timely attention.

54.5 Automatic rebalance

Automatic rebalance lets the orchestrator adjust within configured increments, ceiling, and retry limits. Use conservative limits and require approval for material cost increases.

54.6 Never-rebalance policy

Never-rebalance preserves the original budget allocation. A stalled or underfunded subtask may then fail rather than consume additional funds.

54.7 Rebalance increments and limits

Configure **rebalance increment** size and maximum automatic retries in Team job defaults. Conservative increments reduce surprise spend.

54.8 High-cost approvals

Crossing high-cost thresholds triggers **approval requirements** on the mandate. Watch for waiting-for-owner states when spend spikes.

54.9 Export cost data

Export cost breakdowns from completed reports or audit CSV when enabled. Includes per-worker attribution and rebalance events.

54.10 Interpret final cost reports

Final cost reports show estimated versus actual spend, rebalance history, and unspent budget. Compare to mandate `maxChainCostUsd` before starting similar jobs.

55. Run and Monitor Work

55.1 Worker acceptance

After accept, workers acknowledge and transition subtasks to **running**. Reject or timeout returns the subtask to negotiating or failed states.

55.2 Running states

Chain lifecycle moves through `discovering → negotiating → running → partial → synthesizing → completed|failed`. UI maps these to human-readable Team job status.

55.3 Heartbeats

Workers send heartbeats so the orchestrator can distinguish progress from disconnection. Missing heartbeats feed stall detection but should not be treated as proof of malicious behavior.

55.4 Partial results

Workers emit `task.chain.partial` with intermediate artifacts when more output is coming. Orchestrator waits or merges partials per termination policy.

55.5 Stall detection

Stall detection uses missed heartbeats and configured timeouts. Trigger retry, reassignment, or owner prompt per stall policy.

55.6 Retry and reassignment

Retry re-offers a subtask; **reassignment** cancels the stalled worker slot and awards a backup. Release worker capacity before assigning replacements.

55.7 Waiting for owner input

Jobs pause in **waiting_for_owner** when approvals, iteration continue/stop, or rebalance decisions are needed. Resolve in desktop Social—EnvoyGo shows the state but cannot act.

55.8 Worker failure

Worker **failure** marks subtasks failed with reason codes. Orchestrator may synthesize partial reports or fail the chain per termination policy.

55.9 Cancel a subtask or whole job

Cancel a single subtask or entire chain from Team jobs. Downstream workers receive cancel intents; audit records who initiated termination.

55.10 Audit progress

Filter audit by `chainId` and correlation IDs to reconstruct timeline: plan, bids, accepts, partials, merge, iteration rounds, publish.

56. Multi-Round Iteration

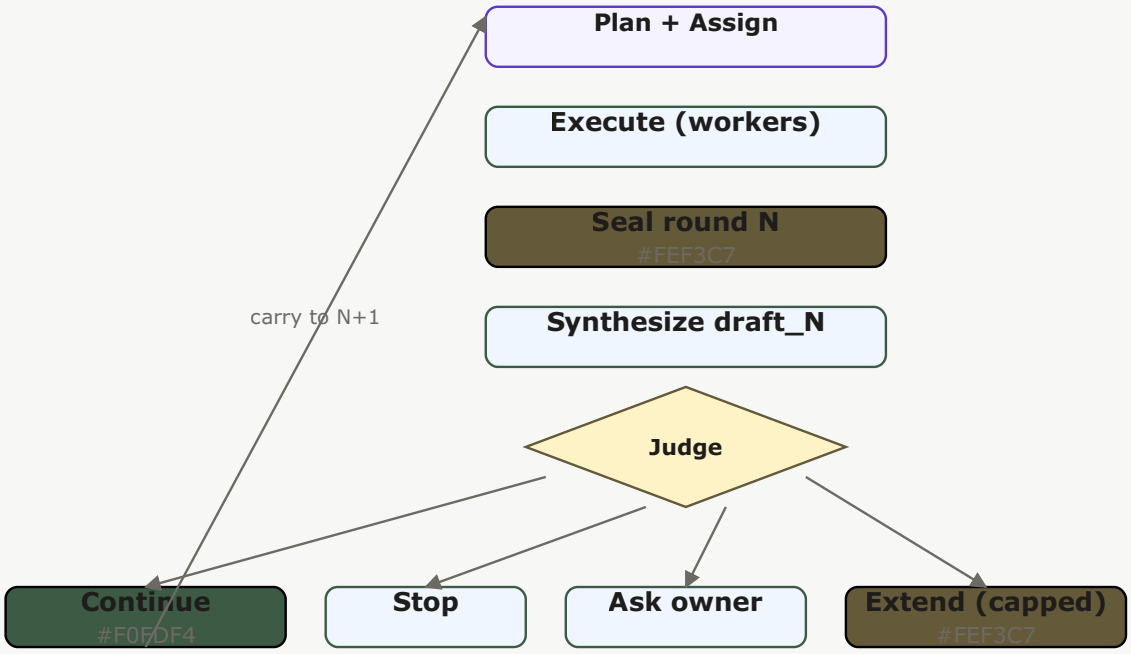

Figure 10 — Multi-round iteration: within a round, plan → execute → seal → synthesize draft. The judge then continues, stops, asks the owner, or extends (capped). Summaries carry into the next round.

56.1 Why a Team job may need another round

Some goals benefit from reviewing a first draft and requesting targeted follow-up. Multi-round iteration adds that loop without allowing unbounded autonomous work.

56.2 Draft, judge, and replan

Phase 47 **seal** → **draft** → **judge** → **replan** closes a round, synthesizes a draft report, decides whether to continue, and optionally launches another planning pass with carried summaries.

56.3 Extend work within a round

Extend within a round (Phase 47B) appends capped extra steps before seal when local heuristics say more work in the same round will help.

56.4 Maximum rounds and extension limits

Defaults preserve single-round behavior unless the owner or template opts in. Maximum rounds and per-round extensions are hard caps that limit cost and duration.

56.5 LLM judge mode

LLM judge mode asks the configured model whether another round would improve the result. The decision remains bounded by maximum rounds, budget, deadline, and policy.

56.6 Always-stop mode

Always-stop ends after the current sealed round, giving predictable cost and latency.

56.7 Owner-decision mode

Owner-decision mode pauses after a draft and asks the owner whether to stop, continue, or extend specific work.

56.8 Carry summaries into the next round

Summaries and `iterationState` blobs carry forward so the next round does not repeat completed subtasks blindly. Remote assigner handoff preserves this state when configured.

56.9 Stop reasons

Stop reasons include max rounds reached, owner stop, budget exhaustion, judge always-stop, or failed seal. They appear in report metadata and audit.

56.10 Review iteration history

Review iteration history in Team jobs accordion: each draft round, owner Continue/Accept decisions, and final publish. EnvoyGo mirrors read-only.

57. Cross-Home and Cross-Orchestrator Handoff

57.1 When to hand off orchestration

Handoff is useful when another bonded home has better worker visibility or should own a delegated sub-chain. It does not transfer the original owner's unlimited authority.

57.2 Choose a remote assigner

Pick a **remote assigner** bonded peer with `chain.orchestrate` or better worker roster for delegated assignment. Trust must be direct or policy-allowed for handoff.

57.3 Delegate a sub-chain

Delegate a sub-chain via `task.chain.delegate` so another orchestrator runs a subtree under your mandate limits, not unlimited owner authority transfer.

57.4 Parent and child responsibilities

The **parent** orchestrator retains chain ownership and budget; the **child** assigner executes delegated subtasks and returns results upstream.

57.5 Relay chain traffic

Relay chain traffic uses circuit paths for WAN peers. Relays forward envelopes without interpreting payloads or running models.

57.6 Preserve iteration state

Handoff payloads include Phase 47 **iteration knobs** and optional `iterationState` so the remote assigner continues multi-round jobs seamlessly.

57.7 Arbitration records

Arbitration records resolve ordering disputes between orchestrators using seq and timestamp rules when cross-home coordination conflicts arise.

57.8 Failure and recovery

On handoff failure, parent orchestrator should reclaim assignment, fail the subtree, or cancel per policy. Check audit for `handoff` reject reasons.

57.9 Trust requirements

Handoff requires compatible trust tiers, valid mandates, and mutual reachability. Blocked or public peers cannot become remote assigners.

57.10 Audit a handoff

Audit handoff events with sender/receiver orchestrator peer IDs, delegated chain IDs, and iteration state checksums for compliance review.

58. Merge Results and Create Reports

58.1 Collect worker results

The orchestrator collects `task.result` and `task.chain.partial` payloads from each awarded worker before merge or synthesis.

58.2 Text artifacts

Text artifacts store narrative worker output with attribution metadata. Suitable for summaries and research sections.

58.3 Structured artifacts

Structured artifacts hold JSON or typed records (tables, extracted fields). Validators check shape on receipt.

58.4 File artifacts

File artifacts reference vault items or chunked content by ID. Workers do not push raw filesystem paths across the mesh.

58.5 Composite artifacts

A composite artifact bundles attributed worker contributions and an aggregation method. It preserves provenance that would be lost if all text were flattened into one anonymous answer.

58.6 Weighted contributions

Weighted contributions let synthesis emphasize higher-confidence or owner-prioritized worker sections in the composite report.

58.7 Merge strategies

Merge strategies (concatenate, summarize, vote, template-driven) are chosen per report type. Phase 47 draft rounds may use lighter merge before final publish.

58.8 Worker attribution and provenance

Reports preserve **worker attribution** and provenance so readers know which peer produced each section—critical for accountability.

58.9 Synthesize the final report

Synthesize the final report after all required subtasks complete or partial policy allows best-effort merge. Only one terminal publish per iteration round.

58.10 Pin and export a report

Pin important reports in Team jobs for quick access; **export** when CSV or file export is enabled in your build.

58.11 Owner review

Owner review accepts draft iterations (Phase 47), rejects unsafe content, or requests another round before final publish.

59. Team Job Recipes and Defaults

59.1 Save reusable job templates

Save **job templates** with default budgets, award mode, stall/rebalance/iteration policies, and sensitivity. Templates are local to your node—not a marketplace.

59.2 Choose award defaults

Choose **direct assign vs competitive bidding** default in **Settings → AI → Team job defaults**. Most personal teams should stay on direct assign.

59.3 Configure stall policy

Configure **stall policy** (timeouts, auto-rebid, notify owner) per template or globally. Aggressive timeouts reduce cost but increase reassignment churn.

59.4 Configure rebalance policy

Set **rebalance policy** to manual, automatic, or never. Match your appetite for autonomous budget shifts mid-job.

59.5 Configure iteration defaults

Phase 47 **iteration defaults** (`iterationMaxRounds` , judge mode, extend caps) live in defaults or templates. Default `iterationMaxRounds=1` preserves single-round behavior.

59.6 Configure cost visibility

Cost visibility toggles whether workers and owners see bid amounts in UI during competitive mode. Hidden cost UI does not remove ledger tracking.

59.7 Use a saved recipe

Start from a **saved recipe** to pre-fill policies and award mode. Edit goal and preview before committing spend.

59.8 Update or remove a recipe

Update recipes when your team workflow changes; delete obsolete templates to avoid accidental use of stale policies.

59.9 Template marketplace — parked

Parked. Saved recipes are local product features; a mesh-wide marketplace for exchanging templates has no committed release.

60. Team Jobs on EnvoyGo

60.1 View active Team jobs

EnvoyGo **Team jobs** tab lists active jobs mirrored from the home node over JSON-RPC. Status updates when the phone is connected; offline viewing may lag.

60.2 View recent jobs

Recent jobs shows completed or failed chains with timestamps. Use it to reopen reports on mobile without starting new work.

60.3 Open a job detail

Tap a job for detail: lifecycle state, subtask summary, iteration progress line, and links to published reports. Controls that change orchestration are hidden.

60.4 Read reports and artifacts

Read **reports and artifacts** inline when synced. Large file artifacts may require opening on desktop if not cached on the phone.

60.5 Understand read-only mobile behavior

EnvoyGo presents a read-only mirror of active and recent Team jobs. Start, award, rebalance, and orchestration controls remain on the home/desktop experience.

60.6 Return to desktop for orchestration controls

For start, cancel, rebalance, bid accept, or iteration Continue/Accept, switch to **desktop Social** on the home node. EnvoyGo intentionally omits these mutating RPCs.

60.7 Mobile notifications

Mobile notifications (when enabled) alert for job completion or owner-approval waits. Tapping opens read-only detail; act on approvals from desktop.

60.8 Troubleshoot the EnvoyGo mobile mirror

If the mirror is empty: confirm EnvoyGo pairing, home node reachability, and that jobs were started on desktop. Reload after WebSocket reconnect.

61. Agent Network Trust and Safety

61.1 Verify worker identity

Verify worker **peer ID** and card signatures match envelopes on every `task.chain.*` message. Reject mismatched keys or expired mandates.

61.2 Verify owner authorization

Confirm the worker's **owner mandate** authorizes the chain capability and sensitivity requested. Owner DID on card must match bonded contact.

61.3 Bond policy and capability gates

Bond policy and capability gates run before orchestrator logic. Blocked intents never reach worker execution even if UI allowed planning.

61.4 Mandate limits

Every worker request is bounded by a signed mandate that specifies objective, actions, peer scope, sensitivity, cost, expiry, and approval requirements. A worker should reject work outside those bounds.

61.5 Data-sensitivity boundaries

Subtasks declare sensitivity; workers downgrade or reject over-limit data per vault policy. Do not exfiltrate friends-tier content to public-tier peers.

61.6 Cost and deadline limits

Mandate **cost and deadline** limits apply on both orchestrator and worker nodes. Task runtime guards cancel expired or over-budget work.

61.7 Approval requirements

Actions listed in `requiresApprovalFor` pause until owner allow. External agents observing chains cannot bypass approval queues.

61.8 Runtime task guards

Task runtime guards enforce cancellation, collect-N termination, and mandate expiry on workers mid-flight.

61.9 Vault and model isolation

Workers run models and vault access locally under Diplomat → Bond → Brain → Vault isolation. Remote orchestrators never receive raw vault filesystem paths.

61.10 Block and revoke a worker

Block trust to stop all collaboration with a peer. **Revoke** agent mandates on your node if your worker should reject new inbound chain proposals.

61.11 Respond to malicious or misconfigured agents

For misconfigured agents, disable Join, rotate keys, block peer, and cancel active chains. Collect audit evidence before re-enabling.

61.12 Review end-to-end audit trails

Stitch **audit JSONL** by `chainId` and `correlationId` across orchestrator and worker nodes (each side logs its view). No central server holds the trail.

62. Agent Network Connectivity

62.1 Local-network discovery

mDNS / LAN discovery helps find peers on the same network for bonding and lower-latency paths. It does not replace bond establishment for Team jobs.

62.2 Direct peer connections

Direct TCP/QUIC connections are preferred when dial hints show reachable private addresses. Same-LAN workers score higher in Assigner.

62.3 Relay-assisted connections

Relay-assisted circuit paths connect WAN peers when NAT blocks direct dial. Relays do not terminate TLS for chain payloads beyond transport relay.

62.4 Agent card synchronization

Agent cards sync over bond-triggered fetch and capability index updates. Stale sync manifests as missing workers until refresh.

62.5 Capability discovery

Capability discovery queries the index built from bonded peers' cards. Only opted-in workers with matching tags appear.

62.6 Offline workers

Offline workers fail probes and heartbeats; orchestrator marks subtasks stalled and may reassign per policy.

62.7 Reconnection and retry

libp2p reconnects automatically when peers return. Retry discovery after network changes; use relay bootstrap if direct paths fail.

62.8 Multi-relay coordination

Multi-relay setups use community or private bootstrap peers for DHT and circuit relay. Override `TEST_RELAY_ADDR` only in tests—operators configure bootstrap in node settings.

62.9 NAT and firewall considerations

Open firewall ports for outbound mesh traffic; inbound direct dial may require port mapping or relay fallback. Team job reliability improves with working circuit reservations.

62.10 Diagnose worker reachability

Run peer **probe** from Agent Network settings, inspect dial hints, and verify relay reservation logs. Compare LAN vs WAN paths when workers are reachable but slow.

63. Agent Network Troubleshooting

63.1 Join toggle does not take effect

If Join toggle does not stick, restart node, check `capabilityProviderEnabled` in config, and confirm no fleet script overwrote settings. Re-enable and refresh workers on a peer.

63.2 Worker is not visible

Invisible workers: verify bond tier, remote Join enabled, capability tag present, and click **Refresh workers**. Public-tier bonds do not auto-fetch cards.

63.3 Agent card is stale or missing

Force card refresh by re-bonding or manual fetch; cards older than cache TTL may hide new capabilities. Check audit for `agent.card.response` errors.

63.4 No eligible workers

Fix **no eligible workers** by bonding contacts, having them Join, aligning capability tags with plan, and refreshing. UI should block start rather than run solo multi-agent fiction.

63.5 Plan cannot be created

Plan cannot be created when LLM planner fails, capabilities mismatch, or depth/budget constraints violate mandate. Simplify goal or add workers.

63.6 Bid or negotiation does not complete

Stuck **bids**: check competitive mode timeouts, worker Join status, and bond policy. Counter-bid loops exhaust when max negotiation rounds reached.

63.7 Job is stalled

Stalled jobs: inspect heartbeats, stall policy, worker offline state, and owner-approval waits. Manual rebalance or cancel may unblock.

63.8 Worker returns no result

Empty **results** often mean worker rejected mandate, hit sensitivity wall, or crashed locally. Worker-side audit shows deny vs fail reason.

63.9 Artifact cannot be opened

Artifact open failures: verify vault path on orchestrator node, sensitivity approval, and that file artifact IDs still exist. Re-sync library if chunked content missing.

63.10 Budget or approval blocks the job

Budget or **approval blocks** show as `waiting_for_owner`. Resolve approvals or raise mandate limits on desktop, then resume.

63.11 Handoff fails

Handoff fails when remote assigner unreachable, trust insufficient, or iteration state rejected. Parent should fail over or cancel subtree; check `task.chain.handoff` audit.

63.12 Report is partial

Partial reports may publish under best-effort termination policy when some subtasks fail. Review attribution to see which sections are missing.

63.13 Collect diagnostics

Collect **diagnostics**: chain ID, audit excerpts, worker roster snapshot, bond tiers, card timestamps, and network probe results from both orchestrator and worker nodes.

Part VIII — Tasks, Mandates, and Artifacts

64. Task Fundamentals

64.1 What an EnvoyMesh task is

An EnvoyMesh task is a signed, policy-checked request between agents with an objective, requested result, constraints, lifecycle, and attributable artifacts. It is narrower than a Team job, which coordinates several subtasks.

64.2 Task objectives and requested results

State a clear **objective** and **requestedResult** so workers can judge fit before accepting. Vague goals cause unnecessary `task.negotiate` rounds or early `task.reject`; include sensitivity hints when vault content is involved.

64.3 Create a task

On mesh, agents open a task with `task.mandate` then `task.propose`. Over A2A, `message/send` triggers the production executor: Bonds gate → home-owner-signed mandate → `task.propose` → `handleDaemonTaskInbound` (runtime guard + journal).

64.4 Proposal and negotiation

`task.propose` offers concrete work under an accepted mandate; `task.negotiate` adjusts terms. Both are signed agent envelopes—the daemon inbound handler validates mandate bounds before advancing lifecycle state.

64.5 Accept or reject work

Workers reply with `task.accept` or `task.reject`. Acceptance requires bond tier and mandate ceilings still satisfied; rejection should carry a reason auditors can correlate via `correlationId`.

64.6 Follow task state

Track progress in Social, audit JSONL, or A2A `tasks/get` when bridged. Map internal twelve-state lifecycle to nine A2A states when presenting status to external clients.

64.7 Heartbeats and partial results

Emit `task.heartbeat` during long runs so orchestrators do not stall waiting. Partial `task.result` payloads record interim artifacts while mandate `collectCompletedResults` and expiry rules remain enforced.

64.8 Completed and failed tasks

Terminal success requires a signed `task.result` in `completed` state; failures land in `failed` with an auditable reason. A2A pollers see mapped `completed` / `failed` after `tasks/get`.

64.9 Cancel a task

Send native `task.cancel` on mesh or `tasks/cancel` over A2A JSON-RPC. Tokens are owner-scoped—a bearer mapped to one owner cannot cancel another owner's tracked tasks.

64.10 Task feedback

Attach post-completion feedback to the task record for operator review. Feedback does not widen mandate authority or alter artifact content hashes already published.

65. Task Lifecycle

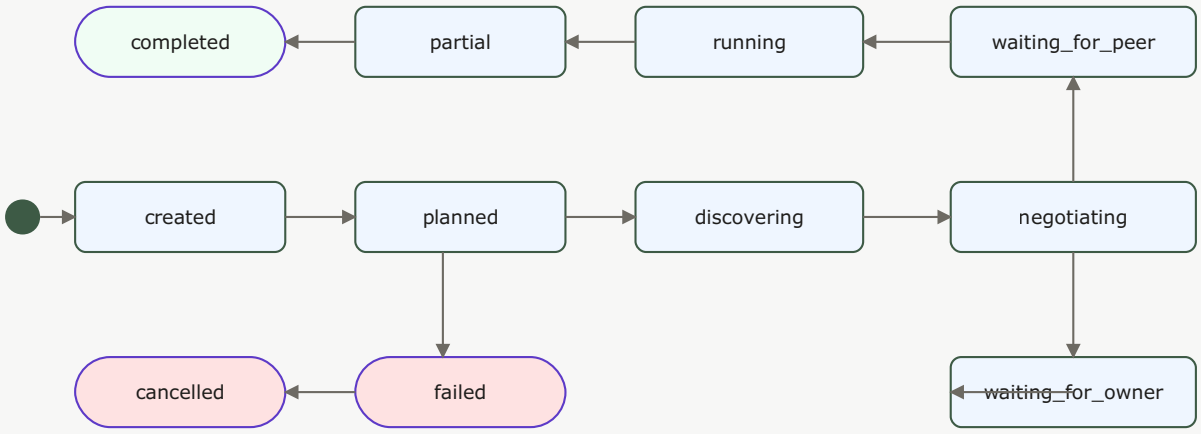

Figure 4 — Task lifecycle: 12 states with three terminal states (completed green, failed/cancelled red). Arrows show legal transitions; negotiation can branch to waiting states; partial results may continue to completion.

65.1 Created

Created means the task record exists but planning or peer interaction has not begun. It is a lifecycle state, not a statement that the product feature is merely planned.

65.2 Task planned

Task planned means the node has derived an execution approach and can proceed to discovery or proposal. The source schema names this state `planned`.

65.3 Discovering

The orchestrator scans bonded peers and capability-index entries within mandate contact scope. Discovery stalls when no worker meets bond tier (`direct` / `referred`) or sensitivity requirements—refresh agent cards before blaming mesh outage.

65.4 Negotiating

Active `task.negotiate` exchanges adjust deliverables, cost, or sensitivity. Either party rejects when a counter-offer exceeds mandate `maxCost`, `maxSensitivity`, or disallowed actions.

65.5 Waiting for a peer

State `waiting_for_peer` means no remote agent has accepted yet. Verify remote Join toggles, bond tier, and dial hints; time out and reassign per orchestrator policy if heartbeats stop.

65.6 Waiting for the owner

`waiting_for_owner` follows `requiresApprovalFor` hits or bond policy that demands owner consent. Clear the Social approval queue on the home node—A2A clients may see `input-required` until then.

65.7 Running

Models, vault reads, and tools execute under Brain/Vault isolation on the worker. The A2A bridge defaults to leaving tasks `running` until a real mesh `task.result` arrives (unless `autoCompleteLocal` is enabled for smoke).

65.8 Partial

Partial state records one or more artifacts while work continues. Mandate `closeOnFirstCompletedResult` may terminate the task as soon as the first acceptable artifact lands.

65.9 Synthesizing

Team and multi-worker flows merge child artifacts into a composite result. Weighted child references preserve worker lineage through the synthesis step.

65.10 Completed

Completed is terminal: the requested work ended successfully and the available result and artifacts were recorded.

65.11 Failed

Failed is terminal: execution ended without a successful result. Preserve the reason and audit trail before retrying.

65.12 Cancelled

Cancelled is terminal after an owner, device, peer, or policy path stops the task. A2A spells the mapped external state `canceled` with one “l”.

66. Mandates and Delegated Authority

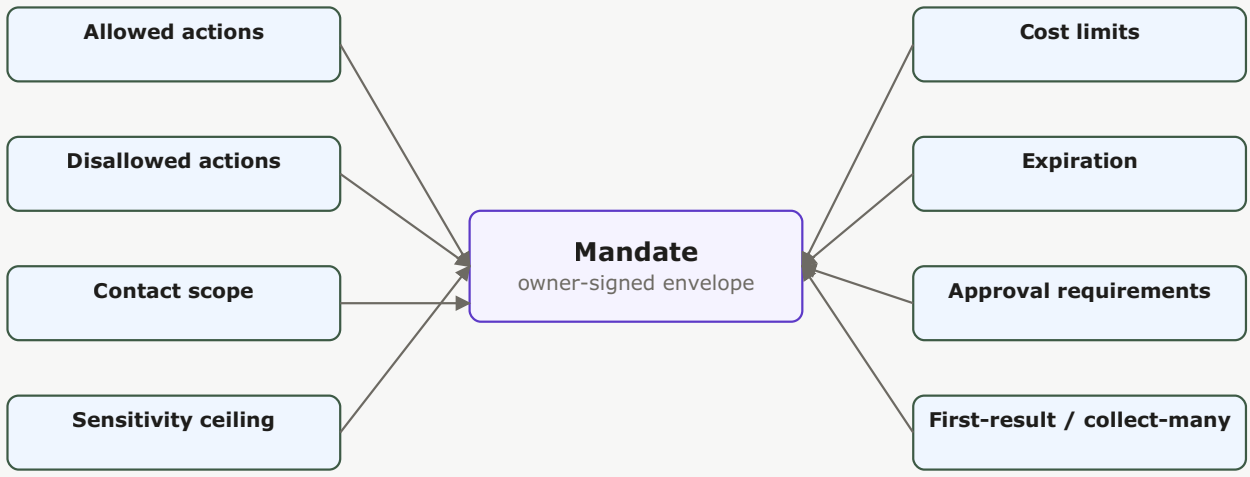

Figure 15 — Mandate anatomy: eight orthogonal dimensions bound what an agent may do. The owner signs the envelope; every dimension is independently enforceable.

66.1 Why agents need mandates

A mandate prevents an agent from interpreting a broad goal as unlimited authority. It defines a verifiable envelope within which planning, tool use, peer contact, and spending are allowed.

66.2 Who issues a mandate

Mandates are always home-owner signed. External A2A bearer tokens identify the owner context but do not sign mandates—the production executor uses the owner's key. Agents act under owner-delegated credentials verified via mandate signature.

66.3 Allowed and disallowed actions

`allowedActions` and `disallowedActions` constrain which intents and tools may run. The task runtime guard denies transitions that would invoke a disallowed action even mid-execution.

66.4 Contact and peer scope

Mandates may restrict participating peer IDs or contact lists. Bonds tiers `self`, `direct`, and `referred` further limit who may receive `task.propose`—strangers cannot accept delegated work.

66.5 Data-sensitivity ceiling

`maxSensitivity` caps vault and knowledge exposure for the whole task. Workers must not return artifacts above the mandate ceiling even when local bond policy would normally allow higher sensitivity.

66.6 Cost limits

`maxCost` bounds authorized spend. Exceeding it stops execution unless the owner issues a new mandate or approves an extension through the approval queue.

66.7 Expiration

`expiresAt` is enforced by the task runtime guard on every inbound intent. Post-expiry proposals, heartbeats, and results are rejected before model or vault access.

66.8 First-result and collect-many policies

Set `closeOnFirstCompletedResult` to stop after the first successful worker; use `collectCompletedResults` when fan-out jobs need N completions before synthesis.

66.9 Approval requirements

List sensitive actions in `requiresApprovalFor` to pause until owner allow in Social. Bridged A2A callers stall in `waiting_for_owner` or see `input-required` until approval clears.

66.10 Agent-specific authorization

Bind mandates to `envoy:agent:<hash>` via owner-signed agent credentials. Remote peers verify the agent is authorized by the stated owner before accepting proposals.

66.11 Proof of intent

Optional signed proof-of-intent documents why the agent initiated work. Use for audit trails—it does not bypass Bonds checks or replace mandate signatures.

66.12 Revoke or cancel authority

Owners revoke mandates or send `task.cancel` to halt further work. Revocation prevents new proposals under the same mandate ID; completed artifacts and audit history remain intact.

67. Artifacts and Results

67.1 Text artifacts

A text artifact contains human-readable output and may include a media type. Use it for summaries, explanations, and reports that do not require a structured schema.

67.2 File artifacts

A file artifact refers to a Vault path and content hash, with optional name, media type, and size. Recipients should verify the hash before trusting downloaded bytes.

67.3 Structured artifacts

A structured artifact carries a schema reference and object data. It is suitable for machine-readable results, tables, records, and interoperability payloads.

67.4 Composite artifacts

A composite artifact contains weighted, attributed child artifacts and an aggregation strategy. Team jobs use it to retain worker lineage through merging.

67.5 Content hashes

File and structured artifacts carry sha256 hashes. Verify hash before trusting downloaded bytes—especially when fetching via authenticated `GET /vault/<path>?hash=…` on the home bridge or relay proxy.

67.6 Display names and media types

Set `displayName` and `mediaType` for UI rendering and A2A Part translation. These labels aid presentation; they never substitute for hash verification on file content.

67.7 Worker provenance

Artifacts record producing agent peer IDs. Composite merges retain weighted child references so Team job attribution survives synthesis.

67.8 Store results in the Vault

File artifacts reference vault paths on the executing node. Path-safety and sensitivity checks run before write; bridged File Parts expose gateway URIs instead of raw filesystem paths.

67.9 Share results

Publish artifact IDs inside signed `task.result` envelopes or share within bonded `knowledge.query` bounds. Do not hand cross-tier peers direct vault paths outside mandate sensitivity.

67.10 Verify a result

Check mandate ID, artifact hashes, result-envelope signature, and bond tier at delivery time. Re-fetch file bytes through vault HTTP with matching `?hash=` before acting on content.

67.11 MCP content mapping

Phase 48 maps MCP TextContent, ImageContent, AudioContent, resource_link, and structuredContent into EnvoyMesh artifacts via `mesh.mcp.call_tool`. The MCP server adapter reverses the mapping when external clients call `mesh.*` tools.

67.12 A2A Part mapping

Text, Data, and File Parts translate through `a2a-artifact-map.ts` into native artifact kinds. File Parts advertise `<gateway>/vault/<encodedPath>?hash=…` URLs served from the home bridge (relay forwards via home-tunnel).

Part IX — MCP and A2A Interoperability

68. Interoperability Overview

68.1 Native EnvoyMesh communication

Native EnvoyMesh communication uses signed envelopes, owner and agent identities, bond policy, and typed intents. It remains the preferred path between EnvoyMesh nodes.

68.2 Why bridges are needed

Claude Desktop, Cursor, and A2A SDKs speak MCP or JSON-RPC—not libp2p envelopes. Opt-in bridge endpoints translate external calls into signed mandates and tool-registry invocations without handing clients a raw mesh socket.

68.3 MCP for tools

MCP target support focuses on the 2025-06-18 tool interfaces: stdio or Streamable HTTP with `tools/list` and `tools/call`. Resources, prompts, and OAuth are future scope.

68.4 A2A for agent discovery and tasks

A2A target support follows v1.0.0 concepts for Agent Cards, unified Parts, task methods, polling, and streaming. EnvoyMesh maps these external calls into its signed task system.

68.5 Trust boundaries

Bridges sit above Diplomat: authenticate callers, enforce size limits, then delegate to Bonds and mandates. Bridge tokens must not exceed the mapped owner identity's intended authority.

68.6 Authentication

MCP server adapter: `ENVOYMESH_BRIDGE_SECRET` or `--bridge-token` matching `bridge.secret` on the node. A2A JSON-RPC: `Authorization: Bearer` from `a2aBridge.bearerTokens[]` (relay: `ENVOYMESH_A2A_BEARER_TOKENS` as `token:envoy:owner:…`). Missing auth fails closed.

68.7 Auditing

Bridge invocations emit audit events (`auditTag: "mcp-server"`, A2A method names). Correlate external request IDs with internal task IDs in JSONL when debugging cross-boundary flows.

68.8 Current compatibility scope

Phase 48 shipped: MCP consumer (`mesh.mcp.*` + `mcpConsumers` config), MCP server (`npx envoymesh mcp-server`), Agent Card at relay `/.well-known/agent-card.json`, JSON-RPC `message/send|stream`, `tasks/get|cancel`, and vault FileArtifact `GET /vault`. OAuth, MCP resources/prompts, and anonymous A2A remain future scope.

69. Use External MCP Servers

69.1 What MCP consumer mode does

Lets the home agent call external MCP servers through `mesh.mcp.list_tools` and `mesh.mcp.call_tool`, backed by `@modelcontextprotocol/sdk` and entries in `node-config.json` → `mcpConsumers`.

69.2 Add an MCP server

Add to `mcpConsumers: [{ name, transport, command?, url?, bearerToken?, allowRemoteHttp?, env? }]`, reload config, then run `mesh.mcp.list_tools` with the consumer `name` to confirm the session starts.

69.3 Stdio transport

Stdio launches a configured local process and exchanges MCP messages over standard input and output. Treat the command as executable code: use only trusted binaries and fixed arguments.

69.4 Streamable HTTP transport

Streamable HTTP connects to an MCP endpoint. EnvoyMesh defaults to safe local or HTTPS destinations and requires an explicit override for remote plain HTTP.

69.5 List external tools

Call `mesh.mcp.list_tools` naming the configured consumer. Returns MCP tool schemas for agent planning; empty or error responses usually mean process exit, bad URL, or bearer mismatch.

69.6 Call an external tool

Invoke `mesh.mcp.call_tool` with tool name and JSON arguments. MCP content blocks map into EnvoyMesh artifacts suitable for task results and audit.

69.7 Content and artifact mapping

Text, image, audio, resource links, and structured MCP content become typed artifacts. Inspect mapped output before publishing to bonded peers above public sensitivity.

69.8 Timeouts and response limits

Consumer sessions honor SDK timeouts and node payload size caps. Oversized MCP responses are rejected before entering the semantic firewall or vault.

69.9 Remote-URL safety

Remote URL validation reduces SSRF risk: prefer HTTPS, avoid private metadata services, keep loopback as the default, and enable remote plain HTTP only for a controlled development network.

69.10 Troubleshoot an MCP consumer

Verify `command` vs `url`, stdio vs Streamable HTTP transport, `allowRemoteHttp` for dev plain-HTTP endpoints, and `bearerToken`. Audit JSONL distinguishes connection failures from schema validation errors.

70. Use EnvoyMesh as an MCP Server

Figure 9 — MCP consumer vs server: the same EnvoyMesh node can consume external MCP tools (Direction A) or expose mesh tools to MCP clients like Claude Desktop (Direction B). Data direction reverses.

70.1 What MCP server mode exposes

The stdio adapter answers MCP JSON-RPC (`initialize`, `tools/list`, `tools/call`) and forwards to the home bridge HTTP listener (default `http://127.0.0.1:3031`), exposing registered `mesh.*` tools.

70.2 Start `envoymesh mcp-server`

Start the adapter through the EnvoyMesh CLI `mcp-server` command (for example, the packaged or workspace CLI invocation documented for your release). It communicates by stdio and forwards calls to the configured local bridge.

70.3 Connect Claude Desktop

Edit `~/Library/Application Support/Claude/claude_desktop_config.json` (macOS) or `%APPDATA%\Claude\claude_desktop_config.json` (Windows):

```
"mcpServers": {
  "envoymesh": {
    "command": "npx",
    "args": ["envoymesh", "mcp-server"],
    "env": { "ENVOYMESH_BRIDGE_SECRET": "YOUR_SECRET" }
  }
}
```

Restart Claude Desktop; confirm **envoymesh** appears under MCP servers. Match `ENVOYMESH_BRIDGE_SECRET` to the node's `bridge.secret`.

70.4 List EnvoyMesh tools

Ask Claude or Cursor to list MCP tools—you should see `mesh.*` entries from the home tool registry. An empty list usually means the bridge listener is down or the bridge secret mismatches.

70.5 Call a mesh tool

MCP `tools/call` reaches the bridge; the node runs Bonds checks and tool handlers locally. Start with read-only tools (contacts, ping) before invoking vault or spend actions.

70.6 Bridge authentication

Set `bridge.secret` on the node and the same value in `ENVOYMESH_BRIDGE_SECRET` or pass `--bridge-token YOUR_SECRET` to the adapter. Misaligned secrets return 401 before any tool runs.

70.7 Local and remote bridge URLs

Default: `npx envoymesh mcp-server --bridge http://127.0.0.1:3031`. For LAN hosts add `--bridge-allow-remote` and point `--bridge` at the node's bridge URL—avoid plain HTTP with live secrets on untrusted networks.

70.8 Error handling and audit tags

Adapter failures surface as MCP tool errors; successful calls log `auditTag: "mcp-server"` on the node. Distinguish bridge 401 (auth) from tool deny (bond/mandate) in audit summaries.

70.9 Current tools-only scope

Current server scope exposes tools. MCP resources and prompts are not automatically translated into the Vault or Library.

70.10 OAuth and MCP resources — future work

Future. Bearer authentication is current; OAuth 2.1 and broader MCP resources/prompts support are deferred until required by a deployment.

70.11 Troubleshoot the MCP server

Run manually: `npx envoymesh mcp-server --bridge http://127.0.0.1:3031`. Confirm the node bridge listener is up, secrets match, and tools are enabled in node config. See `docs/phase-48-interop-smoke.md` for the full checklist.

71. A2A Agent Cards

71.1 What an Agent Card is

A2A Agent Card JSON describes name, skills, capabilities, security schemes, and the JSON-RPC interface URL. EnvoyMesh translates native agent cards through `toA2AAgentCard()` before relay publication.

71.2 Discover the well-known Agent Card

An A2A client fetches `/.well-known/agent-card.json` from the configured relay HTTP origin. Publication is opt-in through A2A bridge settings.

71.3 Identity and provider fields

Fields derive from EnvoyMesh profile and agent-network metadata—display name, provider URL, owner-linked hints. The relay may attach optional Ed25519 signatures (`type: "envoymesh-ed25519"`) so clients can detect tampering.

71.4 Skills and capabilities

Native capabilities map to A2A skills with strength tags from the capability index. Clients use skills for discovery fit—not as authorization; bearer tokens and Bonds still gate task execution.

71.5 Supported interfaces

`supportedInterfaces[0].url` targets `/.well-known/a2a/jsonrpc` on the configured gateway. Fetch the card first, then POST JSON-RPC to that URL with the same bearer token used for tasks.

71.6 Streaming capability

When `capabilities.streaming: true` and metadata includes `x-envoymesh-taskBridgeStatus: "available"`, clients may call `message/stream` for SSE task updates instead of polling `tasks/get` alone.

71.7 Signed Agent Cards

The relay can sign the Agent Card with its Ed25519 control identity so clients can detect alteration. Consumers must still decide whether they trust that signer and endpoint.

71.8 Relay publication

Enable with `--a2a-bridge` / `ENVOYMESH_A2A_BRIDGE=1` and set `--a2a-gateway-url` / `ENVOYMESH_A2A_GATEWAY_URL`. The card is served at `GET /.well-known/agent-card.json` on the relay HTTP port (commonly `:15432`).

71.9 Privacy and field filtering

Sensitive profile fields may be omitted from the public card. Treat published cards as discovery metadata—task authorization still requires bearer tokens, Bonds tiers, and home-owner-signed mandates.

71.10 Troubleshoot card discovery

Run `curl -sS https://relay:15432/.well-known/agent-card.json | jq .` — expect HTTP 200 when the bridge is enabled, 503 when disabled. Verify the gateway URL hostname matches the TLS certificate clients use.

72. A2A Tasks

72.1 A2A JSON-RPC endpoint

The public relay exposes `POST /.well-known/a2a/jsonrpc`; the home bridge uses the loopback `/a2a/jsonrpc` path. The relay authenticates and forwards rather than executing the model itself.

72.2 Bearer-token authentication

Bearer tokens map an external caller to an EnvoyMesh owner identity. Keep tokens unique, rotate them, and bind them to the minimum intended trust relationship.

72.3 Send a task with `message/send`

`message/send` supplies user message Parts and receives an A2A Task. The production executor applies bond policy, mints an owner-authorized mandate, and dispatches through the native task runtime.

72.4 Stream updates with `message/stream`

`message/stream` returns server-sent task updates for clients that need progress without polling. Close abandoned streams and observe gateway timeouts.

72.5 Poll with `tasks/get`

`tasks/get` retrieves the current persisted task mapping and state. Use it after a synchronous request returns working or after reconnecting.

72.6 Cancel with `tasks/cancel`

`tasks/cancel` requests native cancellation for the authenticated owner's task. Owner scoping prevents one token from controlling another owner's task.

72.7 A2A-to-EnvoyMesh policy gates

The production executor calls Bonds `evaluatePolicy` for tiers `self`, `direct`, and `referred`. Bond denial returns A2A state `auth-required` — no home-owner-signed mandate is minted for blocked or public strangers.

72.8 Production task execution

Pipeline: bearer auth → Bonds gate → home-owner-signed `task.mandate` + `task.propose` → `handleDaemonTaskInbound` (runtime guard + journal) → persist mapping for `tasks/get` and `tasks/cancel`. Default leaves tasks `running` until a mesh `task.result` arrives.

72.9 Relay-to-home forwarding

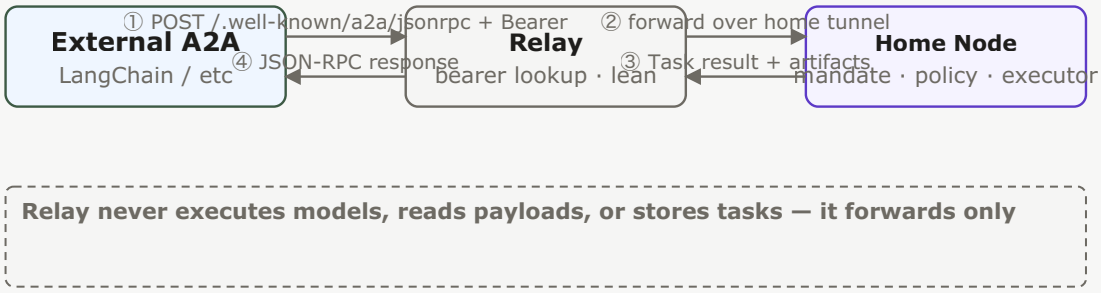

Figure 11 — A2A relay-to-home forwarding: the relay authenticates the bearer token and forwards to the owner's home node, which owns the mandate, policy, model, and task storage. The relay stays lean.

The relay looks up the token owner's home and forwards over the home tunnel. It remains lean: policy, mandates, model execution, task storage, and artifacts stay on the home node.

72.10 Error codes

JSON-RPC errors follow A2A conventions; Bonds denial surfaces as task state `auth-required`. Relay `forwardToHome` preserves upstream HTTP status from the home bridge when the tunnel or node rejects a call.

72.11 Troubleshoot A2A tasks

Confirm bearer token maps to the intended owner, home tunnel is up for relay forwarding, and audit shows mandate/propose acceptance. Poll `tasks/get` with the returned task id; use `tasks/cancel` only for that owner's tracked tasks.

73. A2A State, Artifact, and File Mapping

73.1 EnvoyMesh-to-A2A task states

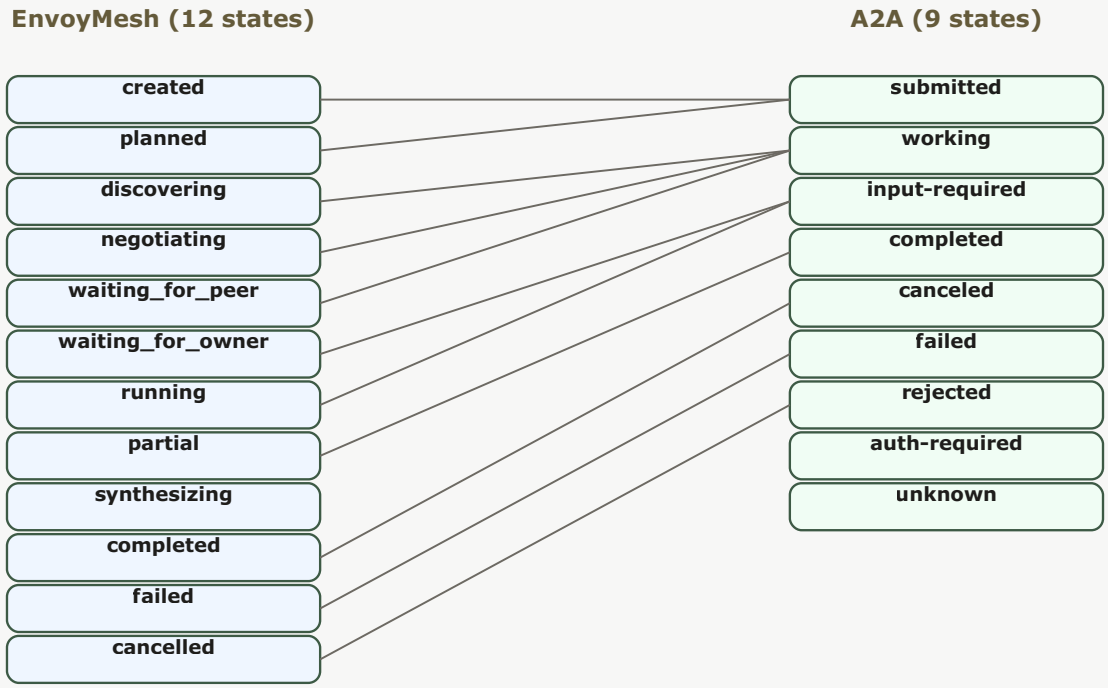

Figure 17 — EnvoyMesh-to-A2A state mapping: 12 internal states collapse to 9 A2A states. Many-to-one merges (e.g. waiting_for_peer + waiting_for_owner → input-required) are handled by a2a-state-map.ts.

Twelve internal lifecycle states collapse to nine A2A states via `a2a-state-map.ts`. Document the mapping when building client UX that polls `tasks/get` or renders SSE events from `message/stream`.

73.2 Submitted, working, and input-required

Fresh A2A tasks often appear `submitted`, then `working` after mandate acceptance through `handleDaemonTaskInbound`. `input-required` mirrors owner-approval stalls or missing parameters the executor cannot infer.

73.3 Completed, failed, and canceled

Terminal A2A states align with mesh `completed`, `failed`, and `cancelled` (A2A spells `canceled` with one "l"). Artifacts attach on completed paths when the mapper finds native results.

73.4 Rejected, auth-required, and unknown

`rejected` follows worker `task.reject` or executor refusal; `auth-required` signals Bonds failure for the bearer-mapped owner; `unknown` covers untracked or expired task IDs not in `a2a-bridge-tasks.json`.

73.5 Text Parts

Inbound message text becomes objective context and may map to TextArtifacts in results. Outbound text artifacts become A2A Text Parts in bridged task payloads.

73.6 Data Parts

Structured JSON Parts map to structured artifacts with schema hints. Validate schema and sensitivity before acting on machine-readable output from external agents.

73.7 File Parts

File Parts carry URIs like `<gateway>/vault/<encodedPath>?hash=…`. Fetch with the same A2A bearer used for JSON-RPC—the relay proxies `GET /vault/*` to the home bridge via home-tunnel.

73.8 Composite results

Composite EnvoyMesh artifacts expand into multiple A2A Parts where the mapper supports child weights and attribution metadata.

73.9 Vault-backed file URLs

File artifacts may be represented as authenticated Vault-backed URLs. The endpoint validates path safety and can check the expected content hash before serving bytes.

73.10 Hash validation and access control

Vault HTTP verifies path safety, A2A bearer auth, and optional `?hash=` against sha256 (hex, base64url, or `sha256:` prefix). Hash mismatch returns 403/404 without leaking whether the path exists.

Part X — Networking and Relays

74. Peer-to-Peer Networking

74.1 Local and internet connectivity

Nodes can discover and dial peers on a local network or across the Internet. The final path depends on advertised addresses, NAT, relay availability, and transport compatibility.

74.2 TCP, QUIC, and WebSocket paths

EnvoyMesh uses libp2p over TCP and QUIC for direct peer links, and WebSocket where relays or NAT require HTTP-friendly transport. The Social UI and EnvoyGo usually reach the home node over WebSocket when you are off-LAN. Transport choice affects reachability only; application messages still require signed envelopes and bond policy after the link is up.

74.3 Local discovery

On the same network, nodes can find each other through mDNS without typing multiaddrs. Use local discovery when testing two machines on one Wi-Fi segment before adding WAN bootstrap peers. Guest networks, VPN split tunneling, or disabled multicast can block mDNS—fall back to printed multiaddrs or relay check-in when LAN discovery fails.

74.4 Distributed discovery

Across the Internet, nodes publish and resolve rendezvous records through configured bootstrap peers and relays (DHT plus relay lookup intents). WAN discovery needs reachable bootstrap multiaddrs and a compatible discovery profile (for example `wan-default` in source runs). Zero bootstrap peers or an empty relay roster in `connectivity-status` usually indicates bootstrap or firewall misconfiguration, not a missing identity.

74.5 Direct connections

When both sides expose reachable addresses, libp2p prefers a direct dial before any relay hop. Direct paths reduce latency and keep relay operators out of the signed-envelope data plane. After every node restart, copy the latest `Listening on:` multiaddr—dynamic ports invalidate saved addresses.

74.6 NAT and firewall behavior

Home routers and corporate firewalls often block inbound TCP unless you forward a port or use circuit relay. Allow outbound TCP from the node process on both peers when diagnosing WAN connectivity. `--connectivity-strict` intentionally fails startup when all bootstrap probes fail; disable it only temporarily for diagnosis, then restore strict mode.

74.7 Connection upgrades

libp2p negotiates identify, stream muxers, and optional relay reservations below the application layer. Successful transport upgrade does not grant trust—bond policy still applies to every intent. Enable `--p2p-debug` or audit `p2p.trace` rows when a peer connects but signed envelope exchange fails afterward.

74.8 Signed envelope streams

Application traffic travels as Ed25519-signed `EnvoyEnvelope` records on libp2p streams, not as opaque bytes trusted by IP alone. The inbound guard checks size, schema, signature, and replay before the bond engine runs. A live TCP session without valid signatures still produces deny or reject outcomes in audit.

74.9 Offline peers and retries

Peers that restart, sleep, or roam may be unreachable until relay registration and advertised addresses refresh. Clients retry discovery with updated multiaddrs; temporary offline status is not the same as a blocked bond. Confirm relay roster freshness and remote check-in before treating a failure as a trust problem.

74.10 Network diagnostics

Run `npm run cli -w @envoymesh/node -- connectivity-status --profile <path>` for bootstrap counts and relay hints; add `--rich` for a text snapshot. Export audit timelines with `--include-p2p-trace` when sharing connectivity evidence. Use the same absolute profile path for the node, CLI, and Social—a mismatched path makes diagnostics look empty even when traffic exists.

75. Relay Services

75.1 Why a relay may be needed

Relays help when NAT, firewalls, or mobility prevent a direct libp2p dial. They provide rendezvous, lookup, optional WebSocket entry, and circuit forwarding—not account login or message decryption authority. Try direct paths first; add a relay when peers cannot learn each other's reachable addresses.

75.2 What a relay can and cannot do

A relay helps with rendezvous, lookup, WebSocket access, and forwarding. It does not run user models, become an identity authority, or receive permission to bypass signed-envelope policy.

75.3 Select a relay

Choose a relay you trust for connectivity metadata: community bootstrap presets, an operator-run fleet node, or a private relay you administer. Record its bootstrap multiaddr and verify it supports the relay protocols your build expects (check-in, lookup, and circuit reservation on current releases). Avoid switching relays frequently while debugging—stale registrations confuse lookup results.

75.4 Connect through a relay

Start the node with `--relay` and `--bootstrap "<relay-multiaddr>"` (or an equivalent Settings entry) so it checks in and publishes a circuit address. Remote peers dial `/p2p-circuit/p2p/<your-peer-id>` when direct paths fail. Confirm both sides use compatible bootstrap lists and the same major protocol version.

75.5 Relay check-in and lookup

Checked-in nodes register with `relay.checkin`; seekers resolve them through `relay.lookup` without learning private home IPs by default. Audit rows such as `relay.checkin.ok` and `relay.lookup.response` confirm healthy registration. An empty roster on the relay usually means clients never completed check-in or used the wrong profile path.

75.6 Routing hints

Relays may return sibling or fleet hints so clients try alternate bootstrap paths before giving up. Hints affect where to dial next, not who may send which intent. Treat hints as optimization; bond policy and signatures still gate every application message.

75.7 Use multiple relays

Configure several bootstrap relays for redundancy when one host is down or geographically distant. Multi-homed clients can check in to more than one relay while keeping a bounded relay book locally. More relays improve reachability options; they do not merge trust stores or identities.

75.8 Privacy when using a relay

Relays see connection metadata—peer IDs, timing, and forwarding paths—not decrypted application payloads inside signed envelopes. Choose relay operators accordingly, especially for sensitive workflows. End-to-end intent authorization still depends on bonds and mandates, not on hiding traffic from your chosen relay.

75.9 Change or remove a relay

Update bootstrap multiaddrs in Settings or launch flags, restart the node, and verify fresh check-in before removing an old relay from your book. Peers caching stale circuit addresses may fail until they rediscover you. Document the change for contacts who pinned your old relay-dependent multiaddr.

75.10 Relay troubleshooting

Run `relay-status` on the relay profile and `connectivity-status` on clients; compare roster totals, bootstrap counts, and recent `p2p.trace` rows. Common fixes: correct `--bootstrap` multiaddr, open outbound TCP, align profile paths, and recopy post-restart listen addresses. See QuickStart WAN troubleshooting and Appendix K for command references.

76. Operate a Relay

76.1 Operator requirements

Operator. Run a relay only if you can maintain a stable host, public reachability, key material, TLS for public HTTP/WebSocket surfaces, access controls, monitoring, upgrades, and abuse response.

76.2 Install the relay

Build or deploy `apps/relay` from a current repository release on a stable host with a public TCP listener. Package installs and source runs both work; keep the relay version aligned with client nodes to avoid reservation handshake skew. Document the bootstrap multiaddr you will give to fleet clients.

76.3 Configure identity and listen addresses

Assign the relay its own libp2p key material and bind to `/ip4/0.0.0.0/tcp/<port>` (or your operator standard). Print and archive the resulting `/ip4/.../tcp/.../p2p/...` multiaddr for bootstrap configuration. Separate relay identity from any personal EnvoyMesh owner profile you use elsewhere.

76.4 Configure public mode

Public mode advertises an externally reachable address (`--advertise-addr` on current builds) so circuit relay reservations work across NAT. Without it, a relay may appear discovery-only—clients connect for lookup but fail reservation handshakes. Match advertised addresses to DNS or firewall rules you actually expose.

76.5 Configure WebSocket access

Enable the relay HTTP/WebSocket surface when thin clients or browser Social instances must tunnel through the relay. Terminate TLS at the edge for production hostnames. Restrict administrative routes from the public Internet even when user WebSocket paths are open.

76.6 Configure administrator access

Protect relay admin APIs and metrics with operator credentials, network ACLs, or mutual TLS as your deployment model requires. Never expose unauthenticated admin endpoints on public interfaces. Rotate credentials when operators leave and audit access changes.

76.7 Publish DNS and TLS endpoints

Map stable DNS names to relay listen addresses and install valid TLS certificates for HTTPS and secure WebSocket. Clients embed these names in bootstrap presets and pairing flows. Keep certificate renewal automated—expired TLS breaks mobile and browser clients silently.

76.8 Monitor health, metrics, roster, and logs

Track process health, relay roster size, lookup latency, and error rates from relay audit snapshots and host metrics. Alert when roster drops unexpectedly or check-in failures spike. Correlate relay-side traces with client `connectivity-status` during incidents.

76.9 Upgrade and back up a relay

Back up relay key material and configuration before upgrades; schedule maintenance when client traffic is low. Roll forward one relay at a time in multi-relay fleets so bootstrap lists always include a healthy peer. Test circuit reservation after upgrade before decommissioning the old binary.

76.10 Respond to abuse

Rate-limit or block peer IDs that flood lookup, reservation, or WebSocket endpoints. Preserve audit evidence with correlation IDs when escalating. Document your abuse contact and takedown process for fleet customers—relays carry connectivity metadata even though they do not read envelope payloads.

77. Multi-Relay Fleets

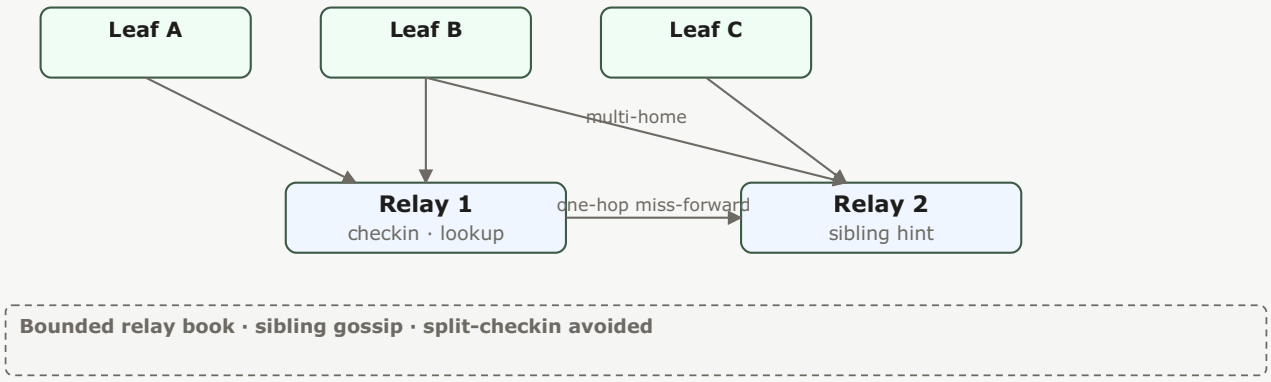

Figure 13 — Multi-relay fleet: leaf nodes multi-home across relays; siblings exchange hints and one-hop miss-forward lookups. The bounded relay book prevents split-checkin failures.

77.1 Why use several relays

Multiple relays improve geographic coverage, uptime, and bootstrap redundancy. Clients can home to several bootstrap entries while keeping a bounded local relay book. Fleet operators standardize presets so end users are not locked to a single community node.

77.2 Configure bootstrap presets

Bootstrap presets bundle known-good relay multiaddrs (for example `public-libp2p` in source runs) so new nodes start with WAN discovery enabled. Operators can ship private presets for enterprise fleets. Presets seed connectivity—they do not import contacts or trust relationships.

77.3 Client multi-homing

A node may check in to several relays and maintain multiple circuit addresses simultaneously. Multi-homing helps roaming users stay reachable when one relay region is degraded. Local relay-book pruning keeps storage bounded; stale entries drop after configured freshness windows.

77.4 Bounded relay books

Each node stores a capped relay book (`relay-book.json` in the profile directory) rather than an unbounded global directory. Eviction policies favor recently verified relays. Operators should monitor whether legitimate relays are aged out too aggressively in long-idle deployments.

77.5 Sibling hints

Sibling hints tell a lookup client about alternate relays in the same fleet when the primary miss occurs. They reduce failed dials during partial outages. Hints are optional optimization; clients must still complete check-in and lookup on the chosen target.

77.6 One-hop lookup forwarding

When a relay does not hold a registration, it may forward lookup to a sibling once rather than building a full hierarchical graph. This covers many fleet topologies today without ancestor/parent/child coordination. Deep multi-hop forwarding remains limited—see 77.10 for deferred hierarchical work.

77.7 Fleet health and diagnostics

Compare roster counts, check-in rates, and lookup success across fleet relays using `relay-status` and relay audit snapshots. Run live WAN validation tests from representative client profiles after configuration changes. Standardize profile paths in runbooks so CLI and UI diagnostics align.

77.8 Live WAN validation

Prove cross-network paths with two profiles on different networks bootstrapping to the same relay, then exercise ping, chat, or audit-verified intents. QuickStart's cross-network relay walkthrough and `npm run poc:discovery` smoke modes are reference flows. Record correlation IDs from both sides when filing connectivity bugs.

77.9 Current coordination limits

Today's multi-relay support covers bounded books, sibling hints, and one-hop miss forwarding—not a complete hierarchical relay graph or global relay marketplace. Plan fleet layouts accordingly. Features marked **Deferred** in 77.10 are design targets, not hidden toggles.

77.10 Full hierarchical relay graph — deferred

Deferred. Current multi-relay coordination supports bounded books, sibling hints, and one-hop miss forwarding; the complete hierarchical ancestor/parent/child graph remains future work.

Part XI — Terminals, Browser, and Advanced Use

78. Terminals

78.1 Open the Terminals view

Open Terminals from Social's navigation or start a session from an eligible chat thread. The view lists active PTY sessions on the home node and offers controls to attach, resize, or end them. EnvoyGo exposes the same capability through its terminals screen when paired to home.

78.2 Create and manage a terminal session

Create a session to spawn a shell PTY on the home desktop node; name or tag sessions when the UI supports it so you can find long-running work. Multiple clients can attach read/write depending on policy. Sessions persist until closed or until the node restarts—save important output elsewhere.

78.3 Understand the home PTY

The terminal process runs as a PTY on the home desktop node. EnvoyGo and the Social UI are clients of that session, so commands execute with the home user's operating-system permissions.

78.4 Use terminal input and output

Type commands in the terminal pane; stdout and stderr stream back over the authenticated WebSocket or JSON-RPC tunnel. Large output may be truncated in mobile clients—prefer desktop Social for heavy logs. Copy/paste behavior follows your platform and browser constraints.

78.5 Use agent-assisted terminal mode

When agent assist is enabled, EnvoyAI may propose shell commands based on your conversation context. Review every proposed command before execution—agent assist does not bypass approvals you configured. Denied commands should appear in audit with an explicit outcome.

78.6 Access terminals from EnvoyGo

EnvoyGo attaches to home-node PTY sessions over the paired JSON-RPC transport; commands still execute on the desktop with its OS permissions. Keep the home node awake and reachable via relay or LAN while using mobile terminals. Treat phone access as remote control of a powerful surface.

78.7 Security and approvals

Terminal access is powerful and can alter files, credentials, or software. Restrict pairing, require approvals for agent-suggested commands, inspect commands before execution, and close abandoned sessions.

78.8 Close a session safely

Exit long-running programs cleanly (`exit`, `Ctrl+D`, or application-specific stop commands) before closing the terminal tab. Abrupt disconnects may leave background jobs running on the home node. Revoke pairing or change approvals if you shared a session unintentionally.

78.9 Troubleshoot terminals

If attach fails, confirm the home node is running, WebSocket or relay paths are healthy, and your session token is valid. Check audit for auth-required or deny rows tied to terminal RPCs. Restart the node only after closing sensitive sessions you do not want orphaned.

78.10 External terminal integrations

Some releases integrate external terminal products through the same home PTY boundary rather than granting them libp2p keys. Configure integrations in Settings and restrict them to trusted networks. External tools inherit home-node OS privileges—apply the same caution as local shell access.

79. Browser

79.1 Open the Browser view

Open Browser from Social or EnvoyGo to browse permitted mesh content. The view resolves `envoy://` URLs through your home node's policy boundary, not the public web by default. Pairing or local node availability is required before content loads on mobile.

79.2 Navigate `envoy://` content

An `envoy://` URL identifies mesh-hosted content by author and path rather than a public web server. Resolution passes through the paired or local node and its trust policy.

79.3 Browse authors and topics

Browse by author DID, published topics, or feeds your bond policy exposes. Strangers may see only public-sensitivity material; bonded contacts may see friends-level notes when authors published them. Empty lists often mean policy denial, not a broken index—check trust tier and sensitivity labels.

79.4 Use history and bookmarks

Browser history and bookmarks are stored locally in your profile for quick return to mesh pages you already accessed. Clearing history does not unpublish remote content. Bookmarks reference `envoy://` paths; if an author moves content, update or remove stale entries.

79.5 Publish from the Browser

Publishing creates or updates mesh-visible content from notes and pages you own, subject to per-item sensitivity toggles in Library. Public items become queryable via `knowledge.query` within rate limits; friends-level items require appropriate bonds. Preview sensitivity before publishing sensitive drafts.

79.6 Subscribe to feed updates

Subscribe to authors or topics to receive feed updates when new mesh content appears and policy allows delivery. Subscriptions respect bond and sensitivity rules—dropping a bond may silently stop updates. Push notifications on EnvoyGo depend on home-node forwarding and platform permission settings.

79.7 Use Browser on EnvoyGo

EnvoyGo renders Browser through the paired home node, mirroring desktop policy results on a smaller screen. Keep the home node online; cached pages may be stale when offline. Read-only browsing does not substitute for Library editing—create notes on desktop when possible.

79.8 Paired-mode requirements

Mobile Browser requires a completed EnvoyGo pairing with a healthy home JSON-RPC session. Without pairing, the phone has no vault, bond store, or signing context to resolve `envoy://` URLs. Re-pair if session tokens expire or after major home-node identity changes.

79.9 Troubleshoot Browser content

When a page fails to load, verify the URL author exists, sensitivity allows your trust tier, and the home node can reach the publishing peer. Audit may show bond deny or schema reject for fetches—not generic HTTP 404 semantics. Retry after bond acceptance or author republish.

80. Advanced Settings

80.1 Node settings

Node settings cover profile identity, display name, discovery profile, listen addresses, and service ports for the home runtime. Changes often require a restart to take effect. Note your profile directory path before editing paths or ports so CLI and Social stay aligned.

80.2 Network and bootstrap settings

Configure bootstrap multiaddrs, presets, strict connectivity mode, and advertised listen addresses here or via equivalent launch flags. Misconfigured bootstrap lists are the most common WAN failure mode. After changes, run `connectivity-status` and inspect audit for bootstrap probe results.

80.3 Relay settings

Enable client relay mode, set bootstrap relays, and manage the local relay book from relay settings. These controls affect how others dial you—not whom you trust. Pair relay changes with `relay-status` on both client and relay operator profiles during rollout.

80.4 AI and model settings

AI settings select providers, model routes, semantic firewall behavior, and EnvoyAI/OpenClaw gateway integration. API keys and model credentials live in profile configuration—back them up securely and never paste them into support bundles. Disable remote models when air-gapped policy requires local-only inference.

80.5 External-agent settings

External-agent settings configure HomeClaw, Hermes, OpenHuman, or custom HTTP bridges, including ports, bearer secrets, and enabled presets. Bridges forward policy-checked tools—they do not receive raw libp2p keys. Rotate bridge secrets after compromise and review action history against audit JSONL.

80.6 Agent Network settings

Agent Network settings control opt-in collaboration, worker visibility, Team job budgets, and orchestration limits. Both sides must opt in and hold appropriate bonds before agents collaborate. Start with manual approvals and small mandates before enabling automatic spend rebalance.

80.7 Knowledge and storage settings

Point the vault at `shared_vault/` (default in source runs) or a configured path; enable Library plugins such as Obsidian or MCP under Knowledge Base. Sensitivity defaults and indexing options live here. Large vault moves require re-index time and disk space on the home node.

80.8 Call and TURN settings

Voice call settings include STUN/TURN URLs, credentials, and platform-specific push topics for EnvoyGo. Misconfigured TURN prevents calls across strict NAT. Video remains limited on current releases—confirm feature status before training users on video workflows.

80.9 Notification settings

Configure push notification providers (APNS/FCM) on the home node and permission prompts on EnvoyGo. Delivery depends on home-node forwarding, relay reachability, and OS battery policies. Test with a low-noise channel before enabling alerts for every chat message.

80.10 Logging and diagnostics

Adjust verbosity, p2p trace capture, and diagnostic exports from logging settings or CLI flags such as `--p2p-debug`. Audit JSONL remains the authoritative allow/deny trail even when console logging is quiet. Redact secrets before sharing logs—see Appendix K.

80.11 Experimental settings

Experimental toggles gate features still receiving validation; interfaces and defaults may change between releases. Enable them only on non-production profiles until release notes mark them **Available**. Document which toggles you enabled when reporting bugs.

80.12 Restore recommended defaults

Restore recommended defaults resets risky or nonstandard settings while preserving identity keys and vault content. Use this after connectivity experiments or failed agent-bridge trials before escalating support. Export a profile backup first if you customized many fields.

Part XII — Privacy, Trust, and Security

81. Identity and Key Safety

81.1 Owner identity

The owner identity is the long-lived root representing the human. It signs device certificates, mandates, and other authorizations, so its private key deserves the strongest backup and access protection.

81.2 Device identity

Each device has its own identity authorized by the owner. This lets you revoke one lost machine without changing the human's owner identity everywhere.

81.3 Agent identity

An agent has a distinct key and an owner-signed credential linking it to the owner. Peers can therefore verify which owner authorized an agent without treating the agent key as the owner key.

81.4 Peer identity

A peer identity is the runtime sender identity used for signed envelopes and networking. It is not interchangeable with owner, device, or agent identity even when one node holds several of them.

81.5 Ed25519 signatures in plain language

Ed25519 lets a sender create a compact signature with a private key and lets others verify it with the public key. Verification proves message integrity and key possession, not that the human is trustworthy.

81.6 DID presentation

DIDs (`envoy:owner:…`, `envoy:device:…`, `envoy:agent:…`) label identities in UI and audit without replacing key verification. Present DIDs alongside fingerprints when teaching someone to verify you. A matching string is not proof unless signature checks succeed.

81.7 Key storage

Private keys stay in the node profile with restrictive file modes (`0o600` on sensitive JSON). EnvoyGo stores pairing secrets in OS secure storage, not owner root keys. Never copy private key files into chat, email, or cloud drives without encryption.

81.8 Backup and recovery

Back up owner key material, device certificates, trust store, and Vault together on encrypted media tested with a restore drill. Losing the only owner key backup may require a new identity. Separate hot backups from offline copies to limit ransomware spread.

81.9 Device certificates

Device certificates are owner-signed documents binding a device public key to your owner identity. Pairing EnvoyGo or adding a laptop mints a new certificate chain. Revoke certificates promptly when hardware is lost—see Chapter 88.

81.10 Revocation

Revocation records invalidate device certificates or mandates without rotating the owner key. Publish revocations from a still-trusted device and audit that peers reject stale credentials on next handshake. Owner-key compromise requires identity migration, not certificate revoke alone.

82. Bonds and Trust Policy

Trust Tier	Meaning	What it allows	Sensitivity ceiling
blocked	Deny all	—	—
public	Stranger	ping · narrow discov	public
referred	Introduced	knowledge · limited	friends
direct	Friend	full collaboration +	friends · trusted

Figure 3 — Bond trust tiers: each tier caps what a contact may do and the maximum data sensitivity. Higher tiers unlock richer collaboration; blocked denies everything.

82.1 What a bond means

A bond records a local trust tier for another owner and drives deterministic policy. Identity answers “who signed”; the bond answers “what may this relationship do.”

82.2 Self trust

Self is the local owner’s highest trust context and can reach private sensitivity within local policy.

82.3 Direct trust

Direct represents a deliberately trusted contact and permits the broadest remote workflows, generally up to friends-level sensitivity unless additional policy restricts them.

82.4 Referred trust

Referred represents limited trust established through introduction or constrained onboarding. Knowledge and Team job operations remain more restricted and may require approval.

82.5 Public trust

Public is the stranger/default tier. Only narrow discovery, ping, introduction, and public-knowledge behaviors are eligible; it is not sufficient for Team job recruitment.

82.6 Blocked trust

Blocked denies communication regardless of advertised capability. Use it for abuse, compromise, or a relationship that should no longer reach the node.

82.7 Capability gates

Capabilities map intents and tool actions to allow, deny, challenge, or approval outcomes. A contact may be direct yet still denied a specific vault action if mandate or sensitivity forbids it. Inspect audit `deny` rows for the missing capability name.

82.8 Sensitivity ceilings

Each trust tier caps maximum **sensitivity** (`public` / `friends` / `trusted` / `private`) for knowledge and data operations. Requests above the ceiling fail closed even when the intent is otherwise allowed. Lower sensitivity before sharing with referred contacts.

82.9 Challenges and approvals

Stranger-tier or high-risk actions may return **challenge** or **approval** outcomes instead of immediate allow. Human approvals land in the approval queue on the home node. Do not bypass approvals by retrying the same payload repeatedly.

82.10 Change or revoke trust

Change trust tier in contact settings or issue signed revocation for devices and mandates. Downgrade takes effect on the next inbound operation; already shared files remain on peer nodes until they delete local copies. Document tier changes for future incident review.

83. Signed Messages and Protocol Safety

83.1 Signed messages

Every envelope is Ed25519-signed over canonical JSON so tampering is detectable. Unsigned or wrongly signed payloads fail inbound guard before policy runs. Signatures prove key possession, not moral trust—pair with bonds.

83.2 Sender and recipient roles

Roles (`human`, `agent`, `system`) are schema-enforced per intent—`chat.message` requires human-to-human, task intents require agent-to-agent. Role mismatch rejects at validation. UI choices must match the intended role path.

83.3 Typed intents

Intents are typed (`chat.message`, `knowledge.query`, `task.propose`, …) with Zod-validated payloads. Unknown intents fail closed. Agents and integrations must use the correct intent for the operation, not opaque blobs.

83.4 Message and correlation identifiers

`messageId` identifies one envelope; `correlationId` stitches multi-step flows in audit across peers. Include correlation IDs when sharing diagnostics. Replay dedup uses message IDs within the inbound guard window.

83.5 Schema validation

Inbound payloads pass schema validation before bond evaluation. Malformed JSON or field violations return structured errors without touching Vault or models. Client bugs show up as validation failures in audit, not silent drops.

83.6 Signature verification

Verification recomputes canonical JSON and checks Ed25519 signatures against the sender public key, which must hash to `senderPeerId`. Failed verification denies before policy. Never disable verification for convenience.

83.7 Replay protection

The inbound guard rejects duplicate `messageId` values within a replay window to limit replay attacks. Clock skew affects ordering but not signature validity. Restarting nodes does not reset peer replay state mid-session.

83.8 Rate and size limits

Diplomat enforces rate and size caps on streams before expensive work. Oversized chat or file payloads deny early. Burst traffic from one peer may throttle—back off rather than splitting into many tiny messages.

83.9 Malformed message handling

Malformed messages are rejected with audit summaries; guards do not crash the node on bad input. Persistent malformed traffic from a peer is grounds for block tier. Capture one sample for diagnostics without enabling verbose payload logging in production.

83.10 Protocol versioning

Protocol version fields gate incompatible peers during handshake. Mixed-version fleets should upgrade relays and nodes together per release notes. Version mismatch manifests as connect failures, not partial silently broken chat.

84. Security Architecture

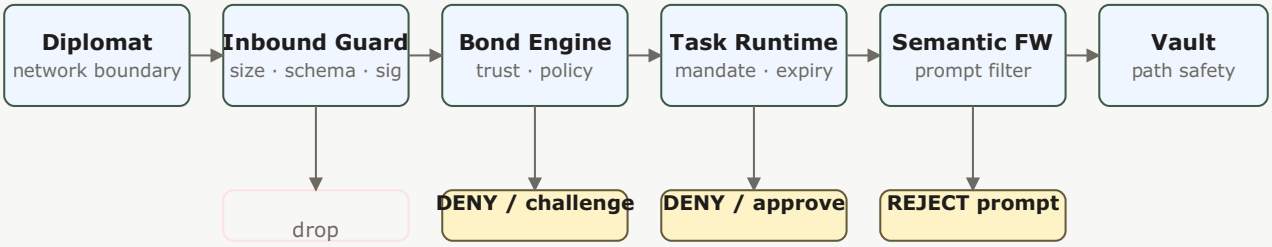

Figure 5 — Security pipeline: six ordered layers from network boundary to Vault. Each can deny, challenge, or require approval; the chain fails closed and no single layer is trusted alone.

84.1 Network boundary

The Diplomat (network boundary) accepts bytes and connections but has no direct model or filesystem authority. It parses, limits, and forwards only validated requests.

84.2 Inbound guard

Inbound guard checks size, schema, signature, and replay before bond engine. It has no Vault or model access —only accept or reject. Most user-visible “message failed” traces start here or at bond deny.

84.3 Bond policy engine

The Bond Engine turns trust tier, intent, capability, and sensitivity into an allow, deny, challenge, or approval decision before privileged work proceeds.

84.4 Task runtime guard

Task runtime guard enforces mandate expiry, cancellation, collect-N termination, and action lists during agent work. Even allowed bonds cannot exceed an expired mandate. Review task journal alongside audit when jobs stall mid-flight.

84.5 Semantic firewall

The semantic firewall (part of the model boundary, historically called the Brain layer) rejects empty, oversized, or control-character-laden model prompts and normalizes excessive newline runs before a model sees them.

84.6 Model boundary

The model router receives only approved context after bond and task guards; external agents never bypass it via bridge tools. Prompts pass the semantic firewall before provider calls. Model output does not auto-execute vault writes.

84.7 Vault isolation

The Vault enforces path safety and explicit operations. Neither a remote peer nor an external agent receives unrestricted filesystem access.

84.8 Path and file safety

Vault operations resolve paths against an allow-list; `../` and unsafe symlinks deny. Remote peers never get arbitrary filesystem paths—only explicit vault intents. Validate local paths when indexing new Library folders.

84.9 SSRF protections

MCP and bridge URL validation restricts unsafe remote destinations and plain HTTP defaults, reducing the chance that an integration can probe internal services.

84.10 External-agent isolation

Bridge and MCP adapters expose curated tools, not shell or libp2p. Tokens authenticate bridge HTTP; mandates scope each tool call. Compromise of an external agent is contained by bond + mandate, not full node access.

84.11 Relay trust boundary

A relay is a connectivity service, not a trusted brain. End-to-end signatures and home-node policy remain necessary even when the relay operator is reputable.

84.12 Defense in depth

Security stacks Diplomat → inbound guard → bond engine → task guard → semantic firewall → vault path checks. No single layer is sufficient—connectivity success does not imply authorization. Design integrations assuming deny-by-default at each hop.

85. Privacy Controls

85.1 Profile visibility

Choose which profile fields each trust tier may fetch—public bios vs friends-only photos. Publishing overly open defaults exposes you on discovery topics. Revisit after changing trust relationships.

85.2 Contact disclosure

Contact cards show only what bond policy and your disclosure settings permit. Introduction flows reveal minimal proof text until upgrade. Do not embed third-party phone numbers in signed proof unless intended.

85.3 Knowledge sensitivity

Index and query knowledge with sensitivity tags; referred and public tiers cannot read private-indexed chunks. Re-tag before sharing summaries in Team jobs. Mis-tagged content is a privacy bug, not a crypto failure.

85.4 Conversation retention

Conversation history stays on your home node unless you export it. Retention controls (where offered) prune local indexes, not remote peer copies of messages they already accepted. Align retention with backup policy (Chapter 89).

85.5 Agent memory

Agent memory and session context live in home-node stores governed by mandate and settings. Clearing agent memory does not delete peer chat logs. Scoped mandates limit how much history tools may retrieve.

85.6 Vault sharing

Vault sharing uses explicit intents with sensitivity and path safety—no hidden folder sync to strangers. Team jobs pull only mandated vault slices. Audit vault retrieve denials when agents “cannot find” a file.

85.7 Model-provider privacy

Cloud model providers receive prompts you send through the configured adapter—review their terms and prefer local models for sensitive topics. Semantic firewall reduces exfiltration patterns but is not a full DLP suite. Disable cloud routing for classified workflows.

85.8 Relay privacy

Relays see connection metadata and encrypted/signed frames they forward—they are not trusted readers of plaintext chat. Avoid putting secrets in relay-visible routing hints. Choose relays you tolerate for availability, not for confidentiality.

85.9 Audit-log privacy

Audit JSONL stores structured summaries, not full message bodies by default. Protect audit files like keys—`0o600` and encrypted backup. Redact tokens before sharing logs externally.

85.10 Delete local data

Local delete removes threads, vault objects, or profile fields from **your** node; peers may retain copies. Use block and revoke for ongoing abuse. Secure-delete media if OS support exists before decommissioning hardware.

86. Audit and Activity History

86.1 Why EnvoyMesh records activity

Audit records make policy and automation reviewable. EnvoyMesh records structured summaries and correlation identifiers rather than treating agent activity as an opaque model transcript.

86.2 Audit-event fields

Audit events carry `eventId`, `createdAt`, `type`, `intent`, `outcome`, `summary`, optional `remotePeerId`, `correlationId`, and `latencyMs`. Learn the field glossary in Settings → Activity help. Summaries are human-readable; correlate IDs for multi-hop traces.

86.3 Correlation across peers

Use shared `correlationId` values to follow one user action across bond, relay forward, and tool calls on both sides. CLI `audit --include-p2p-trace` expands traces when debugging WAN paths. Ask contacts for their side’s ID only over a verified channel.

86.4 Policy allow and deny records

Allow and deny rows prove policy decisions with intent names and reasons—essential for “why was this blocked?” disputes. Approvals appear as separate outcomes. Export filtered audit slices for support, redacted.

86.5 Task and Team job records

Team job lifecycle events append to task journal and audit with state transitions (`discovering`, `running`, `completed`, …). Correlate job ID with chat threads that spawned the work. Failed jobs retain error summaries without raw model transcripts.

86.6 Tool and approval records

Tool invocations and human approval queue entries log mandate action, tool name, and outcome. Denied tools name missing capabilities. Use these rows to tune mandates without disabling audit.

86.7 External-agent records

Bridge and MCP traffic tags external-agent identity separately from native mesh peers. Cross-check bearer auth failures vs bond denials. External compromise investigations start in these rows.

86.8 Network diagnostics

Network diagnostic audit entries record relay reservation, dial failures, and connectivity snapshots—no message plaintext. Pair with `connectivity-status` CLI during outages. Do not enable verbose libp2p logging routinely in production.

86.9 Inspect an end-to-end flow

Pick one failed message or job, note its correlation ID, and walk audit chronologically on home and peer if available. Identify whether failure was guard, bond, transport, or model. Stop after the first definitive deny reason—avoid random setting changes.

86.10 Retention, backup, and protection

Audit logs grow unbounded without operator rotation—archive JSONL to encrypted backup media. Include audit in disaster-recovery drills. Restrict read access to owner-trusted devices.

87. Respond to Security Incidents

87.1 Lost device

From a trusted device, revoke the lost device, rotate any bridge or relay tokens it held, and review recent activity. If the lost device held the only owner-key backup, recovery may require creating a new identity.

87.2 Compromised owner key

Treat an exposed owner private key as a root compromise. Disconnect affected nodes, preserve evidence, rotate dependent credentials, notify trusted contacts, and migrate to a new owner identity because signatures from the old key can no longer be trusted.

87.3 Suspicious contact

Lower trust to public or block, preserve audit and recent threads, and verify identity out of band before restoring direct tier. Do not execute file opens or tool approvals from suspicious threads. Report coordinated harassment via block and documented audit export.

87.4 Misbehaving agent

Pause or revoke the agent mandate, disable bridge tokens, and inspect tool audit for unexpected vault or mesh calls. Narrow `allowedActions` before re-enabling. Treat repeated mandate violations as potential prompt injection or compromised integration.

87.5 Compromised external agent

Rotate bridge bearer tokens, disable the external agent's MCP registration, and review all tool calls since last known good. External agents never had libp2p—containment is token + mandate scope. Re-enable only with fresh secrets and tighter mandates.

87.6 Malicious file or knowledge content

Do not open unknown attachments; quarantine downloads outside default Vault open paths. Re-index knowledge sources if malicious content was ingested. Warn contacts if your node forwarded malware-signed-as-you due to key compromise.

87.7 Relay incident

If a relay operator reports abuse or outage, rotate home tunnel tokens and verify Agent Card URLs still point to your node. Relays cannot decrypt chat but can disrupt availability—have a secondary bootstrap relay in config. Document incident time window for audit review.

87.8 Revoke, block, and pause

Use block tier for contacts, revoke for devices and agents, and pause Team jobs from Agent Network UI. Order: stop ongoing harm (revoke/block), then investigate audit, then restore with tighter policy. Pausing is reversible; blocked contacts need deliberate unblock.

87.9 Preserve diagnostics

Copy relevant audit JSONL segments and correlation IDs before clearing logs or reinstalling. Remove bearer tokens, private keys, and recovery phrases from shared bundles. Store evidence encrypted with incident date in filename.

87.10 Report a vulnerability

Report security defects through the project's coordinated disclosure channel listed in release notes or repository SECURITY policy. Include reproduction steps and version—not live keys. Do not test exploits against production peers without permission.

Part XIII — Manage Devices and Data

88. Device Management

88.1 View devices

Open Settings → Devices to list owner-authorized machines and EnvoyGo pairings with creation dates and last activity hints. Each entry maps to a device certificate, not the owner root key. Use this view before revoking stale hardware.

88.2 Add a desktop device

Install Social/Tauri on the new computer, restore or create device identity from owner authorization flow, and approve the new certificate from an existing trusted device. Copy profile data via backup restore (Chapter 89) rather than hand-copying key files over chat.

88.3 Pair EnvoyGo

On desktop Social open Pairing → show QR; in EnvoyGo tap Pair and scan `envoy://pair?…`. Approve the pending device on home if the queue prompts. Confirm chat loads through HomeRemote before retiring an old phone pairing.

88.4 Understand separate device identities

Devices can share one owner identity while retaining independent device keys and certificates. This supports targeted revocation and audit attribution.

88.5 Review device activity

Filter audit and device list by device ID to see which machine sent messages or invoked tools. Unexpected device IDs after travel warrant revoke. EnvoyGo actions appear attributed to the paired phone device, not desktop.

88.6 Revoke a device

Select the device → Revoke certificate; the node rejects new sessions immediately. Rotate bridge or relay tokens that device held. Physical access after revoke still reads old local caches—encrypt disk on shared PCs.

88.7 Move to a new computer

Take a full profile backup, install on the new host, restore keys and Vault, then revoke certificates for the old PC if retiring it. Verify mesh listen addresses and update contacts if your public multiaddr changed. Send a test DM before decommissioning.

88.8 Replace a lost phone

Revoke lost EnvoyGo pairing from desktop first, then pair a replacement phone with a fresh QR. Assume the lost phone's pairing token is compromised if unlocked. Do not clone pairing files between phones manually.

88.9 Device synchronization boundaries

EnvoyGo syncs selected NodeService views—not a full mesh replica. Desktop and mobile may show different Settings depth. Conversation state authoritative on home; mobile cache clears on re-pair.

89. Back Up and Restore

89.1 Backup strategy

Use a layered backup: protect owner and device credentials separately from replaceable application binaries, and back up configuration, trust, Vault content, and important records on a tested schedule.

89.2 Identity keys

Export owner and device private keys only into encrypted backup archives; never store plaintext keys in cloud sync folders. Test import on an isolated machine yearly. Loss of owner key without backup is identity loss.

89.3 Configuration

Back up `node-config`, relay tokens, model provider settings, and bridge secrets with secrets redacted in secondary copies. Version-control non-secret config templates separately. Restore config before starting node after OS reinstall.

89.4 Contacts and trust

Include `trust-records.json` and peer directory in backup—losing trust store turns friends into strangers locally. Export before major migrations. Restored trust must match still-valid remote keys.

89.5 Conversations and sessions

Conversation indexes and session stores live in profile JSON/JSONL; back them with the home profile. Mobile holds minimal cache—re-pair refreshes from home. Large media may live in Vault paths included in 89.6.

89.6 Vault and Library

Vault and Library files need filesystem-level backup alongside indexes. Chunk stores and search indexes rebuild slowly—prefer consistent snapshot while node stopped. Verify random file hash after restore.

89.7 Audit and task history

Archive `audit-events.jsonl`, `task-journal.jsonl`, and approval queues for compliance. Rotation policies prevent unbounded disk use. Restored audit on new hardware preserves historical correlation IDs.

89.8 Restore and verify

Restore to a clean install, import keys and data, start node offline to verify, then enable network and send test message. Compare owner DID and device list with pre-disaster records. Revoke devices that should not return post-restore.

89.9 Disaster-recovery checklist

Maintain a printed or offline checklist: owner key backup location, relay bootstrap, trusted contacts to notify, revoke order for devices, and last verified restore date. Run tabletop exercise annually. Store checklist without live secrets.

90. Updates and Migration

90.1 Check the installed version

Check **About** in Social/Tauri or `envoy --version` on CLI against release notes before upgrading. Note relay and mobile app versions separately—mixed versions cause handshake surprises. Record build hash when reporting bugs.

90.2 Update the desktop application

Quit the app cleanly, run the installer or bundle update, relaunch, and confirm identity loaded in status. Desktop updates replace binaries only—profile directory persists. Roll back binary if startup fails, not by deleting profile.

90.3 Update EnvoyGo

Update EnvoyGo from the app store or sideload channel your fleet uses; re-pair if release notes require new pairing schema. Test home connection and one voice call after update. Keep desktop home node on compatible API version.

90.4 Update OpenClaw extensions

Update OpenClaw/HomeClaw extensions per bundled compatibility matrix in release notes. Restart bridge after extension update. Mismatch shows as gateway errors in agent status, not mesh failures.

90.5 Update a relay

Upgrade relay binaries with `--advertise-addr` preserved; restart during low traffic if operator. Community relay users depend on operator schedule—private relays you control should follow same version as nodes. Verify reservation after upgrade.

90.6 Configuration compatibility

Read migration notes for renamed config keys or JSONL schema bumps. Automatic migrations run at startup; failed migration backs up `.bak` files beside originals. Do not hand-edit migrated files while node is running.

90.7 Data migrations

Large data migrations may re-index Vault or rebuild trust views—allow time on first boot after upgrade. Monitor audit for migration summary events. Keep pre-migration backup until indexes stabilize.

90.8 Roll back safely

To roll back, install previous binary version and restore profile backup if new version wrote incompatible data. Revoke tokens issued only on new build if security fix motivated rollback. Never roll back owner keys—only application bits.

90.9 Review release notes

Read release notes for security fixes, breaking protocol changes, and experimental flags before clicking update. Appendix J lists maturity labels for planned features. Schedule upgrades after backup, not before travel.

Part XIV — Help and Troubleshooting

91. Troubleshooting Basics

91.1 Check node status

Start with the application status surfaces: confirm the node service is running, identity loaded, model/agent state is expected, and at least one network path is available.

91.2 Restart safely

Quit Social or the Tauri wrapper cleanly so the node can flush JSONL appenders. Restart the node process (or relaunch the desktop app) and wait until status shows identity loaded and mesh listening. If the profile was mid-write, check for `.tmp` files beside `trust-records.json` before deleting anything.

91.3 Check connectivity

Run `connectivity-status --rich` from the CLI and confirm at least one of: mDNS peers, bootstrap dial, or relay reservation. Compare your bootstrap multiaddrs with the contact's advertised addresses. If direct dial fails but relay works, treat it as NAT/firewall—not a bond or identity problem.

91.4 Check agent status

Open Settings → AI and confirm the agent mandate is present and not expired. For EnvoyAI, verify OpenClaw Gateway responds on its configured port (default 18789). External agents should show bridge health on port 3031; audit rows tagged `bridge` explain auth or timeout failures.

91.5 Review recent activity

Open Activity or run `audit --limit 40 --include-p2p-trace` and sort by time around the failure. Follow `correlationId` across rows—bond deny, guard reject, and relay forward each produce distinct summaries. Note the intent name (`chat.message`, `knowledge.query`, etc.) before changing trust or network settings.

91.6 Find logs

Operational history lives in your profile directory as JSONL: `audit-events.jsonl`, `task-journal.jsonl`, `approval-queue.jsonl`, and `discovery-events.jsonl`. Relay operators also get relay-manager snapshots in relay profile audit logs. Console output from `npm run node:dev` supplements but does not replace these files.

91.7 Collect a diagnostic report

A useful diagnostic bundle includes version, platform, relevant configuration with secrets removed, recent logs, audit correlation IDs, peer/relay status, and exact reproduction steps.

91.8 Remove private data before sharing diagnostics

Before sharing logs, copy only the relevant time window and redact `owner-key*`, device keys, `bridge-config.json` bearer tokens, model API keys, and raw envelope payloads. Replace peer display names with labels if needed; keep correlation IDs intact so support can trace flows.

91.9 Ask the community for help

Gather version, platform, profile path, reproduction steps, and redacted audit excerpts with correlation IDs. State which feature status label applies (Available, Beta, Experimental). Community channels are announced in release notes for 0.2.2—avoid posting secrets in public threads.

92. Installation and Startup Problems

92.1 Installer will not run

Confirm the download matches your CPU architecture and macOS/Windows version in release notes. On macOS, if Gatekeeper blocks the DMG, use System Settings → Privacy & Security → Open Anyway once. On Windows, unblock the installer file property if SmartScreen quarantined it.

92.2 Operating system blocks the app

macOS: approve the app under Privacy & Security after first launch; notarized builds should not require disabling SIP. Windows: allow the app through Defender/Firewall when prompted for inbound mesh traffic. Corporate MDM may block unsigned or unknown publishers—request an exception or install from source with your own signing.

92.3 Application does not start

Launch from terminal with logging enabled (`npm run node:dev -- --profile <path>`) to capture startup exceptions. Verify the profile directory is writable and not on a sync folder that locks files (iCloud, OneDrive). A corrupt `trust-records.json` or missing owner key prevents UI load—restore from backup rather than deleting the profile.

92.4 Node runtime does not start

Check Node.js version against `package.json` engines and rerun `npm install` from the repo root for source installs. Packaged desktop builds embed the runtime—reinstall if the bundled binary was quarantined. Look for port conflicts on WebSocket/API ports configured in node settings.

92.5 OpenClaw runtime is unavailable

Confirm OpenClaw Gateway is installed and listening (default 18789). Run `./scripts/setup.sh` or `.\scripts\setup.ps1` after upgrades to refresh extensions. Windows slim bundles may omit optional extensions—compare with macOS bundle list in release notes.

92.6 Required extension is missing

List enabled OpenClaw extensions in Gateway settings and compare with the platform bundle in Chapter 9. Re-run setup scripts to copy missing extensions into the expected paths. Mesh chat and bonds do not require optional channel extensions—only enable what your agent workflow needs.

92.7 Firewall or antivirus warning

Allow outbound TCP/QUIC to bootstrap peers and relays; inbound direct dial may need a firewall rule on the home node. Antivirus hooks on `%AppData%` or `~/.local/share/envoymesh` can block JSONL writes—add an exclusion for the profile path. Document which ports you opened before retrying WAN discovery.

92.8 Update fails

Ensure the updater can write beside the install directory and profile path. Back up the profile and vault before major upgrades. If auto-update fails, download the new installer manually and install over the existing app without deleting user data.

92.9 Reinstall without losing data

Uninstall or replace the application bundle only—never delete the profile directory or `shared_vault/`. Note your absolute profile path from Settings or Appendix K before reinstalling. After reinstall, point the app at the same `--profile` path or restore from your encrypted backup.

93. Identity and Pairing Problems

93.1 Identity creation fails

Ensure the profile directory is empty or use a new `--profile` path for a fresh owner. Disk full or permission denied on key write shows in console as ENOENT/EACCES—fix filesystem access first. Do not run two nodes against the same profile simultaneously.

93.2 QR code cannot be scanned

Increase screen brightness and disable camera macro blur; QR must include the full `envoy://pair?` payload. Regenerate the invitation if it expired—tokens are time-bound. For LAN onboarding, confirm both devices share the same network segment without client isolation.

93.3 Invitation is invalid

Compare the scanned URI with what the sender displayed—truncated copies break signature verification. Check clock skew; some invitation formats embed expiry timestamps. Ask the sender to regenerate from Contacts → Invite rather than forwarding a screenshot.

93.4 Identity verification fails

Verify the sender's public key hashes to the claimed peer ID and the envelope signature verifies. If verification fails after a key rotation, ensure revocation records propagated and both sides refreshed trust. Audit rows `malformed or unsigned envelope` indicate transport corruption or version skew, not necessarily malice.

93.5 Bond request is missing

Bond requests require the recipient to be online or reachable via relay for `bond.request` delivery. Public-tier peers receive a challenge flow—not an automatic bond; complete referral or manual approval. Check Activity on both sides for `bond.request` / `bond.challenge` intents.

93.6 LAN onboarding fails

Confirm mDNS is not blocked by guest Wi-Fi or VPN split tunneling. Print multiaddrs from the host node and dial manually if discovery fails. Firewall on the host must allow inbound mesh ports for LAN onboarding handoff.

93.7 EnvoyGo pairing fails

EnvoyGo must scan a home-node QR while the desktop node is running and WebSocket-reachable. Off-LAN pairing needs relay/circuit path to home—verify home tunnel and pairing token in Settings → Devices. Revoke stale device certificates if an old phone retains a broken session.

93.8 Recover missing identity data

Restore `owner-key.pem` and device keys from your encrypted backup into the original profile path. Never invent new keys for the same owner ID—peers will reject mismatched signatures. If only vault data is missing, re-index from backup; identity loss without backup cannot be cryptographically recovered.

94. Messaging, Files, and Calls

94.1 Contact appears offline

Offline usually means no active libp2p connection—not necessarily a blocked bond. Run connectivity checks and confirm the contact's node is running. Relay-assisted paths may lag behind direct; wait one heartbeat interval before assuming permanent offline.

94.2 Message is not delivered

Confirm bond tier allows `chat.message` (direct or referred with approval). Inspect audit for deny vs guard reject vs relay forward failure. Large payloads may hit envelope size caps—try a smaller message or file chunk path.

94.3 Group message is missing

Verify all members share the same room ID and room sync completed (`chat.room.sync`). A member on an old build may not decode new room envelope versions—align versions. Check whether the missing message was sent while you were offline; request room sync from the host peer.

94.4 File transfer fails

Raw file sharing may require owner approval when `allowRawFiles` triggers bond policy. Confirm vault path safety and size limits on both nodes. If transfer stalls mid-stream, inspect relay circuit stability—resume after reconnect rather than duplicating sends.

94.5 Audio message will not play

Confirm the audio codec and container match what Social/EnvoyGo expects for the release. Download completed before play—partial files fail decode silently in some clients. Check bond policy did not strip attachments from the chat envelope.

94.6 Voice call cannot connect

Voice calls need working peer connectivity plus TURN/STUN when NAT blocks direct media. Verify TURN credentials in Settings and that UDP is not blocked on restrictive networks. Both parties must be on builds that support voice signaling for the current protocol version.

94.7 Background call notification is missing

On mobile, confirm notification permissions and that EnvoyGo background refresh is enabled. iOS Focus modes and Android battery savers can delay push until the app foregrounds. Incoming call signaling still requires home node reachability—check relay path if away from LAN.

94.8 TURN configuration problems

Validate TURN server URL, username, and credential expiry—stale credentials produce ICE failed states. Test with a known-good public TURN service before blaming mesh signaling. Document whether the failure is gather timeout (firewall) or relay allocate reject (bad creds).

94.9 Duplicate or delayed events

Duplicate messages often indicate reconnect replay—check inbound guard dedup and whether two devices share one identity. Delayed events on relay paths are normal under load; compare timestamps in audit, not UI order alone. If duplicates persist, ensure only one active session per device certificate.

95. Agent, Model, and Tool Problems

95.1 EnvoyAI does not respond

Verify OpenClaw Gateway is up and the bridge on 3031 accepts the configured bearer token. Check model router configuration and semantic firewall rejections in audit. An expired agent mandate stops all agent-role intents until renewed on desktop.

95.2 Model provider fails

Confirm API keys and base URLs for the selected provider in model settings. LiteLLM or adapter errors surface in audit with provider name and HTTP status. Try a local model or different provider to isolate network vs quota failures.

95.3 Tool is missing

Open the tool registry on the home node and confirm the tool is registered for your agent mandate. MCP-imported tools require an active MCP client session; mesh-native tools need matching capabilities on the agent card. Restart the agent runtime after adding tools so the registry reloads.

95.4 Tool call is denied

Tool denial usually means mandate `allowedActions`, bond tier, or missing capability (`vault.retrieve`, `task.execute`, etc.). Read the audit deny reason verbatim—it distinguishes bond deny from mandate approval_required. Raise trust or approve the pending action on desktop rather than retrying blindly.

95.5 Approval is pending

Open Approvals on desktop and resolve items matching the task's correlation ID. Approvals expire with mandates—check `expiresAt` if the queue looks empty but tasks stay waiting. External agents cannot approve on your behalf; only the owner device can clear owner prompts.

95.6 Trigger does not run

Verify trigger schedules, cron expressions, and that the node was running at fire time. Triggers respect mandate bounds—disallowed actions fail silently in audit, not always in UI toasts. Check whether experimental toggles gate the trigger feature for your build.

95.7 Digest is missing

Digests batch activity on a schedule—confirm digest generation is enabled and the agent had events in the window. Empty digests may mean no qualifying audit rows or semantic firewall dropped the summary prompt. Inspect `task-journal.jsonl` for failed digest tasks.

95.8 Memory or session result is unexpected

Session memory is local to the agent runtime—clearing bridge cache or rotating sessions resets context. Compare what the model returned with vault retrieval citations; RAG may pull unexpected public items. If memory looks like another user's data, stop and verify profile path—never share one profile between owners.

95.9 External agent does not reply

Ping the external agent's message port (HomeClaw 8010, Hermes 8020, OpenHuman 8021) from the home node host. Confirm bridge bearer token matches on both sides and the agent process logs show inbound `envoymesh-message` traffic. Compatibility presets do not guarantee the external runtime is running—start it separately.

96. Knowledge and Browser Problems

96.1 File cannot be added

Vault enforces path safety—illegal paths or symlinks outside allowed roots reject adds. Check disk space and file permissions under `shared_vault/`. Very large files may need chunking settings; watch audit for size cap denials.

96.2 Search returns no result

Confirm the item was indexed: run library refresh and check sensitivity labels match your query scope. Local search only covers your vault; remote queries need bond-permitted `knowledge.query`. Rebuild the index if `shared_vault` was restored from backup without re-indexing.

96.3 Remote knowledge is denied

Remote deny usually means bond tier caps sensitivity—public peers only see `public` items; referred caps at `public` for knowledge.query; direct caps at `friends`. Public knowledge queries are rate-limited (default 5/min)—wait for window reset. Inspect audit for `requested sensitivity exceeds` vs `peer is blocked`.

96.4 Shared content cannot be opened

Browser and Library require `library.read` permission for the reader's bond tier and item visibility. Confirm content hash matches if integrity check failed—do not open mismatched blobs. EnvoyGo mirrors Browser through home—home node must be online.

96.5 Content hash does not match

Re-download or re-export the item; hash mismatch means bytes changed in transit or on disk. Compare sha256 from publisher metadata with local file. If IPFS pin is stale, fetch from the original publisher rather than a cached gateway.

96.6 IPFS export fails

IPFS export is optional—confirm Helia/Kubo sidecar is running if your build includes it. Check sidecar logs for connect errors to the IPFS daemon. Omit IPFS entirely if you only need local vault storage—export is not required for mesh sharing.

96.7 `envoy://` page does not load

`envoy://` pages resolve through home Browser routing—verify the URI scheme handler and that the item is published. Off-LAN access needs home reachability via EnvoyGo or desktop with relay path. Broken hashes or missing vault paths show as blank pages with errors in home audit.

96.8 Feed update is missing

Feed notify requires referred+ bond for inbound notifications; publisher must have sent `feed.notify`. Check bond tier and that feed subscription is enabled on the reader node. Metadata-only notify does not push full content—open Library/Browser to fetch the item.

96.9 Restore damaged content

Restore files from backup into the same vault path layout; run re-index afterward. Do not hand-edit chunk manifests unless you understand vault chunking—prefer republishing from source. If corruption is widespread, isolate the profile and scan disk health before continuing.

97. Network and Relay Problems

97.1 Direct connection fails

Collect dial hints from both peers and attempt manual dial from CLI if UI shows disconnected. Symmetric NAT often blocks direct TCP—configure relay bootstrap and verify circuit reservation succeeds. Compare libp2p versions if identify handshake fails immediately.

97.2 Local discovery fails

mDNS requires multicast on the LAN—guest networks and VPNs frequently block it. Use printed multiaddrs for lab setups. Confirm both nodes advertise the same discovery profile (e.g. local vs wan-default).

97.3 Relay lookup fails

Verify bootstrap multiaddr includes `/p2p/<relay-id>` and relay HTTP check-in succeeds. Run `relay-status` and inspect audit for `relay.lookup` failures. Override with a private relay if community relay is down—do not disable bootstrap entirely.

97.4 Community relay is unavailable

Community relay at the default bootstrap may be busy or undergoing deploy—retry with backoff. For production, run a private relay with `--advertise-addr` in public mode. Circuit reservation failure often means version skew on the relay host, not client misconfig.

97.5 Multiple relays disagree

Nodes may check in to different relays with divergent routing hints—standardize bootstrap lists across your fleet. Compare relay-manager snapshots in audit for conflicting parent/child records. Prefer one organization relay as primary bootstrap to reduce split views.

97.6 Firewall or NAT restriction

Map required outbound ports for bootstrap and relay TCP. Inbound direct dial needs port forwarding or UPnP where supported; otherwise rely on circuit relay. Document corporate proxy rules—libp2p does not traverse HTTP proxies without explicit tunnel setup.

97.7 Peer remains offline

Peer offline on your UI may still be online to others—verify from a third mutual contact if possible. Check last-seen in peer directory and recent `system.ping` results. Long offline periods may mean sleep, profile migration, or revoked device cert.

97.8 Agent card cannot be fetched

Agent cards fetch over bonded paths—public bonds do not auto-fetch worker cards. Force refresh from Agent Network settings after bond upgrade. Audit `agent.card.request` / `agent.card.response` for deny or timeout; stale cards hide capabilities.

97.9 Collect network diagnostics

Bundle: app version, OS, profile path, bootstrap multiaddrs, `connectivity-status --rich` output, redacted `audit-events.jsonl` with correlation IDs, and relay reservation result. Include both peers' perspectives for connection issues. See Appendix K.5 for CLI commands.

98. Integration Problems

98.1 OpenClaw extension is missing

Compare installed OpenClaw extensions with Chapter 9 platform bundle lists. Re-run setup script after clone or upgrade. Windows essential set is slimmer—install missing extensions manually or switch to source/macOS bundle.

98.2 HomeClaw cannot connect

Default HomeClaw message port is 8010—confirm process is listening on the home node host. Bridge bearer token in `bridge-config.json` must match HomeClaw's expected secret. HomeClaw runtime is externally maintained; verify its logs independently from EnvoyMesh audit.

98.3 Hermes cannot connect

Hermes defaults to port 8020—test with curl or netcat from localhost on the home machine. Apply the Hermes compatibility preset then restart both bridge and Hermes after config edits. Check firewall loopback rules if bridge is containerized.

98.4 OpenHuman cannot connect

OpenHuman listens on 8021 by default in compatibility presets. Confirm OpenHuman's envoymesh adapter is enabled and using the same bridge URL as desktop Settings. Treat agent-side errors as external-runtime issues once bridge auth succeeds.

98.5 Bridge authentication fails

A 401 response usually means the Bearer token is missing or mismatched. Confirm both sides use the same secret, the header uses `Bearer`, and the URL points at the correct bridge rather than the OpenClaw gateway.

98.6 External tool call fails

Inspect bridge logs for tool name, mandate action, and bond decision on the failing call. External tools map to mesh capabilities—missing `vault.retrieve` denies knowledge tools. Retry with a minimal tool invocation to isolate schema vs policy failures.

98.7 MCP client cannot connect

MCP clients connect to the home MCP adapter—confirm port, bearer token, and that the node exposes MCP when bridge is enabled. stdio MCP servers need correct command paths in registry config. Client and server must agree on protocol version supported by your build.

98.8 MCP server is rejected

Rejected MCP servers usually fail capability or auth checks at registration time. Verify server manifest tools do not require disallowed actions for the active mandate. Check audit for `missing capability for` messages naming the intent.

98.9 A2A Agent Card is unavailable

Fetch `/.well-known/agent-card.json` from the home or relay public base URL with a valid bearer when required. Relay forwarding needs an active home tunnel for the token owner. Card JSON must be signed and fresh—republish after capability changes.

98.10 A2A task fails or is not found

Locate the task id from `tasks/send` response and poll `tasks/get` with the same bearer. Map internal states via Chapter 73— `auth-required` means bond/mandate denial, not transport failure. Cancel only tasks owned by the bearer-mapped owner; unknown id means expired or never created on this node.

99. Frequently Asked Questions

99.1 Does EnvoyMesh require an account?

No central EnvoyMesh account is required. You create local cryptographic identities and may optionally use third-party model providers, relays, mobile push services, or integrations that have their own accounts.

99.2 Where is my data stored?

Everything lives on **your devices**, primarily the home node profile directory: Vault files, trust store, conversation indexes, audit JSONL, and identity keys. EnvoyGo keeps pairing tokens and cached UI state on the phone—not a second copy of the full Vault unless a feature explicitly caches media. Relays forward traffic; they are not your data store.

99.3 Can a relay read my messages?

Relays can observe connection metadata and forward encrypted/signed application traffic, but they are not authorized to impersonate a sender or bypass home policy. Avoid placing unnecessary sensitive data in routable metadata.

99.4 Can I use EnvoyMesh without a relay?

Yes, on the same LAN with mDNS or direct multiaddrs, or over known peer routes without relay reservation. Many WAN setups still use a **relay** for discovery and circuit relay when NAT blocks direct dial. Relay assists connectivity; it does not replace home-node policy or signing.

99.5 Can I use more than one device?

Yes. One **owner identity** can authorize multiple **device certificates**—desktop Social/Tauri, additional computers, and EnvoyGo via QR pairing. Each device has its own key for audit and revocation. Mobile is a thin client to home; it does not duplicate the full mesh node.

99.6 Can I use my own model?

Yes. Configure providers in Settings → AI → Model (LiteLLM-compatible endpoints, local runners, or cloud APIs you trust). The semantic firewall still filters prompts; bonds and mandates still gate tool use. Provider traffic is subject to that provider's privacy terms.

99.7 Can I use an external agent?

Yes, through the **Ext Agent bridge** (HomeClaw, Hermes, or OpenHuman) and MCP adapters on the home node. External agents call mesh tools (`mesh.findKnowledge` , etc.)—they do not receive raw libp2p sockets. Enable bridge auth, scope mandates, and review audit for external tool calls.

99.8 What happens when a contact is offline?

Signed messages queue on the sender's home node and retry when a path opens—direct LAN, relay circuit, or later online presence. Delivery indicators may lag until the remote node acknowledges. Neither side loses message integrity; duplicates are avoided by protocol IDs where implemented.

99.9 Can strangers recruit my agent?

No. Team jobs require bonded contacts and opted-in capability providers. Public strangers are not recruitable workers in the current product.

99.10 Can I revoke an agent or device?

Yes. **Revoke device certificates** for lost laptops or phones from a trusted desktop node; revoke or narrow **agent mandates** to stop automation. Blocked trust stops new contact operations. Revocation is local and signed—peers learn on next verified interaction.

99.11 Is EnvoyMesh a replacement for MCP or A2A?

No. EnvoyMesh uses its own signed native protocol and provides MCP and A2A bridges so other ecosystems can use selected tools, discovery, and tasks.

99.12 Which features are experimental or planned?

See Appendix J for the authoritative list. In short: Beta/Experimental items are implemented but still being validated (interfaces may change); Planned items are designed but not shipped as complete features (notably video calling and broad anonymous worker discovery); Parked items are intentionally deferred without a committed date (EnvoyGo full-node mode, global reputation, multi-hop commerce); Deferred items are designed but not yet built (Filecoin persistence, full hierarchical relay graph); Future items are scoped for later interop work (MCP resources/prompts, OAuth 2.1). Always confirm against the current release notes before relying on any non-Available capability.

Part XV — Website and Content System *(editors and operators)*

> Part XV is a website and editorial content map. End users can skip this part; use Parts I–XIV and the appendices instead.

100. Website Information Architecture

100.1 Homepage

Lead with the one-sentence value proposition (private mesh for people and agents), the primary install CTA, and three feature pillars (private messaging, personal AI, Agent Network). Link to Use Cases and Downloads; avoid protocol jargon above the fold.

100.2 Product overview

One paragraph per pillar linking to the dedicated product page. Attach the Available/Desktop/Mobile labels and link each pillar to its in-guide chapter (messaging → Part III, personal AI → Part IV, knowledge → Part V, external agents → Part VI, Agent Network → Part VII).

100.3 Agent Network

Frame as bonded opt-in collaboration — never “marketplace”. Attach the Agent Network overview chapter link (§44) and the Join flow (§45). Call out that strangers cannot recruit the local agent.

100.4 External Agents

List OpenClaw (bundled), HomeClaw, Hermes, OpenHuman with Compatibility-preset labels where applicable; link each to its guide chapter (§38–§42). State that only one external agent is active per bridge.

100.5 Use cases

Curate 6–8 scenarios (personal AI across devices, family mesh, trusted research, small-team Agent Network, Claude Desktop via MCP, A2A delegation, self-hosted relay). Each links to the matching tutorial in §14 or use case in §5.

100.6 How it works

Plain-language architecture diagram (owners → bonds → signed messages → optional relay). Link to §4 and the security model page; keep Ed25519/libp2p in an expandable technical note.

100.7 Security and privacy

Summarize the Diplomat/Bond Engine/semantic firewall/Vault boundaries without claiming “unbreakable” security. Link to §84 and Appendix H checklists; surface the vulnerability-reporting contact.

100.8 Downloads

Per-platform cards (macOS, Windows, iOS, Android, source) with Verified badges and last-verified dates. Link to §8 install steps; surface release notes and Appendix J status boundaries.

100.9 Guide

Entry point to this guidebook: Getting Started, Everyday Use, External Agents, Agent Network, Troubleshooting. Mirror the “Proposed Guide Navigation” tail of this document.

100.10 Community and support

GitHub, discussions, roadmap, release notes, support contact. Keep it actionable — where to file bugs, where to ask questions, where to read the roadmap.

101. Product Pages

101.1 Private messaging

Pitch signed peer-to-peer messaging with bond-gated delivery. Attach Available + Desktop + Mobile labels; link to §16 and §17. Note group chat and audio messages as related.

101.2 Personal AI

Pitch EnvoyAI/OpenClaw as the bundled assistant under owner policy. Attach Available + Desktop label; link to §21–§28. Cross-reference external agents for users who prefer a different runtime.

101.3 Knowledge Base

Pitch local-first notes, Vault files, RAG, and Obsidian integration. Attach Available + Desktop label; link to §29–§35. Note sensitivity labels and federated RAG as differentiators.

101.4 Agent Network and Team jobs

Pitch bonded multi-agent collaboration with attributed reports. Attach Available + Desktop label (EnvoyGo is read-only mirror); link to §44–§63. Emphasize “not a marketplace”.

101.5 External Agents

Pitch the safe bridge for OpenClaw/HomeClaw/Hermes/OpenHuman. Attach Compatibility-preset labels; link to §36–§43. State the one-active-bridge rule.

101.6 Desktop and EnvoyGo

Pitch two surfaces, one identity: desktop home node + EnvoyGo thin client. Attach Available + Desktop + Mobile labels; link to §8 and §13. Surface the macOS/Windows bundle difference (§9.4/§9.5).

101.7 Voice and file sharing

Pitch voice calls (Phase 42I on iOS) and content-addressed file sharing. Attach Available + Desktop + Mobile labels; link to §18 and §19. Mark video calls as Planned.

101.8 Terminals and Browser

Pitch remote terminals and `envoy://` browsing. Attach Available + Desktop label (EnvoyGo mirrors); link to §78 and §79. Surface herdr/TmuxAI as external integrations.

101.9 MCP and A2A

Pitch MCP tool bridging (consumer + server) and A2A agent cards/tasks. Attach Experimental/Beta labels as appropriate; link to §68–§73. Note OAuth/resources as future scope.

101.10 Relays and self-hosting

Pitch optional relays for connectivity and self-hosted fleet operations. Attach Operator label; link to §74–§77. Surface the community relay and the operator fleet guide.

102. External Agent Website Pages

102.1 External Agents overview

Explain the bridge model (no raw P2P for agents). Attach Compatibility-preset guidance; link to §36 and Appendix C matrix. Audience: integrators choosing an agent runtime.

102.2 OpenClaw / EnvoyAI

Detail the bundled runtime, gateway port 18789, canonical extension, macOS/Windows bundle differences. Attach Available + Desktop label; link to §38.

102.3 HomeClaw

Detail the default preset at 8010/message, externally maintained channel. Attach Compatibility-preset label; link to §39. State verification responsibility.

102.4 Hermes

Detail the preset at 8020/message, knowledge-oriented runtime, migration path. Attach Compatibility-preset label; link to §40.

102.5 OpenHuman

Detail the preset at 8021/message, disabled by default, externally maintained. Attach Compatibility-preset + Planned-for-production labels; link to §41.

102.6 Custom agent integrations

Document the `envoymesh-message` adapter contract. Attach Experimental label; link to §42 and the bridge wire contract in §37. Audience: developers.

102.7 Integration status matrix

Render Appendix C as a sortable table (agent × mode × port × status × last-verified). Keep it the single source of truth; every other page links here.

102.8 Security boundary

Explain why agents never hold Ed25519 keys and how Bearer auth gates `/bridge/*`. Link to §37 and §84.10; do not overstate — say “policy-checked”, not “secure”.

102.9 Developer handoff links

Cross-link to `docs/agent_bridge_guide.md`, `docs/openclaw-agent-bridge-adr.md`, `OpenClawExtension/`, and the MCP/A2A design docs. Audience: engineers implementing an agent.

103. Agent Network Website Pages

103.1 Agent Network overview

Define bonded opt-in collaboration; attach Available + Desktop label; link to §44. Emphasize “not a marketplace” and “relays stay lean”.

103.2 Join Agent Network

Step-by-step enable Join + publish profile; link to §45 and §46. Attach screenshots of the Settings → Agent Network tab.

103.3 Agent identity and cards

Explain owner-authorized agent credentials and Agent Cards; link to §47. Surface the A2A Agent Card bridge as the external face.

103.4 Bonded worker discovery

Explain card auto-fetch on bond + capability index; link to §48 and §49. Note that broad anonymous discovery is Planned, not current.

103.5 Team jobs

Define Team jobs (product name) vs chains (code name); link to §50–§58. Attach screenshots of the Chains/Team jobs UI.

103.6 Planning and assignment

Explain orchestrator plan + direct-assign vs competitive bidding; link to §51–§53. Keep LLM planner details in an expandable.

103.7 Bidding and budgets

Explain mandates, cost ceilings, rebalance policies; link to §53 and §54. Surface CSV export and cost visibility controls.

103.8 Multi-round collaboration

Explain iteration (draft → judge → replan); link to §56. State default `iterationMaxRounds=1`.

103.9 Results and provenance

Explain composite artifacts and worker attribution; link to §58 and Appendix G. Emphasize that flattened anonymous answers lose provenance.

103.10 Trust and safety

Summarize bond gates, mandate limits, sensitivity ceilings, approvals; link to §61 and Appendix H.5/H.6 checklists.

103.11 Network connectivity

Explain LAN, direct, relay-assisted paths; link to §62 and Part X. Surface NAT/TURN guidance.

103.12 Feature status and roadmap

Render Appendix J.4–J.11 as the authoritative boundary list; link each item to its design doc. Mark Planned/Parked/Deferred explicitly.

104. Reusable Content Template

104.1 Page title

Concise, action-oriented, ≤ 60 characters. Mirror the noun the user searches for (e.g. “Private messaging”, “Join Agent Network”), not internal jargon.

104.2 One-sentence summary

Lead with what the reader can do, not what the feature is. “Send signed peer-to-peer messages to bonded contacts” beats “A messaging subsystem using Ed25519 envelopes.”

104.3 Availability labels

Render the standard labels exactly: Available, Beta, Experimental, Compatibility preset, Planned, Parked, Desktop, Mobile, and Operator. A page may carry more than one label, such as Available + Desktop.

104.4 What the feature does

Two or three sentences max. Name the user action, the boundary (who/what is involved), and the result. Avoid protocol names unless the page is for developers.

104.5 Why someone would use it

Frame around a real goal (privacy, control, collaboration, cost). One sentence per audience if multiple — separate “for individuals”, “for teams”, “for operators” lines.

104.6 Before you begin

Bullet list of hard prerequisites: a running home node, a bonded contact, an enabled toggle, a configured model. Link each prerequisite to its setup chapter.

104.7 Step-by-step instructions

Numbered steps, one action per step, with the exact UI path (Settings → …) or command. Screenshots or short code blocks where the path is non-obvious. Keep each step independently checkable.

104.8 What happens behind the scenes

Optional expandable section for protocol/crypto detail. Use it to satisfy technical readers without forcing everyone through Ed25519/libp2p jargon. Link to the design doc, do not duplicate it.

104.9 Privacy and safety notes

State the boundary the feature enforces (signed, policy-gated, sensitivity-capped, approval-required) and what it does NOT protect against. Quote the security chapter rather than restating it.

104.10 Troubleshooting

Three to five symptom → cause → fix lines. Link to the matching §91–§98 chapter for deeper diagnosis. Avoid generic “restart the app” advice unless that is genuinely the fix.

104.11 Related topics

Three to five cross-links to adjacent chapters and the next logical action. Helps readers move from “set up” to “use” to “troubleshoot” without backtracking to the TOC.

104.12 Last verified version and date

Every page should record the last EnvoyMesh version and date against which its steps and status were checked. Re-verify after UI, protocol, packaging, or security changes.

105. Editorial and Terminology Guide

105.1 Write for end users first

Address the reader directly (“you”), lead with the task, defer protocol internals to expandable notes. Mirror the tone of §1–§14.

105.2 Progressive disclosure for technical details

Surface the user-facing concept first; link to the deeper guide chapter; reserve code identifiers, schemas, and config keys for the technical layer. Never force a reader to learn Ed25519 to send a message.

105.3 Product terms versus code names

Prefer current product terms such as Team jobs and EnvoyGo. Mention code names such as chains only when they help developers find logs, settings, or protocol references.

105.4 Feature-status language

Use exactly the nine canonical labels from §“Feature status labels” (Available, Beta, Experimental, Compatibility preset, Planned, Parked, Desktop, Mobile, Operator). Never mint new status words; if a capability does not fit, qualify with prose rather than a new label.

105.5 Platform labels

Pair every feature page with a platform label (Desktop, Mobile, Operator). If a feature is Desktop-only today but a Mobile mirror is planned, say “Desktop (Mobile mirror planned)” rather than leaving the platform ambiguous.

105.6 Security claims and evidence

Security statements must identify their boundary and evidence. Say “signed by the sender key and checked by the inbound guard,” not “completely secure.”

105.7 Integration maturity claims

Describe HomeClaw, Hermes, and OpenHuman as compatibility presets and state that their agent-side runtimes are externally maintained. Do not imply equal maturity with the bundled OpenClaw integration.

105.8 Accessibility and inclusive language

Use plain language, alt text for diagrams, sufficient color contrast, and avoid assumed-ability phrasing. Mirror WCAG-AA contrast in website pages.

105.9 Screenshots, diagrams, and alt text

Every screenshot needs alt text describing the action, not the chrome. Diagrams should be SVG with text labels; keep ASCII diagrams as a fallback in code blocks.

105.10 Translation and localization

Translate prose; keep brand names (EnvoyMesh, OpenClaw, etc.), code identifiers, and UI paths in English. Follow the Chinese edition glossary; coordinate locale updates with UI i18n.

105.11 Versioning and review cadence

Bump the guidebook version with each release; re-verify status labels against `docs/implementation-plan.md` and Appendix J. Record the last-verified date on every website page.

Appendices

Appendix A — Glossary

A.1 Agent

An **agent** is an AI identity authorized by an owner to communicate or perform bounded tasks.

A.2 Agent Card

An **Agent Card** is a signed capability description used to discover what an agent can do and whether it joined Agent Network.

A.3 Agent Network

Agent Network is bonded, opt-in collaboration among owners' local agents.

A.4 Artifact

An **artifact** is a typed task result: text, file, structured data, or a composite bundle.

A.5 Bond

A **bond** is the local trust relationship and tier assigned to another owner.

A.6 Capability

A **capability** is an advertised or authorized operation such as task execution or knowledge query.

A.7 Contact

A **contact** is a known owner or agent represented in the local directory and relationship UI.

A.8 Device

A **device** is an owner-authorized installation with its own key and certificate.

A.9 DID

A **DID** is a decentralized identifier presentation derived from cryptographic identity. EnvoyMesh owner/device/agent DIDs use the `envoy:owner:` / `envoy:device:` / `envoy:agent:` prefixes (see §10.6).

A.10 EnvoyAI

EnvoyAI is EnvoyMesh's bundled personal-agent experience powered by OpenClaw.

A.11 EnvoyGo

EnvoyGo is the current iOS/Android thin client paired to a home node.

A.12 External agent

An **external agent** is a separately maintained runtime connected through the local HTTP bridge.

A.13 Library

The **Library** organizes knowledge items for local search, sharing, publishing, and browsing.

A.14 Mandate

A **mandate** is an owner-signed authorization bounding an agent task.

A.15 Owner

The **owner** is the long-lived human identity and root authorization key.

A.16 Peer

A **peer** is a runtime network identity that signs and transports envelopes.

A.17 Relay

A **relay** assists reachability, lookup, and forwarding without becoming the application authority.

A.18 Task

A **task** is a signed lifecycle for delegated work and typed results.

A.19 Team job

A **Team job** coordinates several agent subtasks and merges their attributed results.

A.20 Vault

The **Vault** is path-safe local storage and indexing for private files and knowledge.

Appendix B — Feature and Platform Matrix

B.1 macOS

macOS — Tauri desktop bundle with embedded node; fuller OpenClaw extensions; DMG/notarized install. Home node runs all mesh features; EnvoyGo pairs as mirror. Profile under Tauri app data area (Appendix K.1).

B.2 Windows

Windows — Installer with slimmer OpenClaw essential set; profile in `%AppData%` / `%USERPROFILE%\.envoymesh\`. Allow firewall for inbound peers when prompted.

B.3 EnvoyGo on iOS

EnvoyGo iOS — Flutter thin client; QR pair to home; chat, calls, terminals, Browser mirror; no standalone mesh node.

B.4 EnvoyGo on Android

EnvoyGo Android — Same mirror scope as iOS; home node must stay reachable via WebSocket/relay when off-LAN.

B.5 Home-node-only features

Home-node-only — Identity, Vault, agents, Team orchestration, MCP/A2A bridges, full Settings. Required for authoritative signing and policy.

B.6 EnvoyGo mobile read-only mirrors

EnvoyGo mirrors — Read-heavy remote UI; AI engine and bridge config read-only on phone; change on desktop.

B.7 Operator features

Operator — Relay deployment, bootstrap lists, `--advertise-addr`, fleet manifest CLI; not end-user Social features.

B.8 Available, Beta, Experimental, Planned, and Parked features

Status labels — Available, Beta, Experimental, Planned, Parked, Deferred per front matter; Appendix J is canonical over marketing copy.

Appendix C — External Agent Matrix

C.1 EnvoyAI / OpenClaw

EnvoyAI / OpenClaw — Bundled personal agent; Gateway default 18789; bridge 3031; EnvoyMesh-maintained extensions on desktop.

C.2 HomeClaw

HomeClaw — Compatibility preset; external runtime; message port 8010; bearer auth via bridge-config.

C.3 Hermes

Hermes — Compatibility preset; external runtime; port 8020; verify adapter logs separately.

C.4 OpenHuman

OpenHuman — Compatibility preset; external runtime; port 8021; human-in-loop workflows external to mesh.

C.5 Custom `envoymesh-message` agents

Custom envoymesh-message — HTTP message adapter; you maintain agent process; match bridge token and JSON schema.

C.6 MCP-compatible applications

MCP applications — Clients attach to home MCP adapter; tools map to registry; Bearer auth; no OAuth resources yet (J.11).

C.7 A2A-compatible agents

A2A agents — Public card at `/.well-known/agent-card.json`; JSON-RPC tasks; relay home-tunnel forwarding when enabled.

C.8 Runtime ownership and verification status

Verification — Record last tested version/date per integration; compatibility preset ≠ equal maturity to EnvoyAI.

Appendix D — Agent Network Quick Reference

D.1 Membership checklist

Membership checklist: owner mandate valid → Join toggle on → signed Agent Card published → capability tags match plan → direct bond to orchestrator.

D.2 Worker eligibility checklist

Worker eligibility: bonded direct contact → remote Join enabled → card lists required capability → probe succeeds → not blocked tier.

D.3 Team job state reference

Team job states: track orchestrator state machine (Chapter 64); terminal: completed/failed/cancelled; stall triggers rebalance policy.

D.4 Award modes

Award modes: competitive vs single-assign per job settings; competitive waits for bids before award.

D.5 Budget and rebalance policies

Budget/rebalance: mandate `maxCost` and job budget caps; rebalance when worker offline or stall timeout fires.

D.6 Iteration modes

Iteration modes: single-round vs multi-round collaboration; owner approval may pause between rounds.

D.7 Artifact types

Artifact types: text, file, structured, composite—validate hash and sensitivity before merge (Appendix G).

D.8 Troubleshooting decision tree

Decision tree: bond? → card fresh? → capability match? → mandate OK? → audit correlation → then network/probe.

Appendix E — Trust-Level Reference

E.1 Self

Self is the bond tier for your own owner, devices, and locally authorized agents. `evaluatePolicy` returns `{ action: "allow", maxSensitivity: "private" }` —the highest ceiling. Mandates and capability checks still apply; self tier does not bypass inbound guard or semantic firewall.

E.2 Direct

Direct (friends tier) is a mutual bond with explicit trust. Policy allows intents subject to `limitSensitivity(requested, "friends")` —friends-tier knowledge and library reads proceed; trusted/private items require owner approval when requested sensitivity exceeds friends. Chat, tasks, and Agent Network worker discovery among direct bonds are the default collaboration path.

E.3 Referred

Referred is introduction-backed trust—stronger than public, weaker than direct. `knowledge.query` caps at **public** sensitivity; `library.read` caps at **friends** visibility. `feed.notify`, intro intents, `system.ping`, and `bond.request` are allowed at public sensitivity; most other intents return `approval_required` (`referred peer requires approval`).

E.4 Public

Public is stranger/unbonded tier. Allowed: `system.ping`, `social.intro.sync`, public `knowledge.query`, public `library.read`, with rate limits on public knowledge (default 5 queries/minute). `bond.request` and `social.intro.propose` return `challenge` (referral or manual approval). All other intents are `deny` (`public peers cannot use this intent`).

E.5 Blocked

Blocked peers are hard-denied: `evaluatePolicy` returns `{ action: "deny", reason: "peer is blocked" }` for every intent. Use block for abuse or revoked relationships; unblock requires explicit trust restoration. Blocked status is local—your node will not send or accept application traffic regardless of remote reachability.

E.6 Typical permissions

Typical permissions by tier (before mandate and capability gates):

Tier	Chat / tasks	Knowledge query max	Library read max	Agent card fetch
Self	Yes (local)	private	private	N/A
Direct	Yes	friends	friends	Yes (bonded)
Referred	Approval usually	public	friends	After approval

Tier	Chat / tasks	Knowledge query max	Library read max	Agent card fetch
Public	Deny	public (rate-limited)	public	Challenge/deny
Blocked	Deny	Deny	Deny	Deny

Raw file sharing (`allowRawFiles`) always returns `approval_required` regardless of tier.

E.7 Knowledge-sensitivity limits

Knowledge-sensitivity limits use ordered ranks: public < friends < trusted < private. Bond tier sets the ceiling; requesting higher sensitivity yields `approval_required` (`requested sensitivity exceeds <tier>`). Item visibility in handlers is checked against `maxSensitivity` from policy—not bond tier alone.

E.8 Agent Network eligibility

Agent Network eligibility requires direct (or higher) bonds for worker discovery and card fetch; public bonds do not auto-fetch agent cards. Workers must opt in (`capabilityProviderEnabled`) and advertise matching capability tags on signed Agent Cards. Team jobs still enforce mandate bounds, budgets, and per-action approval independently of bond tier.

Appendix F — Task-State Reference

F.1 EnvoyMesh states

EnvoyMesh states: `created → planned → discovering → negotiating → waiting_for_peer | waiting_for_owner → running → partial → completed | failed | cancelled` (Chapter 65).

Typical transitions: `created → planned` (orchestrator accepts the objective); `planned → discovering| negotiating` (worker search or bid exchange); `negotiating → waiting_for_owner` (approval needed); `running → partial` (interim result, more work pending); `partial → completed` (final merge); any non-terminal state → `cancelled` (owner/peer/policy cancel). `completed`, `failed`, and `cancelled` are terminal.

F.2 Valid state transitions

Valid transitions: forward along lifecycle; `partial` may precede terminal success; reject/cancel intents from negotiating or running per mandate.

F.3 Terminal states

Terminal states: `completed`, `failed`, `cancelled` —no further task intents except audit; collect-N mandates may close early on first completion.

F.4 A2A state equivalents

A2A equivalents: twelve internal states map to nine A2A states via `a2a-state-map.ts` (Chapter 73)—document mapping for client UX.

F.5 Cancellation behavior

Cancellation: owner or mandate holder sends `task.cancel`; in-flight work should heartbeat until ack; A2A clients use `tasks/cancel` for tracked ids.

Appendix G — Artifact and Content Mapping

G.1 Text artifacts

Text artifacts — UTF-8 summaries and chat extracts; map to A2A Text Parts; apply semantic firewall before model ingestion.

G.2 File artifacts

File artifacts — Vault-backed paths with optional `?hash=` verification; size and path safety enforced at serve time.

G.3 Structured artifacts

Structured artifacts — JSON with schema hints; validate before automation; map to MCP/A2A Data Parts.

G.4 Composite artifacts

Composite artifacts — Bundles of child artifacts with attribution weights; expand to multiple Parts when bridging.

G.5 MCP content mapping

MCP mapping — Tool results become typed content blocks; preserve correlation IDs for audit stitch.

G.6 A2A Part mapping

A2A Part mapping — Text/Data/File Parts ↔ native artifact types (Chapter 73); hash-check File Part URIs before fetch.

Appendix H — Privacy and Security Checklists

H.1 First-time setup

First-time setup: create owner key → backup `owner-key.pem` → first device cert → set display profile → test ping with low sensitivity.

H.2 Add a contact

Add contact: verify out-of-band identity → scan full QR → complete bond/challenge → start at referred unless mutual direct trust intended.

H.3 Add a device

Add device: owner-signed device certificate → record device ID → pair EnvoyGo or secondary desktop → revoke lost devices promptly.

H.4 Connect an external agent

External agent: generate bridge bearer → compatibility preset → test localhost port → minimal tool call → review audit before broad mandate.

H.5 Join Agent Network

Join Agent Network: direct bonds → enable Join → publish card → refresh workers → trial single-worker task before Team job.

H.6 Start a Team job

Start Team job: mandate bounds set → eligible workers visible → plan approved → budget/deadline realistic → monitor heartbeats.

H.7 Operate a relay

Operate relay: `--advertise-addr` for WAN → bootstrap multiaddr documented → monitor relay-manager audit → no LLM/vault on relay host.

H.8 Respond to a lost device

Lost device: revoke device cert immediately → rotate bridge tokens if exposed → review audit for post-loss traffic → re-pair from backup owner key only on trusted hardware.

Appendix I — Quick Reference Cards

I.1 Pair a contact

Pair contact: Contacts → Invite → show QR → other scans → complete bond flow → confirm direct/referred tier in trust UI.

I.2 Pair EnvoyGo

Pair EnvoyGo: home Settings → Devices → show pair QR → scan in EnvoyGo → verify WebSocket connected on phone.

I.3 Change trust

Change trust: open contact → trust tier → confirm policy implications (Appendix E) → approve if lowering requires re-bond.

I.4 Add knowledge

Add knowledge: Library → Add → pick vault-safe path → set sensitivity label → re-index if search misses.

I.5 Approve an action

Approve action: Desktop Approvals queue → read mandate context → Allow/Deny → task resumes or cancels per policy.

I.6 Connect an external agent

Connect external agent: Settings → External agent → preset → paste bearer to agent config → test message round-trip.

I.7 Join Agent Network

Join Agent Network: Settings → Agent Network → Join → verify card published → Refresh workers on peer.

I.8 Start a Team job

Start Team job: Agent Network → New job → select workers → set mandate → launch → watch state in job panel.

I.9 Cancel a task

Cancel task: open task → Cancel → confirm mandate allows cancel → audit records terminal cancelled state.

I.10 Revoke a device

Revoke device: Settings → Devices → Revoke → confirm cert revoked → remove pairing on device app.

I.11 Collect diagnostics

Collect diagnostics: Appendix K bundle checklist → redact secrets → attach correlation IDs → CLI connectivity-status.

Appendix J — Status and Roadmap Boundaries

J.1 Available features

Available (0.2.2) — intended for current use on supported platforms:

- Signed messaging, groups, audio messages, voice calls, file/profile sharing (Chapters 11–14)
- Personal AI via EnvoyAI/OpenClaw and external-agent bridges (Part VI)
- Vault, Library, knowledge query, Browser/`envoy://` publishing (Part V)
- Agent Network, Team jobs, mandates, approvals (Part VII)
- Terminals, relays, MCP tool bridge, A2A agent card + JSON-RPC tasks (Parts VIII–IX, X)
- Desktop Social (macOS/Windows) and EnvoyGo thin client (iOS/Android) per Chapter 9

Confirm exact packaging in release notes before production rollout.

J.2 Beta and experimental features

Beta / Experimental — implemented but still receiving validation; interfaces may change:

- Experimental toggles in Settings (§80.11)—enable only on non-production profiles
- MCP stdio live servers and extended interop smoke paths (Phase 48 docs)
- A2A home-tunnel forwarding and artifact mapping edge cases (Part IX)
- IPFS/Helia sidecars when bundled—optional content experiments, not core chat
- Multi-relay coordination under load—works but operator tuning may be required

Report issues with the **Beta** or **Experimental** label and redacted audit excerpts.

J.3 Platform-specific features

Platform-specific boundaries:

- **macOS desktop** — fuller OpenClaw extension bundle; Tauri notarization path (Chapter 9.2–9.4)
- **Windows desktop** — slimmer extension set; user AppData profile paths (9.3, 9.5)
- **EnvoyGo iOS/Android** — thin client only: chat, calls, terminals, Browser mirror, read-only Team status; no local vault, agent runtime, or MCP/A2A server (9.1, 9.9)
- **Home-node-only** — mesh identity, Vault indexing, Team orchestration, bridge endpoints, full Settings (9.8)
- **Operator** — relay binary, fleet manifests, bootstrap tuning (Part X, Appendix K)

Do not infer desktop availability from mobile mirrors or vice versa.

J.4 Planned video calling

Planned. Voice calling is available; video calling remains architecturally anticipated but is not a current user feature.

J.5 Planned broad or anonymous discovery

Planned boundary. Contact- and capability-scoped discovery exists, but open anonymous worker recruitment and marketplace behavior are not current Agent Network features.

J.6 Parked: EnvoyGo as a full mesh node (EnvoyGo remains a thin client)

Parked. EnvoyGo remains a home-paired thin client. Running it as an independent full mesh node has no committed release.

J.7 Parked global reputation

Parked. Local feedback and reputation signals exist, but a federated global reputation ledger is intentionally deferred.

J.8 Parked multi-hop commerce

Parked. Multi-hop commerce, payment, and receipt workflows are outside the current collaboration product.

J.9 Deferred Filecoin persistence

Deferred. Helia and Kubo IPFS paths are available, but Filecoin-based long-term persistence is not part of the current release.

J.10 Deferred hierarchical relay graph

Deferred. Multi-relay sibling coordination exists; a full hierarchical relay graph is not complete.

J.11 Future MCP resources and OAuth

Future. MCP currently focuses on tools and Bearer-authenticated bridges. Resources, prompts, and OAuth 2.1 remain future interoperability work.

J.12 Other roadmap references

Other roadmap references (documented direction, not current general features):

- Video calling (J.4)—voice only today
- Broad/anonymous worker recruitment (J.5)
- EnvoyGo as full mesh node (J.6—parked; thin client remains product path)
- Global reputation ledger (J.7), multi-hop commerce (J.8)
- Filecoin persistence (J.9), full hierarchical relay graph (J.10)
- MCP resources/prompts/OAuth 2.1 (J.11)

See `docs/implementation-plan.md` for phase numbers; design docs alone do not imply shipment.

Appendix K — Support Reference

K.1 Application data locations

EnvoyMesh keeps state outside the application install directory. **Source / developer runs** default to `./data/default` for the profile (identity, trust, tasks, approvals, bridge config) and `./shared_vault/` for Library content. **Packaged desktop builds** use OS-specific user data paths (for example `~/.local/share/envoymesh/` on Linux, `%AppData%` or `%USERPROFILE%\.envoymesh\` on Windows, and the Tauri app data area on macOS—confirm the exact path in release notes for your installer). The vault may appear as `shared_vault/` beside the profile or under a `vault/` subdirectory depending on platform packaging. Always back up the whole profile directory **and** the vault together before migration. Include only the relevant subtree in support bundles; remove `owner-key*`, device keys, `bridge-config.json` secrets, model API keys, and unrelated personal files.

K.2 Default ports

Common defaults include external-agent bridge `3031`, OpenClaw Gateway `18789`, relay HTTP `15432`, and HomeClaw/Hermes/OpenHuman message ports `8010` / `8020` / `8021`. Confirm configuration because operators may override every value.

K.3 Public endpoints

Public A2A routes include `/.well-known/agent-card.json` and `/.well-known/a2a/jsonrpc` when the relay bridge is enabled. Keep home-only bridge and administrative endpoints private.

K.4 Log locations

Primary operational history is append-only **JSONL** in the profile directory, not a separate syslog tree. Key files include `audit-events.jsonl` (allow/deny outcomes and connectivity traces), `task-journal.jsonl`, `approval-queue.jsonl`, `discovery-events.jsonl`, and `share-events.jsonl`, plus JSON state such as `trust-records.json` and `peer-directory.json`. Relay operators also generate relay-manager snapshot rows inside relay profile audit logs. Console output from `npm run node:dev` or the desktop wrapper is supplementary—prefer redacted audit excerpts with correlation IDs when opening support tickets. Strip bearer tokens, envelope payloads, and key material before sharing any log file.

K.5 Diagnostic commands

From the repository root (adjust `--profile` to your absolute profile path):

```
npm run typecheck                    # TypeScript build check
npm test                             # Unit tests
npm run test:orchestrator -- dev     # Fast dev loop (~35s, no E2E)
npm run test:orchestrator -- full    # Full gate incl. libp2p E2E + smoke (~10 min)
npm run node:dev -- --profile ./data/default
npm run cli -w @envoymesh/node -- --help
npm run cli -w @envoymesh/node -- connectivity-status --profile ./data/default --rich
npm run cli -w @envoymesh/node -- relay-status --profile ./data/default
npm run cli -w @envoymesh/node -- audit --profile ./data/default --limit 40 --include-p2p-trace
```

See `QuickStart.md` for setup scripts, cross-network relay walkthroughs, and the end-to-end verification checklist. Global `envoymesh doctor` is available after `npm i -g .` from the repo root.

K.6 Common error themes

EnvoyMesh surfaces failures by **theme** in audit summaries, CLI output, and bridge responses rather than a single printed error-code handbook. Common patterns:

- `auth-required` — bearer or session authorization failed (missing/invalid token, or trust tier too weak for the requested A2A/MCP/task action). Fix pairing tokens, bridge secrets, or bond level before retrying.
- **Bond deny** — `evaluatePolicy` returned deny (for example `peer is blocked`, `public peers cannot use this intent`, expired mandate, disallowed action, or sensitivity above mandate). Inspect trust tier and mandate bounds; raising trust requires explicit human approval, not a connectivity tweak.
- **Schema / guard reject** — inbound guard rejected malformed, oversized, replayed, or unsigned envelopes (`malformed or unsigned envelope`, `envelope exceeds maximum size`, `replayed message`). Usually indicates version skew, corrupt payloads, or attack traffic—not a relay routing issue.

A2A JSON-RPC may also return `-32001` with an `auth-required:` message when Authorization headers are missing on relay-proxied task endpoints. Capture the audit row's `summary` and `correlationId` instead of inventing numeric codes when filing issues.

K.7 Support and community links

In-repo documentation: start with `QuickStart.md`, `README.md`, `docs/implementation-plan.md`, and the scenario/design docs referenced from QuickStart (for example `docs/UserStory.md`, `docs/scenarios.md`).

Source repository: https://github.com/allenpeng0705/EnvoyMesh — use GitHub Issues for bug reports and feature discussion when that repository is your distribution channel. There is no separate commercial support portal documented in this release; enterprise operators should maintain internal runbooks.

Before opening an issue: reproduce on a current build, note platform (macOS/Windows/EnvoyGo), profile path, feature status label (**Beta** / **Experimental**), and redacted `audit-events.jsonl` excerpts with correlation IDs. Placeholder community chat/forum links are not bundled with 0.2.2—watch release notes for official channels as they are announced.

Information-architecture proposal for website editors. The two lists below are not end-user chapters. They are a suggested navigation skeleton for the public website, derived from the guidebook structure. Editors should treat them as a starting point and adapt to the actual site information architecture.

Proposed Primary Website Navigation

- **Product**
- **Agent Network**
- **External Agents**
- **Use Cases**
- **How It Works**
- **Security**
- **Downloads**
- **Guide**
- **Community**

Proposed Guide Navigation

- Getting Started
- Conversations and Sharing
- Personal AI
- Knowledge and Library
- External Agents
- Agent Network and Team Jobs
- Tasks and Artifacts
- MCP and A2A
- Networking and Relays
- Privacy and Security
- Settings and Data
- Troubleshooting
- FAQ